Logistic, Poisson and Count Regression with Mixed Models

A plain-English, code-first guide in R

Binary outcomes ▪ counts ▪ rates and offsets ▪ overdispersion ▪ quasi-likelihood ▪ negative binomial ▪ zero-inflation and hurdles ▪ model checking ▪ likelihood-ratio, Wald and score tests ▪ random effects, GLMMs and GEEs

Every code block in this book was executed to produce the output and figures shown.
Built with R version 4.3.3 (2024-02-29) using stats, MASS, lme4, glmmTMB, pscl, AER, sandwich and lmtest.

Contents

How to use this book **1**

What this book is . . . 1

Who it is for . . . 1

How the chapters fit together . . . 2

Setting up R . . . 2

The datasets used in this book . . . 2

A note on reproducibility . . . 3

Conventions . . . 3

1 The big picture: choosing a model **4**

1.1 Why ordinary regression is not enough . . . 4

1.2 The decision tree . . . 5

1.3 The same table, in words . . . 7

1.4 What the distributions actually look like . . . 7

1.5 Three questions to ask of any dataset . . . 8

2 The anatomy of a GLM **10**

2.1 The three moving parts . . . 10

2.2 The consequence: effects multiply . . . 11

2.3 How R fits these models . . . 12

2.4 Getting things out of a fitted model . . . 13

3 Logistic regression I: the basics **16**

3.1 Probability, odds, and log-odds . . . 16

3.2 The model . . . 17

3.3 A worked example: low birth weight . . . 17

3.4 Predicted probabilities . . . 20

3.5 Categorical predictors and reference levels . . . 22

3.6 Interactions on the logit scale . . . 23

4 Logistic regression II: checking and extending **25**

4.1 Turning probabilities into decisions . . . 25

4.2 The ROC curve and AUC . . . 26

4.3 Calibration . . . 27

4.4 Residuals and influence . . . 29

4.5 Grouped (binomial) data . . . 30

4.6 Overdispersion in binomial models . . . 31
4.7 Complete separation . . . 32
4.8 Other links for binary data . . . 33

5 Poisson regression I: modelling counts 35
5.1 Where the Poisson comes from . . . 35
5.2 The model . . . 35
5.3 A first fit . . . 35
5.4 Interpreting a log-link coefficient . . . 37
5.5 Why not just log-transform and use `lm`? . . . 38
5.6 Counts with a fixed upper limit . . . 39
5.7 Checking a Poisson fit quickly . . . 39

6 Rates, exposure and offsets 41
6.1 The problem . . . 41
6.2 The mathematics, which is short . . . 41
6.3 A simulated example where it matters . . . 41
6.4 The alternative syntax . . . 43
6.5 Predicting from a model with an offset . . . 43
6.6 When exposure is not time . . . 43
6.7 Should the coefficient really be 1? . . . 44
6.8 A real-data example . . . 44

7 Overdispersion 46
7.1 What it is . . . 46
7.2 Why it happens . . . 46
7.3 Why it matters: a simulation . . . 47
7.4 Detecting it . . . 48
7.5 Rule out the boring explanation first . . . 51
7.6 Your options once overdispersion is real . . . 52
7.7 Underdispersion . . . 52

8 Quasi-Poisson and robust standard errors 54
8.1 Quasi-Poisson . . . 54
8.2 Robust (sandwich) standard errors . . . 57
8.3 Quasi-Poisson versus negative binomial . . . 58

9 Negative binomial regression 61
9.1 Where it comes from . . . 61
9.2 Fitting it . . . 62
9.3 Poisson versus negative binomial, side by side . . . 62

9.4 Does it fit? Compare the predicted distributions . . . 64
9.5 The two flavours: NB2 and NB1 . . . 65
9.6 Reporting a negative binomial model . . . 66
9.7 Things that go wrong . . . 66

10 Too many zeros: zero-inflated and hurdle models 68
10.1 Two kinds of zero . . . 68
10.2 The zero-inflated model . . . 68
10.3 A worked example . . . 69
10.4 Hurdle models . . . 71
10.5 Which model reproduces the data best? . . . 72
10.6 The Vuong test, and why to be careful with it . . . 74
10.7 A case with real zero-inflation . . . 74

11 The rest of the count family 76
11.1 Truncated counts . . . 76
11.2 Underdispersion and the Conway–Maxwell–Poisson . . . 77
11.3 Beta-binomial: overdispersion for proportions . . . 77
11.4 Ordinal and multinomial outcomes . . . 78
11.5 When the relationship is not a straight line . . . 78
11.6 Continuous non-negative outcomes . . . 79

12 Checking a count model 81
12.1 The four kinds of residual . . . 81
12.2 Randomised quantile residuals: the tool that works . . . 82
12.3 Simulation-based checks you can always do . . . 83
12.4 Is the variance function right? . . . 85
12.5 Influence and outliers . . . 85
12.6 A checklist you can run every time . . . 87

13 Tests, comparisons and model selection 88
13.1 The three classical tests . . . 88
13.2 Comparing nested models . . . 89
13.3 Comparing non-nested models: AIC and BIC . . . 90
13.4 Akaike weights . . . 90
13.5 Tests specific to count models . . . 91
13.6 How much does the model explain? . . . 92
13.7 Cross-validation . . . 92
13.8 A note on stepwise selection . . . 93

14 Mixed models I: why and how 94

14.1 The problem: not all observations are independent . . . 94
14.2 How bad is it? A simulation . . . 94
14.3 Fixed effects versus random effects . . . 95
14.4 The random-intercept model . . . 95
14.5 Random slopes . . . 97
14.6 Shrinkage: the best feature of mixed models . . . 98
14.7 Specifying random effects in R . . . 99
14.8 REML or ML? . . . 100
14.9 Where are the p-values? . . . 100

15 Mixed models II: GLMMs for binary and count data 102
15.1 Binomial GLMM . . . 102
15.2 Poisson GLMM . . . 104
15.3 Observation-level random effects . . . 106
15.4 Negative binomial GLMM . . . 106
15.5 Zero-inflated GLMM . . . 107
15.6 Predicting from a GLMM . . . 109
15.7 Simulating from a GLMM to check it . . . 109
15.8 A summary of the GLMM toolkit . . . 110

16 Conditional vs marginal effects, and GEE 111
16.1 Two different questions . . . 111
16.2 Seeing the attenuation . . . 111
16.3 Generalised estimating equations (GEE) . . . 113
16.4 GLMM or GEE? . . . 115
16.5 Getting marginal effects out of a GLMM . . . 115

17 When the model will not fit 117
17.1 “boundary (singular) fit” . . . 117
17.2 “Model failed to converge with max|grad| . . . ” . . . 118
17.3 “fixed-effect model matrix is rank deficient” . . . 120
17.4 glmmTMB warnings . . . 120
17.5 A triage flowchart . . . 121
17.6 Warnings you can safely ignore . . . 121

18 A complete worked analysis 123
18.1 The scenario . . . 123
18.2 Step 1: look at the data . . . 124
18.3 Step 2: the naive model (what not to do) . . . 125
18.4 Step 3: add the offset . . . 126
18.5 Step 4: check dispersion . . . 126

18.6 Step 5: negative binomial . . . 127
18.7 Step 6: add the random effect . . . 128
18.8 Step 7: compare everything . . . 128
18.9 Step 8: check the final model . . . 130
18.10 Step 9: report it . . . 131
18.11 The workflow as a checklist . . . 131

19 Cheat sheets **132**
19.1 Which model do I fit? . . . 132
19.2 How do I interpret this coefficient? . . . 132
19.3 The essential diagnostic code . . . 132
19.4 Model comparison quick reference . . . 133
19.5 Ten mistakes to avoid . . . 133
19.6 Glossary . . . 134
19.7 Where to go next . . . 135

How to use this book

What this book is

This is a working guide to the family of regression models you reach for when your outcome is *not* a nice continuous measurement: yes/no outcomes, counts of events, rates per unit of exposure, and any of those measured repeatedly on the same people, animals, schools or clinics.

Everything is explained twice. Once in plain words, with a picture or an analogy. Once in R code that you can copy, run, and change. Every number and every plot in this book was produced by actually running the code shown next to it, so what you see on the page is what you will see in your console.

The one-sentence summary of the whole book

> Ordinary linear regression assumes your outcome is a continuous number that can wander anywhere and whose wobble is the same everywhere; when the outcome is a yes/no or a count, both of those assumptions break, and this book is about the family of models that fix them.

Who it is for

You should be comfortable with:

- ordinary linear regression (`lm`) — what a slope means, what a p-value is, what a residual is;
- basic R — reading in data, `data.frame`, `$`, `summary`.

You do *not* need calculus, matrix algebra, or any prior exposure to generalised linear models. Where mathematics appears it is there to pin down an idea that is already explained in words, and you can skip it on a first read without losing the thread.

How the chapters fit together

Ch	Topic	What you get out of it
1	The big picture	A decision tree: which model for which outcome
2	GLM anatomy	The three moving parts every model here shares
3–4	Logistic regression	Yes/no outcomes, odds ratios, ROC, calibration
5–6	Poisson regression	Counts, rate ratios, offsets and exposure
7	Overdispersion	The single most common problem with count models
8	Quasi-likelihood & robust SEs	The quickest repair for overdispersion
9	Negative binomial	The proper likelihood-based repair
10	Zero-inflation & hurdles	When there are far too many zeros
11	Other count models	Truncation, underdispersion, ordinal, GAMs
12	Model checking	Residuals that actually work for counts
13	Tests	LRT, Wald, score, AIC/BIC, Vuong
14–15	Mixed models	Repeated measures and clustering; GLMMs
16	Marginal vs. conditional	Why GLMM and GEE give different numbers
17	Convergence problems	What the scary warnings mean and how to fix them
18	A complete worked analysis	Everything applied end-to-end
19	Cheat sheets	Tables to keep beside you while you work

If you are in a hurry and you have count data, read Chapters 5, 7, 9 and 12. That is the shortest path to a defensible analysis.

Setting up R

Install the packages once:

```
install.packages(c("MASS", "lme4", "glmmTMB", "pscl", "AER",
                   "lmtest", "sandwich", "car", "emmeans", "ggplot2"))
```

and load the ones we use constantly:

```
library(MASS)       # glm.nb(), and several teaching datasets
library(lme4)       # glmer(), lmer() for mixed models
library(glmmTMB)    # negative binomial / zero-inflated mixed models
library(pscl)       # zeroinfl(), hurdle()
library(AER)        # dispersiontest()
library(lmtest)     # coeftest(), lrtest()
library(sandwich)   # robust standard errors
```

What each package is for, in one line each

MASS adds the negative binomial. lme4 adds random effects. glmmTMB does both at once and adds zero-inflation. pscl does zero-inflated and hurdle models without random effects. AER gives you a formal overdispersion test. lmtest and sandwich give you likelihood-ratio tests and robust standard errors.

The datasets used in this book

All of these ship with R or with one of the packages above, so you can run every example without downloading anything.

Dataset	Package	Outcome variable	Used for
`birthwt`	MASS	low birth weight (0/1)	logistic regression
`cbpp`	lme4	cases out of herd size	binomial GLMM
`quine`	MASS	days absent (count)	overdispersion, NB
`warpbreaks`	datasets	breaks (count)	Poisson basics
`bioChemists`	pscl	articles published (count)	zero-inflation, hurdles
`RecreationDemand`	AER	boating trips (count)	extreme zero-inflation
`epil`	MASS	seizures (count, repeated)	count mixed models
`grouseticks`	lme4	ticks on chicks (count, nested)	Poisson GLMM + OLRE
`sleepstudy`	lme4	reaction time (repeated)	linear mixed models
`InsectSprays`	datasets	insects counted (count)	simple Poisson

A note on reproducibility

Anywhere this book simulates data, a seed is set first, so you will get exactly the numbers printed here:

```
set.seed(2024)
rpois(8, lambda = 3)

#> [1] 5 2 4 4 3 4 2 2
```

Conventions

- Code appears in shaded boxes; output lines begin with `#>`.
- Blue boxes translate an idea into plain English.
- Green boxes hold the key idea you should remember.
- Amber boxes hold rules of thumb — useful, but not laws.
- Red boxes are mistakes people really make. Read those twice.

Try this

> Do not just read the code. Change a number in it, re-run it, and see what moves. Almost everything in this book was learned by somebody breaking a model and then working out why it broke.

CHAPTER 1

The big picture: choosing a model

1.1 Why ordinary regression is not enough

Ordinary linear regression says:

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2).$$

Read that carefully and you will spot three promises hidden in it:

1. **The outcome can be any number.** Positive, negative, fractional.
2. **The spread is constant.** The same σ^2 at every value of x.
3. **The average is a straight line in** x, with no ceiling and no floor.

Now think about a yes/no outcome, coded 0 and 1. Promise 1 is already broken: the outcome can only be 0 or 1, but a straight line will happily predict 1.4 or -0.2. Promise 2 is broken too: for a 0/1 outcome the variance is $p(1-p)$, which is largest at $p = 0.5$ and shrinks to zero at the edges — so the spread *must* change with the mean.

Think about a count: number of hospital visits, number of insect bites, number of accidents. Counts cannot be negative and cannot be fractional. And counts behave in a very particular way: rare events are tightly bunched near zero, common events spread out widely. The variance grows with the mean. Promise 2 breaks again.

Let us see all of this rather than argue about it.

```
set.seed(11)
n  <- 200
x  <- runif(n, -3, 3)
p  <- 1 / (1 + exp(-(0.2 + 1.6 * x)))          # true probability curve
y  <- rbinom(n, 1, p)                          # the 0/1 outcome we observe
cnt <- rpois(n, lambda = exp(1.1 + 0.55 * x))  # a count outcome

par(mfrow = c(1, 2)); par_book()

## ---- Panel 1: linear model on a 0/1 outcome -------------------------------
plot(x, y, pch = 21, bg = adjustcolor(PAL["primary"], 0.35), col = "white",
     cex = 1.1, ylim = c(-0.35, 1.35), xlab = "x", ylab = "y (0 or 1)",
     main = "A straight line on a yes/no outcome")
grid_book()
rect(-4, 1, 4, 1.5, col = adjustcolor(PAL["red"], 0.10), border = NA)
rect(-4, -0.5, 4, 0, col = adjustcolor(PAL["red"], 0.10), border = NA)
abline(h = c(0, 1), col = PAL["red"], lty = 3)
abline(lm(y ~ x), col = PAL["red"], lwd = 2.4)
curve(1/(1 + exp(-(0.2 + 1.6*x))), add = TRUE, col = PAL["green"], lwd = 2.4)
legend("topleft", bty = "n", cex = 0.75, lwd = 2.4,
       col = c(PAL["red"], PAL["green"]),
       legend = c("lm(): leaves [0,1]", "logistic: stays inside"))

## ---- Panel 2: constant variance fails for counts --------------------------
m_lm <- lm(cnt ~ x)
plot(fitted(m_lm), resid(m_lm), pch = 21, bg = adjustcolor(PAL["accent"], 0.45),
```

```
     col = "white", cex = 1.1, xlab = "fitted value", ylab = "residual",
     main = "Residuals from lm() on counts")
grid_book(); abline(h = 0, col = PAL["grey"], lwd = 1.5)
lines(lowess(fitted(m_lm), abs(resid(m_lm)))$x,
      lowess(fitted(m_lm), abs(resid(m_lm)))$y, col = PAL["red"], lwd = 2.2)
lines(lowess(fitted(m_lm), -abs(resid(m_lm)))$x,
      lowess(fitted(m_lm), -abs(resid(m_lm)))$y, col = PAL["red"], lwd = 2.2)
legend("topleft", bty = "n", cex = 0.75,
       legend = "spread widens with the mean", text.col = PAL["red"])
```

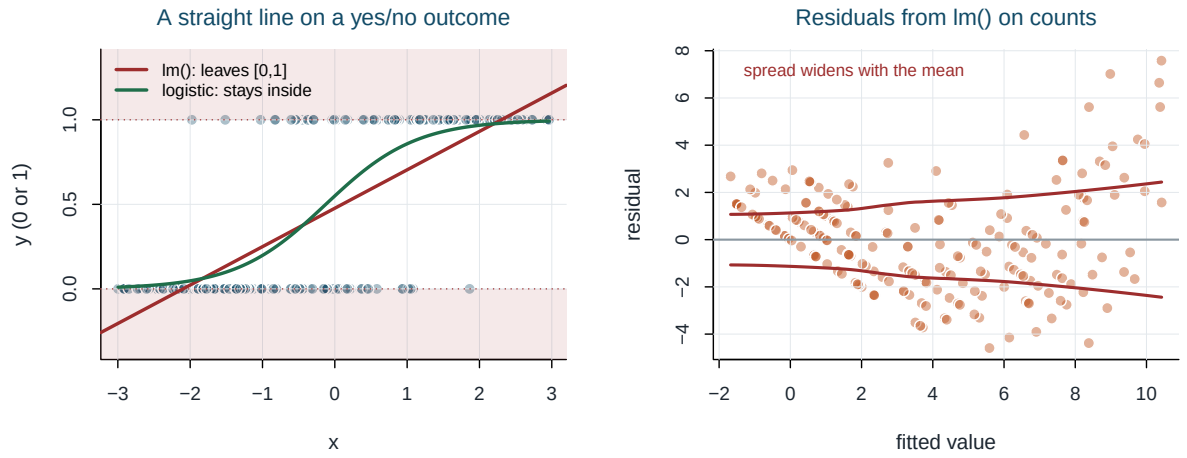


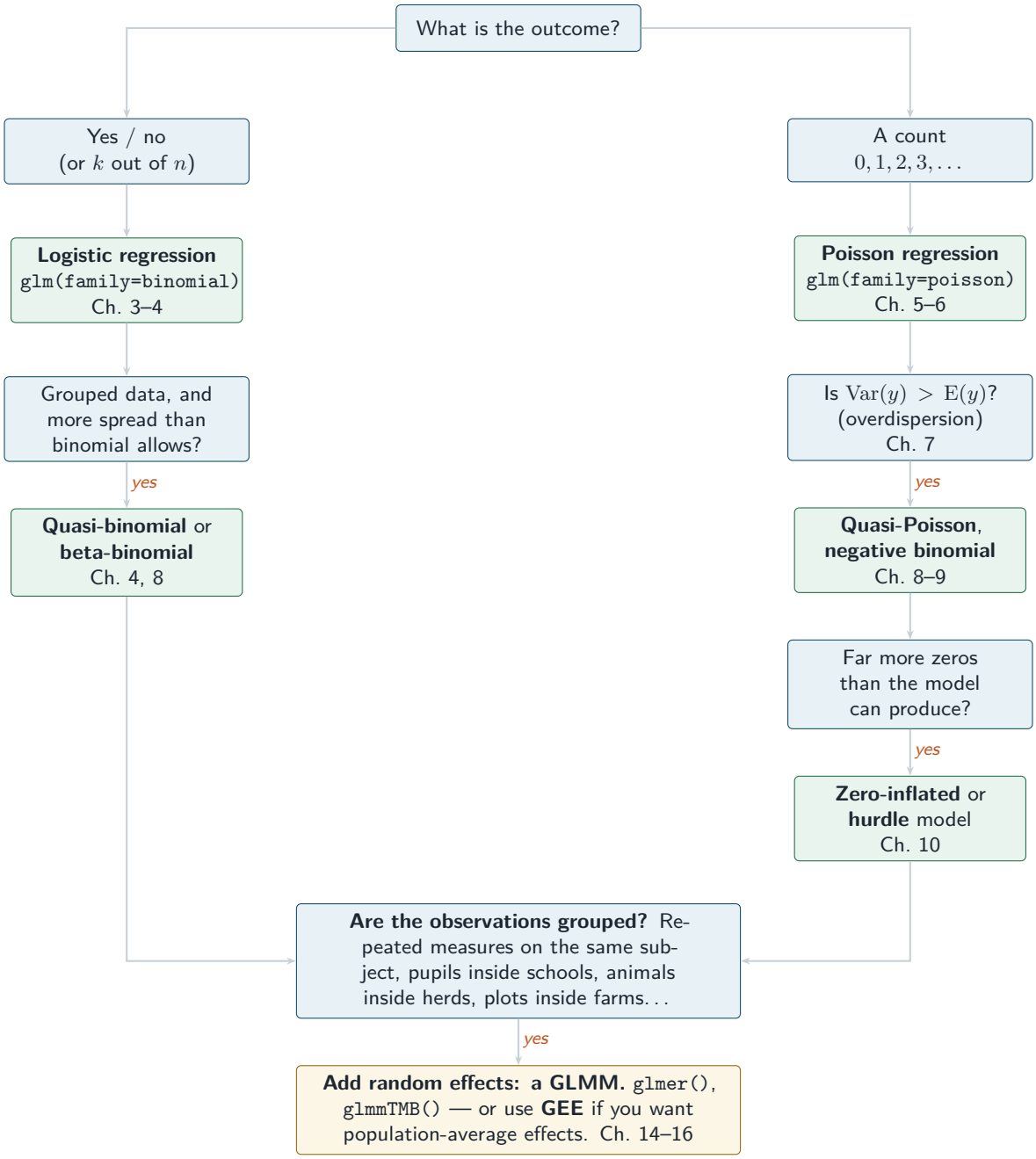
Figure 1.1: Left: a straight line fitted to a 0/1 outcome predicts impossible probabilities (shaded regions). Right: for count data the spread of the residuals fans out as the fitted value grows, so a single constant variance cannot describe it.

Key idea

> The problem is never that the relationship is "non-linear". The problem is that the outcome lives on a restricted scale and its variability is tied to its mean. Generalised linear models fix both at once.

1.2 The decision tree

Here is the map for the whole book. Start at the top with the question *what kind of thing is my outcome?*


What is the outcome?
Yes / no (or k out of n)
A count $0, 1, 2, 3, \ldots$
Logistic regression
glm(family=binomial)
Ch. 3–4
Poisson regression
glm(family=poisson)
Ch. 5–6
Grouped data, and more spread than binomial allows?
Is $\mathrm{Var}(y) > \mathrm{E}(y)$? (overdispersion)
Ch. 7
yes
yes
Quasi-binomial or beta-binomial
Ch. 4, 8
Quasi-Poisson, negative binomial
Ch. 8–9
Far more zeros than the model can produce?
yes
Zero-inflated or hurdle model
Ch. 10
Are the observations grouped? Repeated measures on the same subject, pupils inside schools, animals inside herds, plots inside farms...
yes
Add random effects: a GLMM. glmer(), glmmTMB() — or use GEE if you want population-average effects. Ch. 14–16

1.3 The same table, in words

Your outcome	Model	R call
Continuous, symmetric	Linear regression	`lm(y ~ x)`
Yes / no	Logistic regression	`glm(y ~ x, family=binomial)`
k successes out of n	Binomial regression	`glm(cbind(k, n-k) ~ x, family=binomial)`
Proportion, overdispersed	Quasi-binomial	`glm(..., family=quasibinomial)`
Count, variance $\approx$ mean	Poisson	`glm(y ~ x, family=poisson)`
Count per unit time/space	Poisson with offset	`glm(y ~ x + offset(log(t)), family=poisson)`
Count, variance $>$ mean	Quasi-Poisson	`glm(..., family=quasipoisson)`
Count, variance $>$ mean	Negative binomial	`MASS::glm.nb(y ~ x)`
Count with excess zeros	Zero-inflated	`pscl::zeroinfl(y ~ x | z)`
Count, zeros are a separate process	Hurdle	`pscl::hurdle(y ~ x | z)`
Any of the above, grouped data	GLMM	`lme4::glmer(y ~ x + (1|group), ...)`
Any of the above, want marginal effects	GEE	`geepack::geeglm(...)`

1.4 What the distributions actually look like

Before fitting anything, it helps to know the shapes you are choosing between.

```
par(mfrow = c(2, 3)); par_book()
bar <- function(k, prob, main, col) {
  barplot(prob, names.arg = k, col = adjustcolor(col, 0.75), border = "white",
          main = main, xlab = "y", ylab = "probability", cex.names = 0.75)
}
k <- 0:14
bar(k, dpois(k, 1),  "Poisson, mean = 1",  PAL["primary"])
bar(k, dpois(k, 4),  "Poisson, mean = 4",  PAL["primary"])
bar(k, dpois(k, 9),  "Poisson, mean = 9",  PAL["primary"])
bar(k, dbinom(k, 14, 0.3), "Binomial, n = 14, p = 0.3", PAL["green"])
bar(k, dnbinom(k, mu = 4, size = 10),  "Neg. binomial, mean = 4, theta = 10", PAL["accent"])
bar(k, dnbinom(k, mu = 4, size = 0.5), "Neg. binomial, mean = 4, theta = 0.5", PAL["accent"])
```

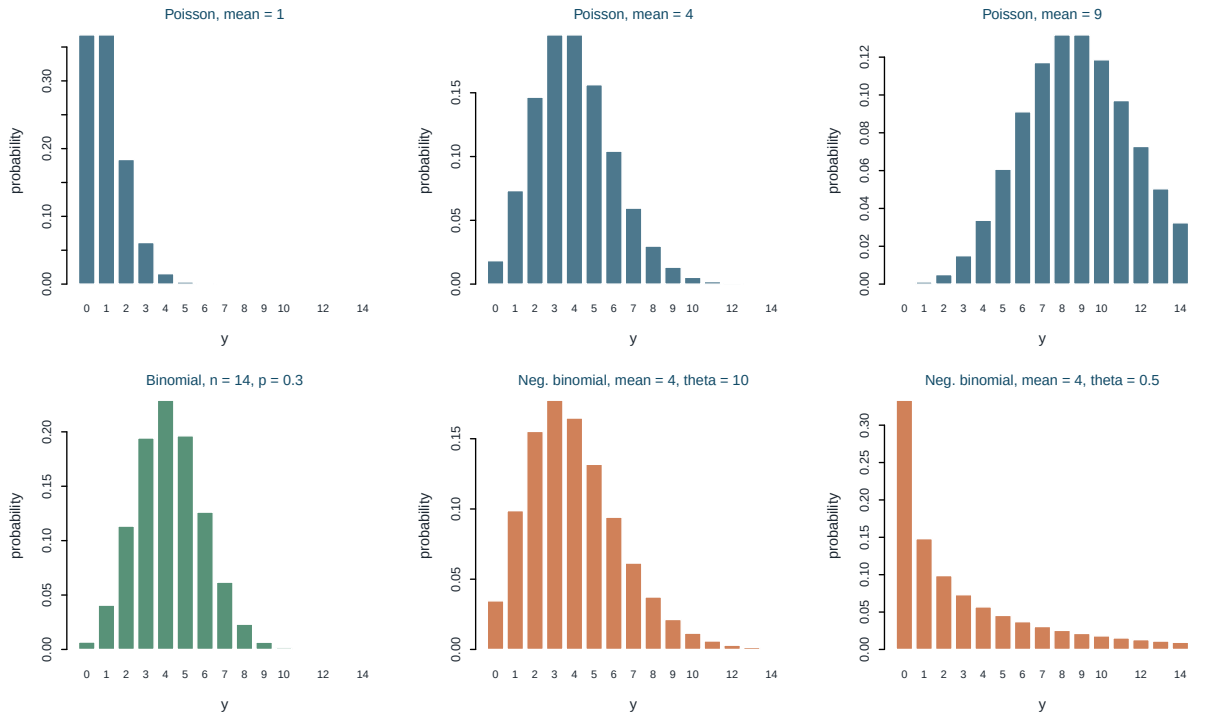


Figure 1.2: The four distributions this book uses. Notice that the Poisson is fully determined by its mean, while the negative binomial has a second knob that lets it be much more spread out at the same mean.

Three things to take away from that figure:

- A Poisson with a small mean is squashed against zero and skewed right. With a large mean it becomes almost symmetric — which is why for large counts, ordinary regression sometimes works "well enough".
- The Poisson has *one* parameter. Once you say the mean is 4, the variance is also 4. You have no freedom left.
- The negative binomial has a second parameter, usually called θ (`theta`) or `size`. Large θ makes it look like a Poisson. Small θ makes it long-tailed and full of zeros *at the same mean*. That extra knob is the whole reason it exists.

Mean and variance, side by side

Normal: mean and variance are unrelated — you can have mean 5 with variance 0.01 or 900.
Poisson: variance = mean, no choice.
Negative binomial: variance = mean + mean$^2/\theta$, always bigger than the mean.
Binomial: variance = $np(1-p)$, biggest at $p = 0.5$.

1.5 Three questions to ask of any dataset

Before you write a single `glm` call, ask:

1. **What can this outcome physically be?** Bounded 0/1? Non-negative integers? Non-negative integers with an upper limit? The answer picks your family.
2. **Was every observation given the same opportunity?** If some people were followed for

two years and some for two months, you need an offset (Chapter 6). Forgetting this is the most common serious error in count regression.

3. **Are any two rows more alike than two random rows?** Two visits by the same patient, two pupils in the same class, two chicks in the same brood. If yes, you need random effects (Chapters 14–15) or robust standard errors, or your p-values will be far too small.

The most expensive mistake in this whole book

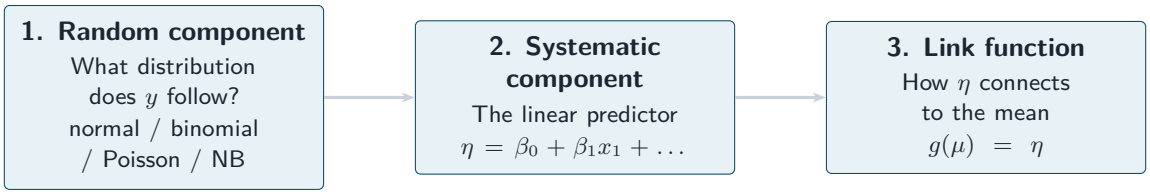
Ignoring clustering does not usually change your estimates very much. It changes your *standard errors* — often by a factor of two or three. That turns $p = 0.20$ into $p = 0.02$ and produces a confident published conclusion that is simply not supported by the data.

CHAPTER 2

The anatomy of a GLM

Every model in this book — logistic, Poisson, negative binomial, and their mixed versions — is built from the same three parts. Learn the three parts once and you never have to memorise anything again.

2.1 The three moving parts



2.1.1 Part 1: the random component

This is your answer to "what kind of thing is my outcome?". It fixes the relationship between the mean and the variance, which is really the thing that matters:

Family	Outcome	Mean	Variance
Gaussian	any real number	μ	σ^2 (free)
Binomial	$0/1$, or k of n	$\mu = np$	$np(1-p)$
Poisson	$0, 1, 2, \ldots$	μ	μ
Negative binomial	$0, 1, 2, \ldots$	μ	$\mu + \mu^2/\theta$
Gamma	positive continuous	μ	μ^2/ν

2.1.2 Part 2: the systematic component

This is the familiar part — a weighted sum of your predictors:

$$\eta_i = \beta_0 + \beta_1 x_{1i} + \beta_2 x_{2i} + \cdots + \beta_p x_{pi}.$$

It is called the *linear predictor* and it is written η ("eta"). Note that η can be any number at all, from $-\infty$ to $+\infty$. That is exactly the problem the third part solves.

2.1.3 Part 3: the link function

A probability must live in $[0, 1]$. A count mean must be positive. But η can be anything. The link function g is the translator:

$$g(\mu_i) = \eta_i \qquad \Longleftrightarrow \qquad \mu_i = g^{-1}(\eta_i).$$

Link	$g(\mu)$	Inverse $g^{-1}(\eta)$	Used with
identity	μ	η	Gaussian
logit	$\log \frac{\mu}{1-\mu}$	$\frac{e^\eta}{1+e^\eta}$	binomial
probit	$\Phi^{-1}(\mu)$	$\Phi(\eta)$	binomial
log	$\log \mu$	e^η	Poisson, NB, gamma

Here are the two links that matter most, drawn:

```
par(mfrow = c(1, 2)); par_book()
eta <- seq(-6, 6, length.out = 400)

plot(eta, plogis(eta), type = "l", lwd = 3, col = PAL["primary"],
     xlab = expression(eta ~ " (the linear predictor)"),
     ylab = expression(mu ~ " = probability"),
     main = "Inverse logit: any number -> (0, 1)")
grid_book(); abline(h = c(0, 1), lty = 3, col = PAL["grey"])
abline(h = 0.5, v = 0, lty = 3, col = PAL["accent"])

plot(eta, exp(eta), type = "l", lwd = 3, col = PAL["accent"],
     xlab = expression(eta ~ " (the linear predictor)"),
     ylab = expression(mu ~ " = expected count"),
     main = "Inverse log: any number -> positive")
grid_book(); abline(h = 0, lty = 3, col = PAL["grey"])
```

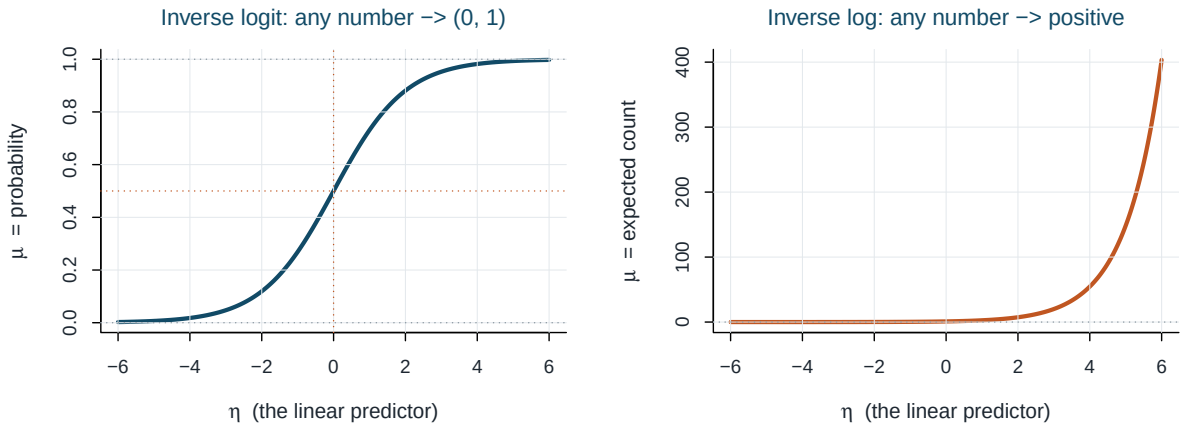


Figure 2.1: The two workhorse inverse links. The logistic curve squashes any number into (0,1). The exponential curve turns any number into something positive.

Why we bother with links

Because we want to keep the convenient "add up the effects" structure of linear regression, but we need the answer to come out on a legal scale. So we do the adding-up on an unrestricted scale (η), and then squash the result into the legal range at the very end.

2.2 The consequence: effects multiply

This is the single most important practical consequence of using a log or logit link, and the source of nearly every interpretation mistake.

On the η scale, effects add:

$$\log \mu = \beta_0 + \beta_1 x \quad \Longrightarrow \quad \text{a one-unit rise in } x \text{ adds } \beta_1 \text{ to } \log \mu.$$

On the scale you actually care about, effects *multiply*:

$$\mu = e^{\beta_0} e^{\beta_1 x} \quad \Longrightarrow \quad \text{a one-unit rise in } x \text{ multiplies } \mu \text{ by } e^{\beta_1}.$$

```
beta1 <- 0.55
exp(beta1)                # multiplier for a 1-unit increase in x

#> [1] 1.733

(exp(beta1) - 1) * 100    # ... expressed as a percentage change

#> [1] 73.33
```

So a coefficient of 0.55 on the log scale means the expected count is multiplied by 1.733 — a 73% increase — for every one-unit increase in x.

Key idea

Raw GLM coefficients are on the *link* scale and are almost never the thing to report. Exponentiate them. For a log link you get a **rate ratio** (or incidence rate ratio, IRR). For a logit link you get an **odds ratio** (OR).

2.3 How R fits these models

R does not use least squares here. It uses *maximum likelihood*, computed by an algorithm called iteratively reweighted least squares (IRLS). You do not need the details, but two by-products of it appear in every printout and are worth knowing.

2.3.1 Deviance

The deviance is the GLM version of "residual sum of squares". Formally it is

$$D = 2\left[\ell_{\text{saturated}} - \ell_{\text{model}}\right],$$

where ℓ is the log-likelihood and the "saturated" model is the one that fits every data point perfectly. Smaller deviance means a better fit. R prints two of them:

- **Null deviance** — how badly an intercept-only model fits.
- **Residual deviance** — how badly *your* model fits.

The drop from null to residual is the analogue of the regression sum of squares.

2.3.2 Degrees of freedom

Residual degrees of freedom = number of observations − number of estimated coefficients. You will use the pair (residual deviance, residual df) constantly in Chapter 7 to detect overdispersion.

Let us look at an actual printout and label every piece of it.

```
data(warpbreaks)
m <- glm(breaks ~ wool + tension, family = poisson, data = warpbreaks)
summary(m)


#>
#> Call:
#> glm(formula = breaks ~ wool + tension, family = poisson, data = warpbreaks)
#>
#> Coefficients:
#>             Estimate Std. Error z value Pr(>|z|)
#> (Intercept)   3.6920     0.0454   81.30  < 2e-16 ***
#> woolB        -0.2060     0.0516   -3.99  6.5e-05 ***
#> tensionM     -0.3213     0.0603   -5.33  9.7e-08 ***
#> tensionH     -0.5185     0.0640   -8.11  5.2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> (Dispersion parameter for poisson family taken to be 1)
#>
#>     Null deviance: 297.37  on 53  degrees of freedom
#> Residual deviance: 210.39  on 50  degrees of freedom
#> AIC: 493.1
#>
#> Number of Fisher Scoring iterations: 4
```

Reading that output, piece by piece:

Line in the printout	What it means
`Estimate`	The coefficient on the *link* scale (here, log counts).
`Std. Error`	Its standard error, from the curvature of the likelihood.
`z value`	Estimate / Std. Error. A Wald statistic.
`Pr(>|z|)`	Two-sided p-value from the standard normal.
`Dispersion parameter ...taken to be 1`	R is *assuming* variance = mean. Chapter 7 checks whether that is true.
`Null deviance`	Fit of the intercept-only model.
`Residual deviance`	Fit of this model.
`AIC`	$-2\ell + 2k$; used to compare models (Chapter 13).
`Fisher Scoring iterations`	How many IRLS steps were needed. Under about 25 is normal; a very large number hints at trouble.

2.4 Getting things out of a fitted model

These accessor functions work on essentially every model object in this book, and knowing them saves an enormous amount of time.

```
coef(m)                          # coefficients on the link scale

#> (Intercept)       woolB    tensionM    tensionH
#>      3.6920     -0.2060     -0.3213     -0.5185

exp(coef(m))                     # ... as multiplicative effects

#> (Intercept)       woolB    tensionM    tensionH
#>     40.1235      0.8138      0.7252      0.5954
```

```r
head(fitted(m), 4)              # fitted means, on the RESPONSE scale

#>     1     2     3     4
#> 40.12 40.12 40.12 40.12

head(predict(m), 4)             # linear predictor eta (link scale) -- careful!

#>     1     2     3     4
#> 3.692 3.692 3.692 3.692

head(predict(m, type = "response"), 4)   # the same as fitted()

#>     1     2     3     4
#> 40.12 40.12 40.12 40.12

deviance(m); df.residual(m); AIC(m); logLik(m)

#> [1] 210.4
#> [1] 50
#> [1] 493.1
#> 'log Lik.' -242.5 (df=4)
```

`predict()` defaults to the link scale

> For a GLM, `predict(m)` returns η, not μ. If you forget `type = "response"` you will get log-counts or log-odds and wonder why your "predicted counts" are negative. This trips up nearly everybody at least once.

Confidence intervals are also worth doing properly. `confint()` on a `glm` object gives *profile likelihood* intervals, which are more accurate than the textbook estimate $\pm 1.96\times$ SE, especially in small samples:

```r
confint(m)                      # profile likelihood (better)

#>               2.5 %  97.5 %
#> (Intercept)  3.6019  3.7799
#> woolB       -0.3073 -0.1051
#> tensionM    -0.4398 -0.2035
#> tensionH    -0.6446 -0.3938

confint.default(m)              # Wald: estimate +/- 1.96 SE (quick and dirty)

#>               2.5 %  97.5 %
#> (Intercept)  3.6030  3.7810
#> woolB       -0.3071 -0.1049
#> tensionM    -0.4394 -0.2032
#> tensionH    -0.6438 -0.3931

exp(confint(m))                 # rate ratios with CI -- the reportable version

#>               2.5 %  97.5 %
#> (Intercept) 36.6685 43.8135
#> woolB        0.7355  0.9003
#> tensionM     0.6441  0.8158
#> tensionH     0.5249  0.6745
```

Rule of thumb

Report exponentiated coefficients with profile-likelihood confidence intervals. That single habit will make your results both correct and readable.

CHAPTER 3

Logistic regression I: the basics

3.1 Probability, odds, and log-odds

Logistic regression is built on a chain of three ways to express the same thing. Get comfortable with the chain and the rest is easy.

$$\underbrace{p}_{\text{probability}} \longrightarrow \underbrace{\frac{p}{1-p}}_{\text{odds}} \longrightarrow \underbrace{\log\left(\frac{p}{1-p}\right)}_{\text{log-odds, or logit}}$$

Odds in one sentence

> The odds is how many times more likely the event is than the non-event: a probability of 0.75 means the event is three times more likely than not, so the odds are 3, often written "3 to 1".

```
p      <- c(0.01, 0.10, 0.25, 0.50, 0.75, 0.90, 0.99)
odds   <- p / (1 - p)
logodds <- log(odds)
data.frame(probability = p, odds = round(odds, 3), log_odds = round(logodds, 3))


#>   probability   odds log_odds
#> 1        0.01  0.010   -4.595
#> 2        0.10  0.111   -2.197
#> 3        0.25  0.333   -1.099
#> 4        0.50  1.000    0.000
#> 5        0.75  3.000    1.099
#> 6        0.90  9.000    2.197
#> 7        0.99 99.000    4.595
```

Look at what each column does at the extremes:

- **Probability** is trapped between 0 and 1. Useless as a regression outcome.
- **Odds** runs from 0 to ∞. Better, but still one-sided, and it is not symmetric: doubling from 0.4 to 0.8 in probability is a very different jump from 0.1 to 0.2.
- **Log-odds** runs from $-\infty$ to $+\infty$ and is symmetric around zero. $p = 0.5$ gives log-odds 0. Perfect for a linear predictor.

```
par(mfrow = c(1, 2)); par_book()
pp <- seq(0.001, 0.999, length.out = 500)
plot(pp, log(pp/(1-pp)), type = "l", lwd = 3, col = PAL["primary"],
     xlab = "probability p", ylab = "logit(p)",
     main = "logit: (0,1) stretched to the whole line")
grid_book(); abline(h = 0, v = 0.5, lty = 3, col = PAL["accent"])

plot(pp, log(pp/(1-pp)), type = "n", xlab = "logit(p)", ylab = "probability p",
     xlim = c(-6, 6), ylim = c(0, 1), main = "and back again")
```

```
grid_book()
lines(seq(-6, 6, length.out = 400), plogis(seq(-6, 6, length.out = 400)),
      lwd = 3, col = PAL["green"])
abline(h = 0.5, v = 0, lty = 3, col = PAL["accent"])
```

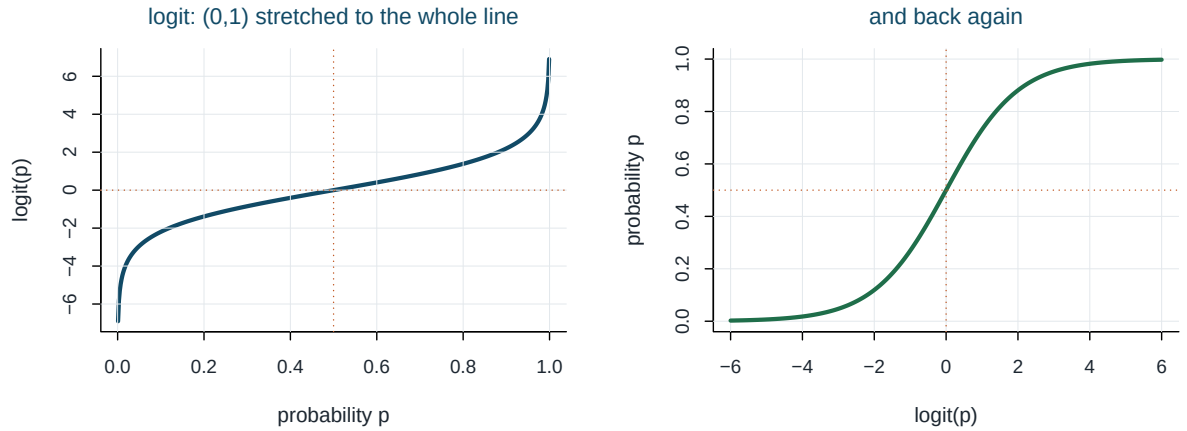


Figure 3.1: Left: the logit transformation stretches probabilities near 0 and 1 out towards infinity. Right: the inverse, which is what the fitted model uses to turn a linear predictor back into a probability.

3.2 The model

For a binary outcome $y_i \in \{0, 1\}$ with $\Pr(y_i = 1) = p_i$:

$$y_i \sim \text{Bernoulli}(p_i), \qquad \log\!\left(\frac{p_i}{1 - p_i}\right) = \beta_0 + \beta_1 x_{1i} + \cdots + \beta_p x_{pi}.$$

Equivalently, solving for p_i:

$$p_i = \frac{1}{1 + e^{-\eta_i}} = \frac{e^{\eta_i}}{1 + e^{\eta_i}}.$$

3.3 A worked example: low birth weight

The `birthwt` data records 189 births at a US hospital. The outcome `low` is 1 if the baby weighed under 2.5 kg. We will model it from the mother's characteristics.

```
data(birthwt, package = "MASS")
bw <- within(birthwt, {
  race  <- factor(race, levels = 1:3, labels = c("white", "black", "other"))
  smoke <- factor(smoke, levels = 0:1, labels = c("no", "yes"))
  ht    <- factor(ht,    levels = 0:1, labels = c("no", "yes"))
  ui    <- factor(ui,    levels = 0:1, labels = c("no", "yes"))
})
str(bw[, c("low", "age", "lwt", "race", "smoke", "ht", "ui")])


#> 'data.frame': 189 obs. of  7 variables:
#>  $ low  : int  0 0 0 0 0 0 0 0 0 0 ...
#>  $ age  : int  19 33 20 21 18 21 22 17 29 26 ...
#>  $ lwt  : int  182 155 105 108 107 124 118 103 123 113 ...
#>  $ race : Factor w/ 3 levels "white","black",..: 2 3 1 1 1 3 1 3 1 1 ...
#>  $ smoke: Factor w/ 2 levels "no","yes": 1 1 2 2 2 1 1 1 2 2 ...
```

```
#>  $ ht   : Factor w/ 2 levels "no","yes": 1 1 1 1 1 1 1 1 1 1 ...
#>  $ ui   : Factor w/ 2 levels "no","yes": 2 1 1 2 2 1 1 1 1 1 ...

table(bw$low)

#>
#>   0   1
#> 130  59
```

So 59 of 189 babies (31.2%) were low birth weight. Always look at this before modelling: if the outcome is very rare, you will need a lot of data to estimate anything precisely.

Events per variable

> A common guideline is at least 10 events *and* 10 non-events for each coefficient you estimate. With 59 events here, models with more than about 5 predictors start to be optimistic. This guideline is soft — more recent work suggests it can be relaxed or tightened depending on the signal — but it stops you from fitting 15 predictors to 30 events.

3.3.1 Fitting

```
fit1 <- glm(low ~ age + lwt + race + smoke + ht + ui,
            family = binomial(link = "logit"), data = bw)
summary(fit1)

#>
#> Call:
#> glm(formula = low ~ age + lwt + race + smoke + ht + ui, family = binomial(link = "logit"),
#>     data = bw)
#>
#> Coefficients:
#>             Estimate Std. Error z value Pr(>|z|)
#> (Intercept)  0.43724    1.19193    0.37   0.7137
#> age         -0.01826    0.03535   -0.52   0.6056
#> lwt         -0.01629    0.00686   -2.37   0.0176 *
#> raceblack    1.28064    0.52669    2.43   0.0150 *
#> raceother    0.90188    0.43436    2.08   0.0379 *
#> smokeyes     1.02757    0.39393    2.61   0.0091 **
#> htyes        1.85762    0.68885    2.70   0.0070 **
#> uiyes        0.89539    0.44849    2.00   0.0459 *
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> (Dispersion parameter for binomial family taken to be 1)
#>
#>     Null deviance: 234.67  on 188  degrees of freedom
#> Residual deviance: 203.95  on 181  degrees of freedom
#> AIC: 219.9
#>
#> Number of Fisher Scoring iterations: 4
```

3.3.2 Reading the coefficients

Everything in the `Estimate` column is a change in *log-odds*. Nobody thinks in log-odds. Exponentiate:

```
or <- exp(cbind(OR = coef(fit1), confint(fit1)))
round(or, 3)
```

```
#>                OR 2.5 % 97.5 %
#> (Intercept) 1.548 0.154 16.829
#> age         0.982 0.915  1.052
#> lwt         0.984 0.970  0.996
#> raceblack   3.599 1.286 10.316
#> raceother   2.464 1.065  5.907
#> smokeyes    2.794 1.310  6.193
#> htyes       6.408 1.719 27.006
#> uiyes       2.448 1.010  5.937
```

Now read them as multipliers of the odds:

- `smokeyes`: OR = 2.79. Mothers who smoked had about 2.8 times the odds of a low-birth-weight baby, holding the other variables fixed.
- `lwt`: OR = 0.984, i.e. each extra pound of the mother's weight multiplies the odds by 0.984 — a 1.6% *decrease* per pound. Small per pound, but over 30 pounds it compounds to a factor of 0.61.
- `raceblack` and `raceother`: these are compared with `white`, because `white` is the first level of the factor and R makes the first level the reference.

The three-part sentence for an odds ratio

> "Holding the other variables constant, a *one-unit increase in X* is associated with the odds of *Y* being multiplied by *exp(beta)*, 95% CI *[lower, upper]*." If you write every result in that shape you will never misreport a logistic regression.

3.3.3 Rescaling to make a coefficient readable

An odds ratio "per pound" is awkward. Rescale the variable to a meaningful unit before fitting, or multiply the coefficient afterwards:

```
## Per 10 pounds instead of per pound
exp(10 * coef(fit1)["lwt"])
```

```
#>    lwt
#> 0.8497
```

```
## Or refit with a rescaled variable -- same thing, easier to report
fit1b <- update(fit1, . ~ . - lwt + I(lwt / 10))
round(exp(cbind(OR = coef(fit1b), confint.default(fit1b))), 3)["I(lwt/10)", ]
```

```
#>     OR  2.5 % 97.5 %
#>  0.850  0.743  0.972
```

Odds ratios are not risk ratios

> An odds ratio of 3 does *not* mean the event is three times as likely. When the outcome is rare (say under 10%) the two are close, but when it is common they diverge badly. If the baseline probability is 0.4, an OR of 3 corresponds to a probability of 0.67 — a risk ratio of

only 1.7, not 3. Say “odds” when you mean odds.

```
par_book()
p0 <- seq(0.005, 0.6, length.out = 300)
for (OR in c(1.5, 2, 3, 5)) {
  odds1 <- OR * p0 / (1 - p0)
  p1 <- odds1 / (1 + odds1)
  rr <- p1 / p0
  if (OR == 1.5) {
    plot(p0, rr, type = "l", lwd = 2.5, col = PAL["primary"], ylim = c(1, 5.2),
         xlab = "baseline probability", ylab = "corresponding risk ratio",
         main = "An odds ratio overstates the risk ratio")
    grid_book()
  } else {
    lines(p0, rr, lwd = 2.5,
          col = c(PAL["accent"], PAL["green"], PAL["red"])[which(c(2,3,5) == OR)])
  }
  abline(h = OR, lty = 3, col = PAL["grey"])
}
legend("topright", bty = "n", cex = 0.8, lwd = 2.5,
       col = c(PAL["primary"], PAL["accent"], PAL["green"], PAL["red"]),
       legend = paste("OR =", c(1.5, 2, 3, 5)))
```

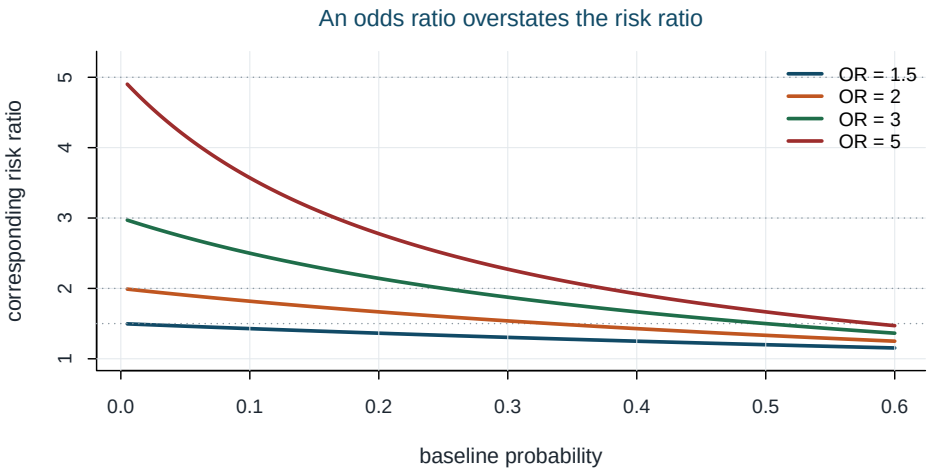


Figure 3.2: How far an odds ratio is from the corresponding risk ratio, as the baseline probability rises. They only agree for rare outcomes.

3.4 Predicted probabilities

Coefficients answer “how much does this change things?”. Predicted probabilities answer “what actually happens to a specific person?” — and are usually far more persuasive to a non-statistical audience.

```
newdat <- expand.grid(
  age   = 25,
  lwt   = seq(80, 250, by = 5),
  race  = "white",
  smoke = c("no", "yes"),
  ht    = "no",
```

```
  ui    = "no"
)
pr <- predict(fit1, newdata = newdat, type = "link", se.fit = TRUE)
newdat$fit <- plogis(pr$fit)
newdat$lo  <- plogis(pr$fit - 1.96 * pr$se.fit)
newdat$hi  <- plogis(pr$fit + 1.96 * pr$se.fit)
head(newdat[, c("lwt", "smoke", "fit", "lo", "hi")], 4)


#>   lwt smoke    fit      lo     hi
#> 1  80    no 0.2105 0.08972 0.4190
#> 2  85    no 0.1973 0.08640 0.3898
#> 3  90    no 0.1847 0.08295 0.3620
#> 4  95    no 0.1728 0.07936 0.3359
```

Always build intervals on the link scale

Compute the confidence interval for η (where the normal approximation is good), *then* push both endpoints through `plogis`. If you build the interval directly on the probability scale you can get limits below 0 or above 1.

```
par_book()
sm <- subset(newdat, smoke == "yes"); ns <- subset(newdat, smoke == "no")
plot(newdat$lwt, newdat$fit, type = "n", ylim = c(0, 0.85),
     xlab = "mother's weight at last period (lbs)",
     ylab = "P(low birth weight)",
     main = "Fitted probabilities, 25-year-old white mother, no HT/UI")
grid_book()
polygon(c(sm$lwt, rev(sm$lwt)), c(sm$lo, rev(sm$hi)),
        col = adjustcolor(PAL["red"], 0.15), border = NA)
polygon(c(ns$lwt, rev(ns$lwt)), c(ns$lo, rev(ns$hi)),
        col = adjustcolor(PAL["primary"], 0.15), border = NA)
lines(sm$lwt, sm$fit, lwd = 3, col = PAL["red"])
lines(ns$lwt, ns$fit, lwd = 3, col = PAL["primary"])
legend("topright", bty = "n", lwd = 3, col = c(PAL["red"], PAL["primary"]),
       legend = c("smoker", "non-smoker"), cex = 0.85)
rug(bw$lwt, col = adjustcolor(PAL["grey"], 0.6))
```

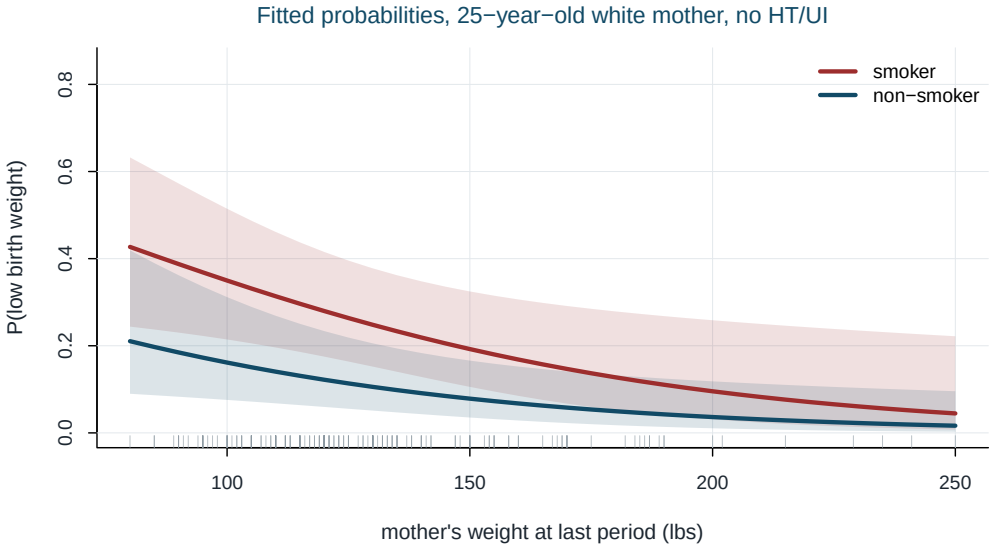


Figure 3.3: Fitted probability of low birth weight against the mother's weight, for smokers and non-smokers, with 95% confidence bands. This is the plot to put in a report.

The `rug` at the bottom shows where the data actually are. Notice the confidence band widens where there are few mothers — above about 200 pounds the model is extrapolating and you should not trust it.

3.5 Categorical predictors and reference levels

R turns a factor with k levels into $k-1$ dummy variables, all compared with the reference level. You control which level is the reference:

```
levels(bw$race)                          # reference is the first level

#> [1] "white" "black" "other"

bw$race2 <- relevel(bw$race, ref = "other")
fit_rl <- glm(low ~ smoke + race2, family = binomial, data = bw)
round(exp(coef(fit_rl)), 3)

#> (Intercept)    smokeyes  race2white  race2black
#>       0.481       3.053       0.330       0.976
```

To test a factor *as a whole* (all its levels at once) you need a single test with the right degrees of freedom, not a glance at the individual p-values:

```
anova(fit1, test = "LRT")

#> Analysis of Deviance Table
#>
#> Model: binomial, link: logit
#>
#> Response: low
#>
#> Terms added sequentially (first to last)
#>
```

```
#>
#>       Df Deviance Resid. Df Resid. Dev Pr(>Chi)
#> NULL                    188        235
#> age    1     2.76       187        232   0.0966 .
#> lwt    1     4.79       186        227   0.0286 *
#> race   2     4.46       184        223   0.1074
#> smoke  1     8.08       183        215   0.0045 **
#> ht     1     6.70       182        208   0.0096 **
#> ui     1     3.93       181        204   0.0474 *
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

That table adds terms one at a time in the order you wrote them and gives a likelihood-ratio test for each. `race` has 2 degrees of freedom because it has three levels. Chapter 13 goes into these tests properly.

Order matters in `anova()`

> `anova.glm` produces *sequential* (Type I) tests. Change the order of terms in the formula and the numbers change. For order-independent tests use `car::Anova(fit1, type = "II")`.

```
car::Anova(fit1, type = "II", test.statistic = "LR")
```

```
#> Analysis of Deviance Table (Type II tests)
#>
#> Response: low
#>       LR Chisq Df Pr(>Chisq)
#> age       0.27  1     0.6043
#> lwt       6.40  1     0.0114 *
#> race      7.83  2     0.0199 *
#> smoke     7.12  1     0.0076 **
#> ht        7.63  1     0.0057 **
#> ui        3.93  1     0.0474 *
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

3.6 Interactions on the logit scale

An interaction means the effect of one variable depends on another. On the logit scale it is a ratio of odds ratios.

```
fit_int <- glm(low ~ smoke * ht + lwt, family = binomial, data = bw)
round(exp(coef(fit_int)), 3)
```

```
#>    (Intercept)       smokeyes          htyes            lwt smokeyes:htyes
#>          3.067          1.954          5.719          0.982          1.225
```

Because the model is multiplicative, the odds ratio for smoking among mothers *with* hypertension is the product of the main effect and the interaction term. It is far safer to let emmeans do this arithmetic:

```
library(emmeans)
emm <- emmeans(fit_int, ~ smoke | ht, type = "response")
pairs(emm, reverse = TRUE)
```

```
#> ht = no:
#>  contrast odds.ratio    SE  df null z.ratio p.value
#>  yes / no       1.95 0.673 Inf    1   1.945  0.0518
#>
#> ht = yes:
#>  contrast odds.ratio    SE  df null z.ratio p.value
#>  yes / no       2.39 3.099 Inf    1   0.674  0.5005
#>
#> Tests are performed on the log odds ratio scale
```

What that table says

It gives the smoking odds ratio separately within each hypertension group, with the standard errors worked out correctly. Doing this by hand from the coefficient table is possible but easy to get wrong, especially with factors of more than two levels.

Interactions are hungry

An interaction between two binary variables splits your events into four cells. With 59 events, a smoke × hypertension interaction is estimated from very few observations — look at the standard error above. Wide intervals here are honest, not a failure.

CHAPTER 4

Logistic regression II: checking and extending

A fitted logistic model raises two separate questions, and people constantly confuse them:

1. **Discrimination.** Can the model tell the cases from the non-cases? Measured by the ROC curve and the area under it (AUC).
2. **Calibration.** When the model says 30%, does it happen about 30% of the time? Measured by calibration plots and goodness-of-fit tests.

A model can discriminate beautifully and be badly calibrated, or be perfectly calibrated and useless at discriminating. You need both.

4.1 Turning probabilities into decisions

```
phat <- fitted(fit1)                  # predicted probabilities
pred <- ifelse(phat > 0.5, 1, 0)      # classify at the usual 0.5 cut-off
tab  <- table(predicted = pred, observed = bw$low)
tab
```

```
#>           observed
#> predicted   0   1
#>         0 118  36
#>         1  12  23
```

```
sens <- tab["1", "1"] / sum(tab[, "1"])     # sensitivity: caught cases / all cases
spec <- tab["0", "0"] / sum(tab[, "0"])     # specificity: caught controls / all controls
acc  <- sum(diag(tab)) / sum(tab)
c(sensitivity = sens, specificity = spec, accuracy = acc)
```

```
#> sensitivity specificity    accuracy
#>      0.3898      0.9077      0.7460
```

```
mean(bw$low == 0)                           # accuracy of "always predict 0"
```

```
#> [1] 0.6878
```

Accuracy is a trap

> Here the model's accuracy is 0.746, and simply predicting "never low birth weight" for everybody scores 0.688. When one class is more common, accuracy mostly measures how common that class is. Never report accuracy alone.

The 0.5 cut-off is a convention, not a law. If missing a case is much worse than a false alarm, lower it.

4.2 The ROC curve and AUC

The ROC curve draws sensitivity against 1− specificity as you slide the cut-off from 1 down to 0. Rather than reach for a package, here is the whole thing in nine lines — it demystifies the plot completely.

```
roc_manual <- function(prob, y) {
  ord <- order(prob, decreasing = TRUE)
  y   <- y[ord]
  tpr <- cumsum(y)       / sum(y)          # true positive rate at each cut-off
  fpr <- cumsum(1 - y)   / sum(1 - y)      # false positive rate
  auc <- sum(diff(c(0, fpr)) * (c(0, head(tpr, -1)) + tpr) / 2)   # trapezoid rule
  list(fpr = c(0, fpr), tpr = c(0, tpr), auc = auc)
}
r <- roc_manual(phat, bw$low)
r$auc


#> [1] 0.7343
```

```
par(mfrow = c(1, 2)); par_book()
plot(r$fpr, r$tpr, type = "l", lwd = 3, col = PAL["primary"],
     xlab = "1 - specificity (false positive rate)",
     ylab = "sensitivity (true positive rate)",
     main = sprintf("ROC curve, AUC = %.3f", r$auc))
grid_book()
polygon(c(r$fpr, 1), c(r$tpr, 0), col = adjustcolor(PAL["primary"], 0.10), border = NA)
abline(0, 1, lty = 2, col = PAL["grey"], lwd = 2)

cuts <- seq(0.01, 0.99, by = 0.01)
se <- sapply(cuts, function(k) mean(phat[bw$low == 1] >= k))
sp <- sapply(cuts, function(k) mean(phat[bw$low == 0] <  k))
plot(cuts, se, type = "l", lwd = 3, col = PAL["red"], ylim = c(0, 1),
     xlab = "probability cut-off", ylab = "rate", main = "The trade-off")
grid_book()
lines(cuts, sp, lwd = 3, col = PAL["green"])
abline(v = cuts[which.max(se + sp)], lty = 3, col = PAL["accent"], lwd = 2)
legend("right", bty = "n", cex = 0.8, lwd = 3,
       col = c(PAL["red"], PAL["green"], PAL["accent"]), lty = c(1,1,3),
       legend = c("sensitivity", "specificity", "best sens+spec"))
```

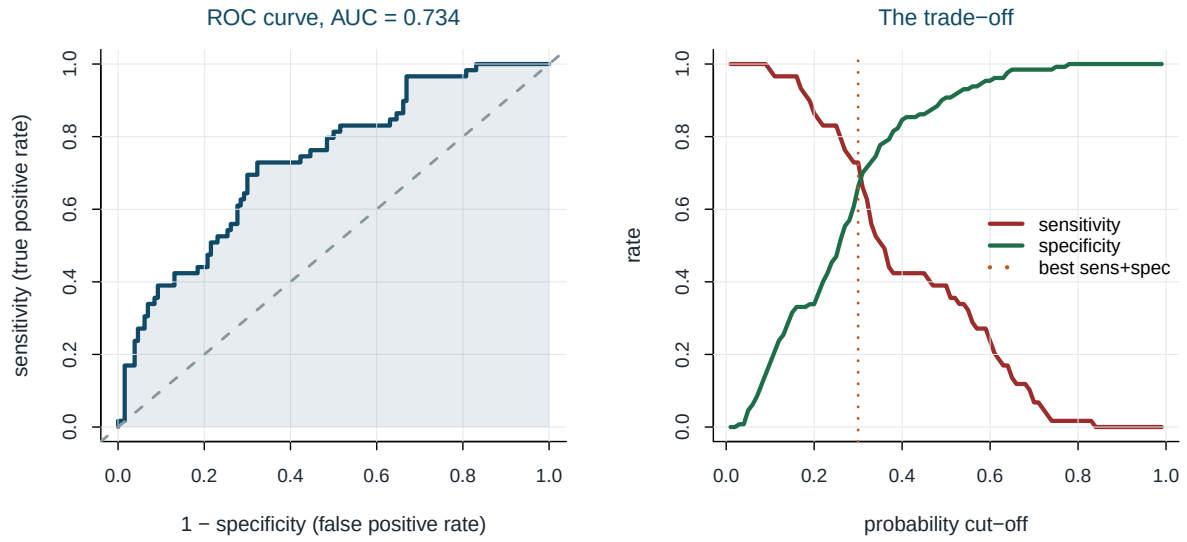


Figure 4.1: Left: the ROC curve. The diagonal is a coin flip; the further towards the top-left corner, the better. Right: how sensitivity and specificity trade off as the cut-off moves.

What AUC actually measures

Pick one case and one non-case at random. The AUC is the probability that the model gives the case a higher score. AUC = 0.5 is a coin flip; 1.0 is perfect. Roughly: 0.6 poor, 0.7 acceptable, 0.8 good, 0.9 excellent — but the right threshold depends entirely on what the model is for.

This AUC is optimistic

We computed the AUC on the same data used to fit the model. That always flatters it. For an honest figure use cross-validation or a held-out sample.

```
set.seed(7)
K     <- 10
folds <- sample(rep(1:K, length.out = nrow(bw)))
oof   <- numeric(nrow(bw))                    # out-of-fold predictions
for (k in 1:K) {
  m_k <- glm(formula(fit1), family = binomial, data = bw[folds != k, ])
  oof[folds == k] <- predict(m_k, newdata = bw[folds == k, ], type = "response")
}
c(in_sample = r$auc, cross_validated = roc_manual(oof, bw$low)$auc)


#>       in_sample cross_validated
#>          0.7343          0.6505
```

4.3 Calibration

Sort the predictions into bins, and compare the average prediction in each bin with the proportion actually observed. On a perfectly calibrated model the points sit on the diagonal.

```
par_book()
g   <- cut(phat, breaks = quantile(phat, probs = seq(0, 1, 0.1)), include.lowest = TRUE)
obs <- tapply(bw$low, g, mean)
exp_ <- tapply(phat,  g, mean)
n_g <- tapply(bw$low, g, length)
se_g <- sqrt(obs * (1 - obs) / n_g)
plot(exp_, obs, pch = 19, col = PAL["primary"], cex = 1.4, xlim = c(0, 0.8),
     ylim = c(0, 0.9), xlab = "mean predicted probability",
     ylab = "observed proportion", main = "Calibration in 10 bins")
grid_book(); abline(0, 1, lwd = 2, lty = 2, col = PAL["grey"])
arrows(exp_, obs - 1.96*se_g, exp_, obs + 1.96*se_g,
       length = 0.03, angle = 90, code = 3, col = PAL["primary"])
lines(lowess(exp_, obs), col = PAL["accent"], lwd = 2.5)
```

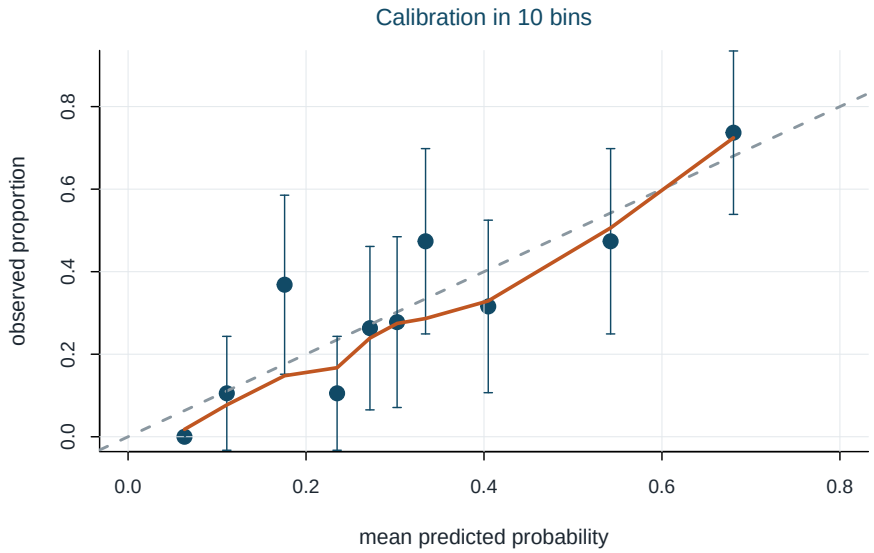


Figure 4.2: Calibration plot with 10 equal-sized bins. Points near the diagonal mean the predicted probabilities can be taken at face value.

4.3.1 The Hosmer–Lemeshow test

The same idea turned into a formal test: bin the predictions into g groups (usually 10), then compare observed and expected counts with a chi-squared statistic on $g - 2$ degrees of freedom.

```
hoslem <- function(y, phat, g = 10) {
  cuts <- quantile(phat, probs = seq(0, 1, length.out = g + 1))
  grp  <- cut(phat, breaks = cuts, include.lowest = TRUE)
  obs1 <- tapply(y, grp, sum);              obs0 <- tapply(1 - y, grp, sum)
  exp1 <- tapply(phat, grp, sum);           exp0 <- tapply(1 - phat, grp, sum)
  X2   <- sum((obs1 - exp1)^2 / exp1) + sum((obs0 - exp0)^2 / exp0)
  data.frame(X2 = X2, df = g - 2, p.value = pchisq(X2, g - 2, lower.tail = FALSE))
}
hoslem(bw$low, phat)


#>      X2 df p.value
#> 1 10.91  8   0.207
```

How to read a goodness-of-fit p-value

Here a *large* p-value is the reassuring one: no evidence of poor fit. But this test is weak. It has little power in small samples, it is sensitive to the arbitrary number of bins, and in very large samples it rejects fit that is practically fine. Use it as a sanity check alongside the calibration plot, never as the only evidence.

4.4 Residuals and influence

Raw residuals $y - \hat{p}$ are almost useless for a binary outcome — they can only take two values for any given $\hat{p}$, so a residual plot looks like two curved stripes. Two better versions exist.

- **Pearson residual:** $r_i = \dfrac{y_i - \hat{p}_i}{\sqrt{\hat{p}_i(1-\hat{p}_i)}}$, which standardises by the expected variability.
- **Deviance residual:** the signed square root of each observation's contribution to the deviance. This is R's default from `residuals()`.

```
par(mfrow = c(1, 2)); par_book()
plot(fitted(fit1), residuals(fit1, type = "deviance"),
     pch = 21, bg = adjustcolor(PAL["primary"], 0.4), col = "white", cex = 1.1,
     xlab = "fitted probability", ylab = "deviance residual",
     main = "Deviance residuals (read the smoother)")
grid_book(); abline(h = 0, col = PAL["grey"], lwd = 1.5)
lines(lowess(fitted(fit1), residuals(fit1, type = "deviance")),
      col = PAL["red"], lwd = 3)

cd <- cooks.distance(fit1)
plot(cd, type = "h", lwd = 2, col = PAL["primary"], xlab = "observation",
     ylab = "Cook's distance", main = "Influence")
grid_book()
abline(h = 4 / nrow(bw), lty = 2, col = PAL["red"])
text(which(cd > 4/nrow(bw)), cd[cd > 4/nrow(bw)],
     labels = which(cd > 4/nrow(bw)), pos = 3, cex = 0.65, col = PAL["red"])
```

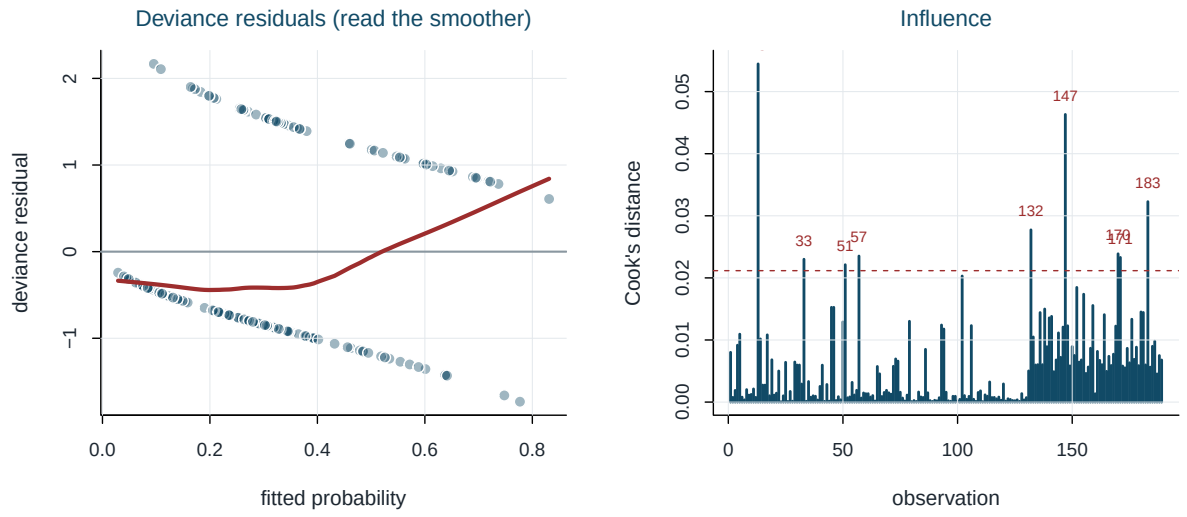


Figure 4.3: Left: the two-stripe pattern that makes raw residual plots useless for binary data; the smoother is what you actually read. Right: influence, via Cook's distance.

What you are looking for

In the left panel, the red smoother should hug zero. A systematic bend means the model has the wrong shape — perhaps a predictor should be curved, or an interaction is missing. In the right panel, tall spikes are observations that would move the coefficients noticeably if deleted. Investigate them; do not delete them without a reason.

A binned residual plot is often more readable than either:

```
par_book()
br <- function(x, r, nbins = 15) {
  b  <- cut(x, breaks = quantile(x, seq(0, 1, length.out = nbins + 1)),
            include.lowest = TRUE)
  data.frame(x = tapply(x, b, mean), y = tapply(r, b, mean),
             se = tapply(r, b, function(z) 2 * sd(z) / sqrt(length(z))))
}
bb <- br(fitted(fit1), residuals(fit1, type = "response"))
plot(bb$x, bb$y, pch = 19, col = PAL["primary"], cex = 1.3,
     ylim = range(c(bb$y - bb$se, bb$y + bb$se, 0)) * 1.2,
     xlab = "mean fitted probability in bin", ylab = "mean residual in bin",
     main = "Binned residuals")
grid_book(); abline(h = 0, col = PAL["grey"], lwd = 1.5)
lines(bb$x, bb$se, lty = 2, col = PAL["red"]); lines(bb$x, -bb$se, lty = 2, col = PAL["red"])
```

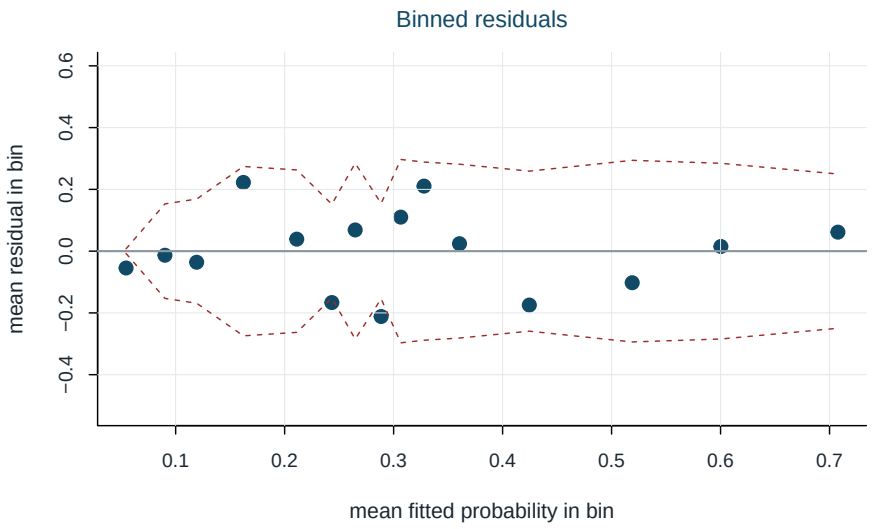


Figure 4.4: Binned residual plot. Average the residuals within bins of fitted value; roughly 95% of the points should fall inside the grey band if the model fits.

4.5 Grouped (binomial) data

Often your data arrive already aggregated: "out of n animals in this herd, k got the disease". That is still logistic regression — the outcome is just binomial rather than Bernoulli.

```
data(cbpp, package = "lme4")
head(cbpp, 4)

#>   herd incidence size period
```

```
#> 1    1         2   14      1
#> 2    1         3   12      2
#> 3    1         4    9      3
#> 4    1         0    5      4
```

```
## Two-column response: successes and failures
mb <- glm(cbind(incidence, size - incidence) ~ period, family = binomial, data = cbpp)
summary(mb)
```

```
#>
#> Call:
#> glm(formula = cbind(incidence, size - incidence) ~ period, family = binomial,
#>     data = cbpp)
#>
#> Coefficients:
#>             Estimate Std. Error z value Pr(>|z|)
#> (Intercept)   -1.269      0.145   -8.76  < 2e-16 ***
#> period2       -1.171      0.291   -4.02 0.000059 ***
#> period3       -1.301      0.313   -4.16 0.000032 ***
#> period4       -1.782      0.413   -4.31 0.000016 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> (Dispersion parameter for binomial family taken to be 1)
#>
#>     Null deviance: 154.82  on 55  degrees of freedom
#> Residual deviance: 114.10  on 52  degrees of freedom
#> AIC: 206.1
#>
#> Number of Fisher Scoring iterations: 5
```

Three ways to give R a binomial response

`glm(cbind(k, n-k) ~ x, binomial)` — successes and failures.
`glm(prop ~ x, binomial, weights = n)` — proportion plus a weight.
`glm(y ~ x, binomial)` with `y` a 0/1 vector or a two-level factor.
All three fit the same model. The first two let you detect overdispersion; the third cannot.

4.6 Overdispersion in binomial models

For grouped data, the binomial says $\mathrm{Var}(k) = np(1-p)$ exactly. Real data often vary more, because animals within the same herd are not independent. Check the same way as for counts:

```
c(residual_deviance = deviance(mb), df = df.residual(mb),
  ratio = deviance(mb) / df.residual(mb))
```

```
#> residual_deviance                df             ratio
#>           114.102           52.000            2.194
```

```
## The Pearson version is the one to trust
X2 <- sum(residuals(mb, type = "pearson")^2)
c(pearson_X2 = X2, df = df.residual(mb), dispersion = X2 / df.residual(mb))
```

```
#> pearson_X2         df dispersion
#>     113.89      52.00       2.19
```

```
pchisq(X2, df.residual(mb), lower.tail = FALSE)     # p-value for "no overdispersion"

#> [1] 0.000001609
```

A dispersion of 2.19 means the data are about 2.2 times more variable than the binomial allows. Standard errors from that model are too small by roughly $\sqrt{2.19} = 1.48$.

The quick repair is a quasi-binomial fit, which multiplies all the standard errors by that square root:

```
mq <- glm(cbind(incidence, size - incidence) ~ period,
          family = quasibinomial, data = cbpp)
summary(mq)$dispersion

#> [1] 2.19

round(cbind(binomial = coef(summary(mb))[, "Std. Error"],
            quasibinomial = coef(summary(mq))[, "Std. Error"]), 4)

#>             binomial quasibinomial
#> (Intercept)   0.1449        0.2145
#> period2       0.2915        0.4313
#> period3       0.3129        0.4630
#> period4       0.4131        0.6113
```

The coefficients are identical; only the standard errors change. The proper repair — modelling *why* herds differ — is a mixed model, which we do in Chapter 15 with this same dataset.

You cannot detect overdispersion in ungrouped 0/1 data

> With a Bernoulli outcome the residual deviance carries no information about dispersion, so the deviance/df ratio is meaningless. If your data are truly one row per individual with no grouping, "binary overdispersion" is not a thing you can test for — what you can have instead is correlation between individuals, which needs a random effect or a robust standard error.

4.7 Complete separation

If some combination of predictors perfectly predicts the outcome, the maximum likelihood estimate runs off to infinity. R warns you — but the warning is easy to miss.

```
sep <- data.frame(x = c(1, 2, 3, 4, 5, 6), y = c(0, 0, 0, 1, 1, 1))
ms  <- glm(y ~ x, family = binomial, data = sep)

#> Warning:  glm.fit:  fitted probabilities numerically 0 or 1 occurred

round(coef(summary(ms)), 3)

#>             Estimate Std. Error z value Pr(>|z|)
#> (Intercept)  -165.32     407521       0        1
#> x              47.23     115264       0        1
```

Notice the absurd estimate with an enormous standard error, and a p-value near 1 despite a perfect relationship. Signs of separation:

- coefficients in the tens or hundreds;
- standard errors in the thousands;
- the warning `fitted probabilities numerically 0 or 1 occurred`;
- residual deviance essentially zero.

Fixes, in order of preference: combine sparse categories; drop the offending predictor; or use penalised likelihood (Firth logistic regression, in the logistf or brglm2 packages), which always gives finite estimates.

4.8 Other links for binary data

The logit is not the only choice.

```
m_logit  <- glm(low ~ lwt + smoke, family = binomial(link = "logit"),    data = bw)
m_probit <- glm(low ~ lwt + smoke, family = binomial(link = "probit"),   data = bw)
m_cll    <- glm(low ~ lwt + smoke, family = binomial(link = "cloglog"),  data = bw)
round(rbind(logit = coef(m_logit), probit = coef(m_probit), cloglog = coef(m_cll)), 3)
```

```
#>         (Intercept)    lwt smokeyes
#> logit         0.622 -0.013    0.677
#> probit        0.354 -0.008    0.418
#> cloglog       0.191 -0.011    0.529
```

```
sapply(list(logit = m_logit, probit = m_probit, cloglog = m_cll), AIC)
```

```
#>   logit  probit cloglog
#>   230.3   230.2   230.6
```

```
par_book()
nd <- data.frame(lwt = seq(80, 250, 2), smoke = "yes")
plot(nd$lwt, predict(m_logit, nd, type = "response"), type = "l", lwd = 3,
     col = PAL["primary"], ylim = c(0, 0.8), xlab = "mother's weight (lbs)",
     ylab = "P(low birth weight)", main = "logit vs probit vs cloglog")
grid_book()
lines(nd$lwt, predict(m_probit, nd, type = "response"), lwd = 3, lty = 2, col = PAL["accent"])
lines(nd$lwt, predict(m_cll,    nd, type = "response"), lwd = 3, lty = 3, col = PAL["green"])
legend("topright", bty = "n", lwd = 3, lty = 1:3, cex = 0.85,
       col = c(PAL["primary"], PAL["accent"], PAL["green"]),
       legend = c("logit", "probit", "cloglog"))
```

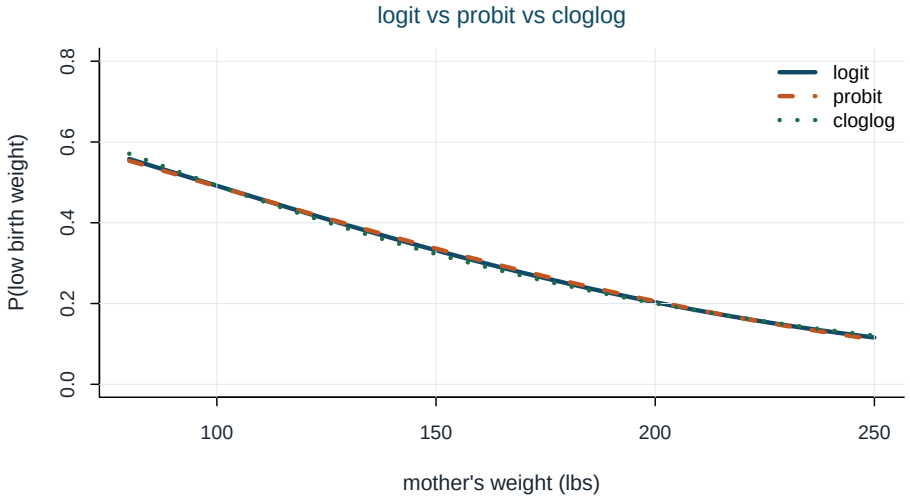


Figure 4.5: Three link functions on the same data. Logit and probit are nearly indistinguishable; the complementary log-log is asymmetric.

Rule of thumb

Use the logit unless you have a specific reason not to. It is the only one whose coefficients exponentiate to odds ratios, which is why every applied field reports it. The probit gives practically identical fitted probabilities. The cloglog is genuinely different and is the right choice when the binary outcome comes from an underlying Poisson process ("did at least one event happen?"), which links it naturally to survival analysis.

CHAPTER 5

Poisson regression I: modelling counts

5.1 Where the Poisson comes from

Imagine events happening independently at a constant average rate: phone calls arriving at a switchboard, mutations along a strand of DNA, potholes per kilometre of road. Count how many occur in a fixed window and you get a Poisson variable:

$$\Pr(Y = k) = \frac{e^{-\mu}\mu^k}{k!}, \qquad k = 0, 1, 2, \ldots$$

with the striking property

$$\mathrm{E}(Y) = \mathrm{Var}(Y) = \mu.$$

The Poisson assumption, in words

> Events happen one at a time, independently of each other, at a rate that does not change within the observation window. Nothing you have already seen makes the next event more or less likely.

That last sentence is exactly what fails in most real data — one accident causes a traffic jam that causes another; one sick child infects the others in the class. When it fails you get overdispersion, which is Chapter 7. First, the model itself.

5.2 The model

$$y_i \sim \mathrm{Poisson}(\mu_i), \qquad \log(\mu_i) = \beta_0 + \beta_1 x_{1i} + \ldots$$

Why the log link? Three reasons, all good:

1. It guarantees $\mu_i = e^{\eta_i} > 0$. Counts cannot have a negative mean.
2. It makes effects multiplicative, which is how count processes usually behave — "40% more accidents", not "3.2 more accidents".
3. It is the canonical link for the Poisson, which makes the fitting algorithm well behaved.

5.3 A first fit

The `warpbreaks` data counts breaks in yarn during weaving, for two wool types under three levels of tension.

```
data(warpbreaks)
par(mfrow = c(1, 2)); par_book()
hist(warpbreaks$breaks, breaks = 12, col = adjustcolor(PAL["primary"], 0.6),
     border = "white", xlab = "breaks", main = "Counts of yarn breaks")

mv <- aggregate(breaks ~ wool + tension, data = warpbreaks,
```

```
                FUN = function(z) c(m = mean(z), v = var(z)))
mv <- data.frame(wool = mv$wool, tension = mv$tension,
                 m = mv$breaks[, "m"], v = mv$breaks[, "v"])
plot(mv$m, mv$v, pch = 19, cex = 1.5, col = PAL["accent"],
     xlim = c(0, 45), ylim = c(0, 250),
     xlab = "group mean", ylab = "group variance",
     main = "Is variance = mean?")
grid_book(); abline(0, 1, lwd = 2.5, col = PAL["primary"])
text(mv$m, mv$v, paste0(mv$wool, mv$tension), pos = 4, cex = 0.7, col = PAL["grey"])
legend("topleft", bty = "n", cex = 0.8, lwd = 2.5, col = PAL["primary"],
       legend = "variance = mean (Poisson)")
```

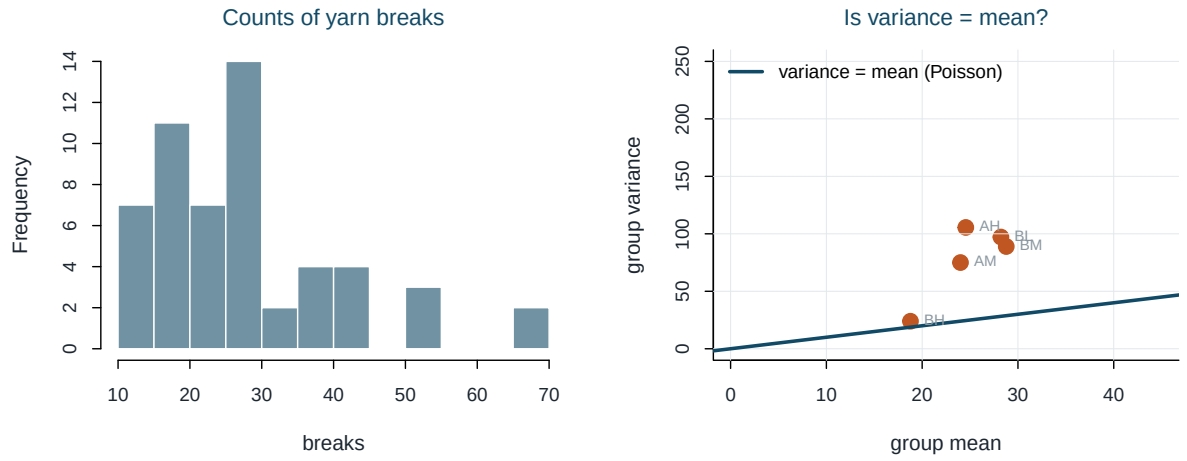


Figure 5.1: Left: the distribution of the counts. Right: group means against group variances. If the Poisson is right, the points should sit near the diagonal.

The variances sit well above the line — a first hint of overdispersion that we will deal with properly in Chapter 7. Fit the Poisson model anyway, so we have something to compare against.

```
pm <- glm(breaks ~ wool + tension, family = poisson, data = warpbreaks)
summary(pm)


#>
#> Call:
#> glm(formula = breaks ~ wool + tension, family = poisson, data = warpbreaks)
#>
#> Coefficients:
#>             Estimate Std. Error z value Pr(>|z|)
#> (Intercept)   3.6920     0.0454   81.30  < 2e-16 ***
#> woolB        -0.2060     0.0516   -3.99  6.5e-05 ***
#> tensionM     -0.3213     0.0603   -5.33  9.7e-08 ***
#> tensionH     -0.5185     0.0640   -8.11  5.2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> (Dispersion parameter for poisson family taken to be 1)
#>
#>     Null deviance: 297.37  on 53  degrees of freedom
#> Residual deviance: 210.39  on 50  degrees of freedom
#> AIC: 493.1
#>
#> Number of Fisher Scoring iterations: 4
```

5.4 Interpreting a log-link coefficient

```
round(exp(cbind(IRR = coef(pm), confint(pm))), 3)

#>                IRR  2.5 % 97.5 %
#> (Intercept) 40.124 36.668 43.814
#> woolB        0.814  0.735  0.900
#> tensionM     0.725  0.644  0.816
#> tensionH     0.595  0.525  0.675
```

The sentence to write

> "Wool type B is associated with an expected number of breaks 0.81 times that of wool type A (a 19% reduction), holding tension constant."

e^{β} from a log-link model has several names, all the same number: **rate ratio**, **incidence rate ratio (IRR)**, **risk ratio** when the counts are events per person, or just **multiplicative effect**.

A useful trick for small coefficients: for $|\beta| < 0.2$, $e^{\beta} - 1 \approx \beta$, so a coefficient of 0.08 is roughly an 8% increase. Beyond that the approximation drifts:

```
b <- c(0.02, 0.05, 0.1, 0.2, 0.5, 1)
data.frame(beta = b, exact_pct = round((exp(b) - 1) * 100, 1),
           approx_pct = b * 100)

#>   beta exact_pct approx_pct
#> 1 0.02       2.0          2
#> 2 0.05       5.1          5
#> 3 0.10      10.5         10
#> 4 0.20      22.1         20
#> 5 0.50      64.9         50
#> 6 1.00     171.8        100
```

5.4.1 Absolute versus relative effects

A rate ratio hides the baseline. Doubling a rate of 0.001 and doubling a rate of 40 are the same ratio and wildly different in practice. Always show predicted counts too:

```
nd <- expand.grid(wool = c("A", "B"), tension = c("L", "M", "H"))
nd$fit <- predict(pm, newdata = nd, type = "response")
nd

#>   wool tension   fit
#> 1    A       L 40.12
#> 2    B       L 32.65
#> 3    A       M 29.10
#> 4    B       M 23.68
#> 5    A       H 23.89
#> 6    B       H 19.44
```

```
par_book()
bp <- barplot(matrix(nd$fit, nrow = 2), beside = TRUE,
              names.arg = c("Low", "Medium", "High"),
              col = c(adjustcolor(PAL["primary"], 0.75), adjustcolor(PAL["accent"], 0.75)),
```

```
             border = "white", ylim = c(0, 55),
             xlab = "tension", ylab = "expected breaks",
             main = "Model predictions with the raw data overlaid")
legend("topright", bty = "n", fill = c(adjustcolor(PAL["primary"], 0.75),
       adjustcolor(PAL["accent"], 0.75)), border = "white",
       legend = c("wool A", "wool B"), cex = 0.85)
grp <- interaction(warpbreaks$wool, warpbreaks$tension)
xs  <- as.vector(bp)[as.integer(factor(grp, levels = c("A.L","B.L","A.M","B.M","A.H","B.H")))]
points(jitter(xs, amount = 0.15), warpbreaks$breaks, pch = 21,
       bg = adjustcolor("white", 0.9), col = PAL["grey"], cex = 0.9)
```

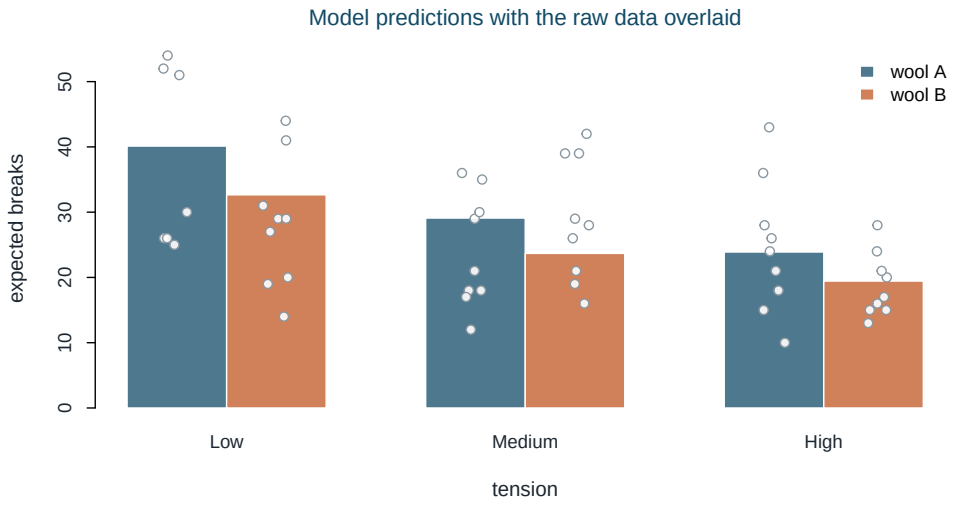


Figure 5.2: Observed counts (points) and Poisson model predictions (bars) for each wool-by-tension combination.

5.5 Why not just log-transform and use `lm`?

It is tempting to fit `lm(log(y + 1) ~ x)`. Three reasons not to:

1. **Zeros.** $\log(0)$ is undefined, so you have to add a fudge constant. The results depend on which constant you pick, which is not a comfortable position to defend.
2. **You change the question.** $\mathrm{E}[\log Y] \neq \log \mathrm{E}[Y]$. The linear model estimates the geometric mean, the Poisson model the arithmetic mean. Those differ, and the arithmetic mean is nearly always what you want.
3. **The variance is still wrong.** Logging helps but rarely produces constant variance for counts, especially with many small values.

```
lm_log <- lm(log(breaks + 1) ~ wool + tension, data = warpbreaks)
round(rbind(
  poisson_IRR = exp(coef(pm)),
  lm_log_ratio = exp(coef(lm_log))
), 3)

#>              (Intercept) woolB tensionM tensionH
#> poisson_IRR        40.12 0.814    0.725    0.595
#> lm_log_ratio       36.84 0.862    0.757    0.624
```

```r
## What each model says the average count is for wool A, low tension:
c(poisson = predict(pm, data.frame(wool = "A", tension = "L"), type = "response"),
  lm_log  = exp(predict(lm_log, data.frame(wool = "A", tension = "L"))) - 1,
  observed_mean = mean(warpbreaks$breaks[warpbreaks$wool == "A" &
                                         warpbreaks$tension == "L"]))
```

```
#>     poisson.1      lm_log.1 observed_mean
#>         40.12         35.84         44.56
```

The Poisson model reproduces the observed group mean exactly; the log-linear model does not, because it is estimating a different quantity.

Rule of thumb

> Use a Poisson-family GLM for counts. Reserve `log()` transformations for continuous positive outcomes such as concentrations, incomes or durations, where the multiplicative structure is real and there are no zeros.

5.6 Counts with a fixed upper limit

If a count has a natural ceiling — 7 correct answers out of 10, 3 seizures out of a maximum of 3 — it is not Poisson. It is binomial. The Poisson is for counts with no upper bound (or a bound so far away it never matters).

Situation	Distribution	R
Number of accidents on a road this year	Poisson	`family = poisson`
Number of students passing out of 30	Binomial	`cbind(pass, 30-pass)`
Number of ticks on a chick (no ceiling)	Poisson / NB	`poisson`, `glm.nb`
Number of days absent out of 365	Binomial (or Poisson if the count is small relative to 365)	either

5.7 Checking a Poisson fit quickly

Three checks take ten seconds and catch most problems.

```r
## 1. Dispersion: should be near 1
sum(residuals(pm, type = "pearson")^2) / df.residual(pm)
```

```
#> [1] 4.262
```

```r
## 2. Does the model reproduce the number of zeros in the data?
c(observed_zeros = sum(warpbreaks$breaks == 0),
  expected_zeros = sum(dpois(0, fitted(pm))))
```

```
#> observed_zeros expected_zeros
#>      0.000e+00      3.323e-08
```

```r
## 3. Do fitted and observed agree on average within groups?
round(tapply(warpbreaks$breaks, warpbreaks$tension, mean), 2)
```

```
#>     L     M     H
#> 36.39 26.39 21.67

round(tapply(fitted(pm),        warpbreaks$tension, mean), 2)

#>     L     M     H
#> 36.39 26.39 21.67
```

Check 1 already tells us the model is in trouble here: a dispersion of 4.26 is well above 1. Chapters 7–9 are entirely about what to do next.

The workflow from here

Fit the Poisson. Measure the dispersion. If it is close to 1, you are done. If it is well above 1, move to quasi-Poisson or negative binomial. If it is above 1 *and* there are far more zeros than the model predicts, consider zero-inflation or a hurdle model.

CHAPTER 6

Rates, exposure and offsets

6.1 The problem

Suppose two clinics report accident numbers. Clinic A saw 40 accidents, Clinic B saw 20. Which is more dangerous?

You cannot say — not until you know Clinic A serves 100,000 people and Clinic B serves 10,000. Then A's rate is 0.4 per thousand and B's is 2.0 per thousand, and B is five times worse.

This is the single most common serious mistake in applied count regression: modelling the raw count when the observations had different amounts of *exposure* — different follow-up times, different populations at risk, different areas, different numbers of trials.

Offset in one sentence

An offset is a predictor whose coefficient you refuse to estimate because you already know it must be exactly 1, which is what turns a model for counts into a model for rates.

6.2 The mathematics, which is short

You want to model the rate μ_i/t_i, where t_i is the exposure:

$$\log\left(\frac{\mu_i}{t_i}\right) = \beta_0 + \beta_1 x_i.$$

Split the log:

$$\log(\mu_i) - \log(t_i) = \beta_0 + \beta_1 x_i$$
$$\log(\mu_i) = \log(t_i) + \beta_0 + \beta_1 x_i.$$

That $\log(t_i)$ sits in the linear predictor like a covariate — but with its coefficient nailed to 1. That is precisely what `offset()` does.

Key idea

`offset(log(exposure))`. Always the *log* of the exposure, because the link is a log. Putting the raw exposure in an offset is a real and surprisingly common bug.

6.3 A simulated example where it matters

```
set.seed(42)
n       <- 400
region  <- factor(sample(c("urban", "rural"), n, replace = TRUE))
## Person-years of follow-up differ a lot -- and urban areas are bigger
pyears  <- round(exp(rnorm(n, mean = ifelse(region == "urban", 8.0, 6.5), sd = 0.6)))
## TRUE rate is 30% HIGHER in rural areas
true_rate <- ifelse(region == "rural", 0.0013, 0.0010)
```

```
events  <- rpois(n, lambda = true_rate * pyears)
dat     <- data.frame(events, pyears, region)
aggregate(cbind(events, pyears) ~ region, data = dat, FUN = sum)

#>   region events pyears
#> 1  rural    190 161076
#> 2  urban    715 688878
```

The truth we built in: the rural rate is 1.3 times the urban rate. Now fit the model two ways.

```
## WRONG: ignores that urban areas were watched for far longer
m_wrong <- glm(events ~ region, family = poisson, data = dat)

## RIGHT: models the rate per person-year
m_right <- glm(events ~ region + offset(log(pyears)), family = poisson, data = dat)

round(cbind(
  wrong_IRR = exp(coef(m_wrong)),
  right_IRR = exp(coef(m_right))
), 3)

#>             wrong_IRR right_IRR
#> (Intercept)     0.931     0.001
#> regionurban     3.917     0.880
```

The model without the offset reports a rural rate ratio of NA — it says rural areas are *safer*, and would be reported as strongly significant. The model with the offset recovers NA, close to the true 1.3. The first answer is not merely imprecise; it points the wrong way.

```
par(mfrow = c(1, 2)); par_book()
boxplot(events ~ region, data = dat, col = c(adjustcolor(PAL["primary"], 0.5),
        adjustcolor(PAL["accent"], 0.5)), border = c(PAL["primary"], PAL["accent"]),
        ylab = "raw event count", main = "Raw counts (misleading)")
grid_book()
boxplot(1000 * events / pyears ~ region, data = dat,
        col = c(adjustcolor(PAL["primary"], 0.5), adjustcolor(PAL["accent"], 0.5)),
        border = c(PAL["primary"], PAL["accent"]),
        ylab = "events per 1000 person-years", main = "Rates (correct comparison)")
grid_book()
```

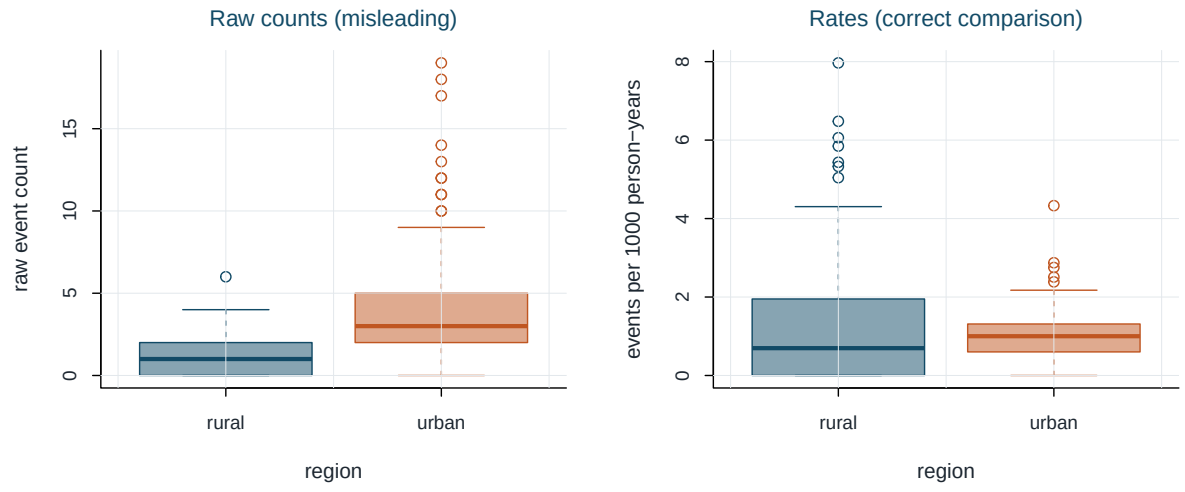


Figure 6.1: Left: raw counts make rural areas look safer, because they were observed for less time. Right: once expressed as rates per person-year, the true pattern appears.

6.4 The alternative syntax

R accepts the offset in the formula or as an argument. They are equivalent:

```
a <- glm(events ~ region + offset(log(pyears)), family = poisson, data = dat)
b <- glm(events ~ region, offset = log(pyears), family = poisson, data = dat)
all.equal(coef(a), coef(b))

#> [1] TRUE
```

Prefer the in-formula version

> Writing `offset(log(pyears))` inside the formula means `predict()` on new data automatically knows it needs a `pyears` column. With the separate `offset =` argument you must remember to supply it yourself at prediction time, and forgetting is silent.

6.5 Predicting from a model with an offset

Two different questions, two different answers.

```
## Q1: expected COUNT for a rural area observed for 2000 person-years
predict(m_right, newdata = data.frame(region = "rural", pyears = 2000),
        type = "response")

#>     1
#> 2.359

## Q2: the underlying RATE per person-year -- set exposure to 1
predict(m_right, newdata = data.frame(region = c("urban", "rural"), pyears = 1),
        type = "response")

#>        1        2
#> 0.001038 0.001180

## ... which is easier to read per 1000 person-years
1000 * predict(m_right, newdata = data.frame(region = c("urban", "rural"), pyears = 1),
               type = "response")

#>     1     2
#> 1.038 1.180
```

Setting exposure to 1 makes $\log(t) = 0$, which removes the offset from the linear predictor and leaves the pure rate.

6.6 When exposure is not time

Exposure means "how much opportunity did this unit have for the event?". It takes many forms:

Outcome	Exposure	Offset term
Hospital admissions per patient	follow-up time	`offset(log(years))`
Crimes per district	population size	`offset(log(pop))`
Defects per manufactured batch	batch size	`offset(log(n_items))`
Species counts per plot	plot area	`offset(log(area))`
Insurance claims per policy	policy-years	`offset(log(exposure))`
Website conversions per campaign	impressions served	`offset(log(impressions))`

6.7 Should the coefficient really be 1?

An offset assumes events scale exactly proportionally with exposure: double the population, double the accidents. That is usually right, but you can test it. Put the log-exposure in as an ordinary covariate and see whether its estimated coefficient is near 1:

```
m_free <- glm(events ~ region + log(pyears), family = poisson, data = dat)
round(coef(summary(m_free))["log(pyears)", ], 4)

#>   Estimate Std. Error    z value   Pr(>|z|)
#>     1.0310     0.0516    19.9628     0.0000

## Formal test: is the offset model (coefficient fixed at 1) adequate?
anova(m_right, m_free, test = "LRT")

#> Analysis of Deviance Table
#>
#> Model 1: events ~ region + offset(log(pyears))
#> Model 2: events ~ region + log(pyears)
#>   Resid. Df Resid. Dev Df Deviance Pr(>Chi)
#> 1       398        401
#> 2       397        401  1    0.361     0.55
```

The estimated coefficient is close to 1 and the likelihood-ratio test gives a large p-value, so fixing it at 1 is fine here.

If the coefficient is clearly not 1

> Then the relationship is not proportional — perhaps larger regions have economies of scale, or a saturating process. Leave `log(exposure)` in as a free covariate and say so. You are no longer modelling a simple rate, and you should explain what you are modelling instead.

6.8 A real-data example

The `epil` data records seizure counts for epilepsy patients. Each patient contributes a baseline count over 8 weeks and four follow-up counts over 2 weeks each. The exposure differs between those periods, so the counts are not comparable without an offset.

```
data(epil, package = "MASS")
head(epil[, c("y", "trt", "base", "age", "period", "subject")], 5)

#>   y     trt base age period subject
#> 1 5 placebo   11  31      1       1
```

```
#> 2 3 placebo   11  31      2       1
#> 3 3 placebo   11  31      3       1
#> 4 3 placebo   11  31      4       1
#> 5 3 placebo   11  30      1       2

## Model follow-up seizures per 2-week period, adjusting for the 8-week baseline
epil$lbase <- log(epil$base / 4)          # baseline rescaled to a 2-week rate
epil$lage  <- log(epil$age)
me <- glm(y ~ lbase + trt + lage, family = poisson, data = epil)
round(exp(cbind(IRR = coef(me), confint.default(me))), 3)

#>                IRR 2.5 % 97.5 %
#> (Intercept)  0.093 0.042  0.205
#> lbase        3.402 3.191  3.625
#> trtprogabide 0.983 0.895  1.081
#> lage         1.784 1.438  2.213
```

The treatment IRR of 0.983 suggests a small reduction in seizure rate on progabide. But these are repeated measures on the same patients, so these standard errors are not trustworthy — we return to this dataset properly in Chapter 15.

Offsets do not fix clustering

An offset handles *unequal opportunity*. It does nothing about *non-independence*. If the same patient appears five times in your data, you still need a random effect or a robust standard error on top of the offset.

CHAPTER 7

Overdispersion

If you remember one chapter from this book, make it this one. Overdispersion is the default state of real count data, and ignoring it produces confident, publishable, wrong conclusions.

7.1 What it is

The Poisson model insists that

$$\text{Var}(Y \mid x) = \text{E}(Y \mid x) = \mu.$$

Overdispersion means the data are more variable than that:

$$\text{Var}(Y \mid x) = \phi\mu, \qquad \phi > 1.$$

ϕ is the *dispersion parameter*. If $\phi = 1$ the Poisson is fine. If $\phi = 3$, your data bounce around three times as much as the Poisson allows.

The everyday version

> The Poisson assumes that once you know the average, you know exactly how much the counts wobble. Real data almost always wobble more, because there is something going on that you have not measured.

7.2 Why it happens

Five common causes, in rough order of frequency:

1. **Unobserved heterogeneity.** Individuals differ in ways you did not record. Some people are simply accident-prone. Mixing many different Poisson rates together produces something more variable than any single Poisson.
2. **Clustering / correlation.** Repeated measures on the same subject, pupils in classes, chicks in broods. Non-independence inflates variance.
3. **Contagion.** One event makes another more likely. Infections, traffic jams, cascading equipment failures.
4. **Excess zeros.** A subgroup that structurally cannot have the event at all (Chapter 10).
5. **Model misspecification.** A missing predictor, a missing interaction, or a non-linear effect fitted as linear. This one is worth ruling out *before* you reach for a fancier distribution.

Here is cause 1, shown directly. Mix Poissons with different rates and watch the variance overshoot the mean.

```
set.seed(3)
N <- 5000
y_pois <- rpois(N, lambda = 4)                    # everyone identical
```

```
lam    <- rgamma(N, shape = 0.7, rate = 0.7/4)        # rates vary between units
y_mix  <- rpois(N, lambda = lam)                      # a gamma-Poisson mixture = NB

par(mfrow = c(1, 2)); par_book()
h1 <- table(factor(y_pois, levels = 0:25))
barplot(h1 / N, col = adjustcolor(PAL["primary"], 0.75), border = "white",
        xlab = "count", ylab = "proportion",
        main = sprintf("One shared rate\nmean %.2f, variance %.2f",
                       mean(y_pois), var(y_pois)))
h2 <- table(factor(y_mix, levels = 0:25))
barplot(h2 / N, col = adjustcolor(PAL["accent"], 0.75), border = "white",
        xlab = "count", ylab = "proportion",
        main = sprintf("Rates differ between units\nmean %.2f, variance %.2f",
                       mean(y_mix), var(y_mix)))
```

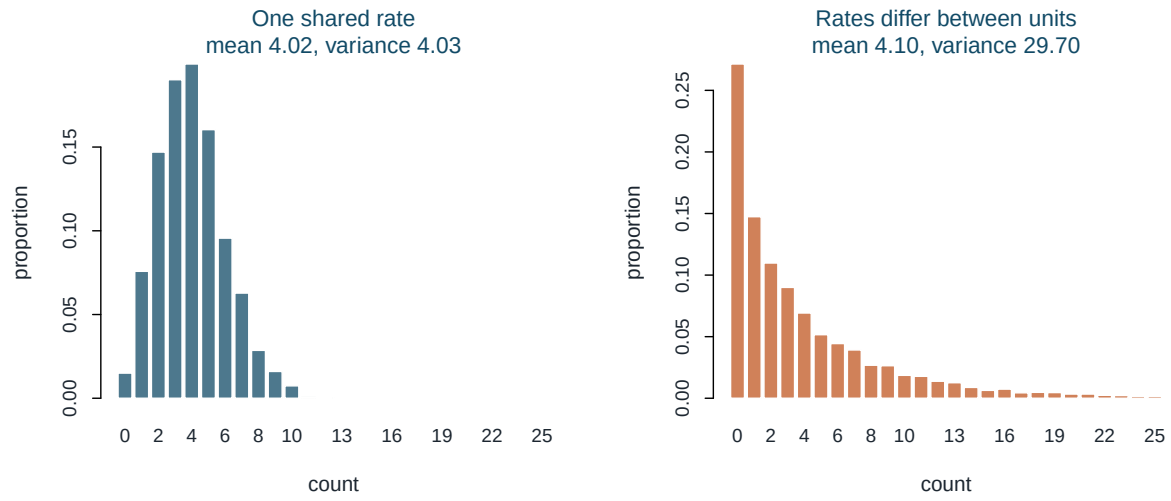


Figure 7.1: Both panels have the same overall mean. On the left every unit shares one rate, so variance equals mean. On the right the rate varies between units, and the resulting mixture has variance far above its mean.

Notice the right-hand panel has more zeros *and* a much longer tail, at the same average. That shape — extra zeros plus a long tail — is the signature of overdispersion.

Key idea

> A mixture of Poissons whose rates follow a gamma distribution *is* a negative binomial. That is not a coincidence or an analogy; it is the mathematical origin of the negative binomial, and the reason it is the natural repair for unobserved heterogeneity.

7.3 Why it matters: a simulation

Overdispersion barely moves the coefficients. It wrecks the standard errors. Here is the damage, measured.

```
set.seed(99)
sim_one <- function(n = 120, theta = 1) {
  x  <- rnorm(n)
  mu <- exp(1.5 + 0 * x)                       # TRUE effect of x is exactly zero
  y  <- rnbinom(n, mu = mu, size = theta)      # but the data are overdispersed
  mp <- glm(y ~ x, family = poisson)
```

```
  mq <- glm(y ~ x, family = quasipoisson)
  c(pois = coef(summary(mp))["x", "z value"],
    quasi = coef(summary(mq))["x", "t value"])
}
res <- t(replicate(1000, sim_one()))

## How often does each model declare a significant effect that does not exist?
round(c(poisson_type1_error = mean(abs(res[, "pois"])  > 1.96),
        quasi_type1_error   = mean(abs(res[, "quasi"]) > 1.96),
        nominal             = 0.05), 3)

#> poisson_type1_error   quasi_type1_error             nominal
#>               0.404               0.061               0.050

par(mfrow = c(1, 2)); par_book()
for (j in 1:2) {
  hist(res[, j], breaks = 40, freq = FALSE, border = "white",
       col = adjustcolor(c(PAL["red"], PAL["green"])[j], 0.6),
       xlim = c(-6, 6), ylim = c(0, 0.45), xlab = "test statistic for x",
       main = c("Poisson: too confident", "Quasi-Poisson: honest")[j])
  curve(dnorm(x), add = TRUE, lwd = 2.5, col = PAL["primary"])
  abline(v = c(-1.96, 1.96), lty = 2, col = PAL["grey"])
}
```

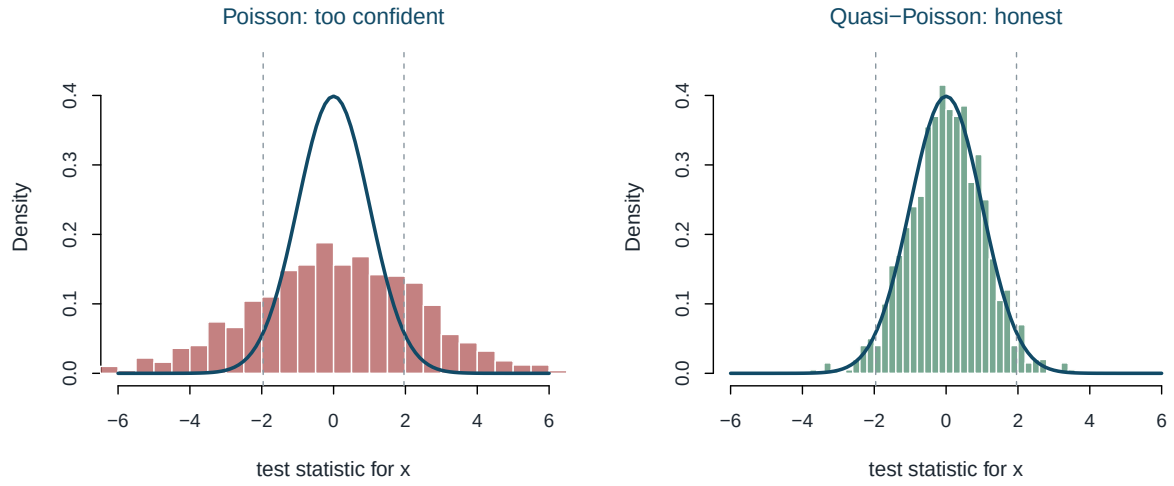


Figure 7.2: 1000 simulated datasets with NO true effect of x. The Poisson model's z-statistics are far too spread out, so it rejects the null far too often. The quasi-Poisson model is calibrated.

The Poisson model rejects a true null hypothesis about 40% of the time instead of 5%. That is not a small distortion. It means roughly 8.1 times as many false discoveries as advertised.

Watch out

Overdispersion makes standard errors too small, confidence intervals too narrow, and p-values too small. It almost never biases the coefficients themselves. So a model that looks great — everything significant! — is exactly the pattern overdispersion produces.

7.4 Detecting it

7.4.1 Method 1: the Pearson dispersion statistic

The workhorse. Compute

$$\hat{\phi} = \frac{1}{n-p}\sum_i \frac{(y_i - \hat{\mu}_i)^2}{\hat{\mu}_i} = \frac{X^2}{\text{residual df}}.$$

```
data(quine, package = "MASS")
str(quine)

#> 'data.frame': 146 obs. of  5 variables:
#>  $ Eth : Factor w/ 2 levels "A","N": 1 1 1 1 1 1 1 1 1 1 ...
#>  $ Sex : Factor w/ 2 levels "F","M": 2 2 2 2 2 2 2 2 2 2 ...
#>  $ Age : Factor w/ 4 levels "F0","F1","F2",..: 1 1 1 1 1 1 1 1 2 2 ...
#>  $ Lrn : Factor w/ 2 levels "AL","SL": 2 2 2 1 1 1 1 1 2 2 ...
#>  $ Days: int  2 11 14 5 5 13 20 22 6 6 ...

m_pois <- glm(Days ~ Eth + Sex + Age + Lrn, family = poisson, data = quine)

dispersion <- function(model) {
  X2 <- sum(residuals(model, type = "pearson")^2)
  df <- df.residual(model)
  c(pearson_X2 = X2, df = df, dispersion = X2 / df,
    p_value = pchisq(X2, df, lower.tail = FALSE))
}
round(dispersion(m_pois), 4)

#> pearson_X2         df dispersion    p_value
#>    1830.19     139.00      13.17       0.00
```

A dispersion of 13.2 is severe. The counts vary about 13 times more than a Poisson permits, so the standard errors printed by `glm` are too small by a factor of roughly $\sqrt{13.2} \approx 3.6$.

7.4.2 Method 2: residual deviance over degrees of freedom

You will see this reported constantly, because it is right there in the `summary` output.

```
c(residual_deviance = deviance(m_pois), df = df.residual(m_pois),
  ratio = deviance(m_pois) / df.residual(m_pois))

#> residual_deviance                df             ratio
#>           1696.71            139.00             12.21
```

Use the Pearson version, not the deviance version

> The deviance/df ratio is convenient but biased, especially when many fitted means are small (under about 5). It can suggest overdispersion when there is none. The Pearson statistic is the one with the theory behind it. When they disagree, believe Pearson.

7.4.3 Method 3: a formal test

`AER::dispersiontest` tests $H_0 : \text{Var} = \mu$ against $\text{Var} = \mu + \alpha\, f(\mu)$.

```
library(AER)
dispersiontest(m_pois)                    # tests Var = mu + alpha*mu   (NB1-style)

#>
#>  Overdispersion test
#>
#> data:  m_pois
#> z = 5.5, p-value = 2e-08
#> alternative hypothesis: true dispersion is greater than 1
#> sample estimates:
#> dispersion
#>      12.53

dispersiontest(m_pois, trafo = 2)         # tests Var = mu + alpha*mu^2 (NB2-style)

#>
#>  Overdispersion test
#>
#> data:  m_pois
#> z = 5.3, p-value = 5e-08
#> alternative hypothesis: true alpha is greater than 0
#> sample estimates:
#>  alpha
#> 0.6362
```

Both reject decisively. The `trafo = 2` version is the one that corresponds to the negative binomial you get from `glm.nb`.

7.4.4 Method 4: look at it

Numbers are good; pictures are better. Plot squared residuals against fitted values, and compare the observed count distribution with what the model expects.

```
par(mfrow = c(1, 2)); par_book()
pr2 <- residuals(m_pois, type = "pearson")^2
plot(fitted(m_pois), pr2, pch = 21, bg = adjustcolor(PAL["primary"], 0.35),
     col = "white", cex = 1.1, log = "y",
     xlab = "fitted mean", ylab = "squared Pearson residual",
     main = "Squared Pearson residuals")
grid_book(); abline(h = 1, lwd = 2.5, col = PAL["green"])
lines(lowess(fitted(m_pois), pr2), lwd = 2.5, col = PAL["red"])
legend("topright", bty = "n", cex = 0.75, lwd = 2.5,
       col = c(PAL["green"], PAL["red"]),
       legend = c("Poisson expects 1", "actual average"))

## Observed vs expected frequency of each count
maxk <- 40
obs  <- table(factor(pmin(quine$Days, maxk), levels = 0:maxk))
expd <- sapply(0:maxk, function(k) sum(dpois(k, fitted(m_pois))))
plot(0:maxk, as.numeric(obs), type = "h", lwd = 3, col = PAL["primary"],
     xlab = "days absent", ylab = "frequency",
     main = "Observed vs Poisson-expected counts")
grid_book()
lines(0:maxk, expd, lwd = 2.5, col = PAL["red"])
points(0:maxk, expd, pch = 19, cex = 0.5, col = PAL["red"])
legend("topright", bty = "n", cex = 0.8, lwd = c(3, 2.5),
       col = c(PAL["primary"], PAL["red"]), legend = c("observed", "Poisson expects"))
```

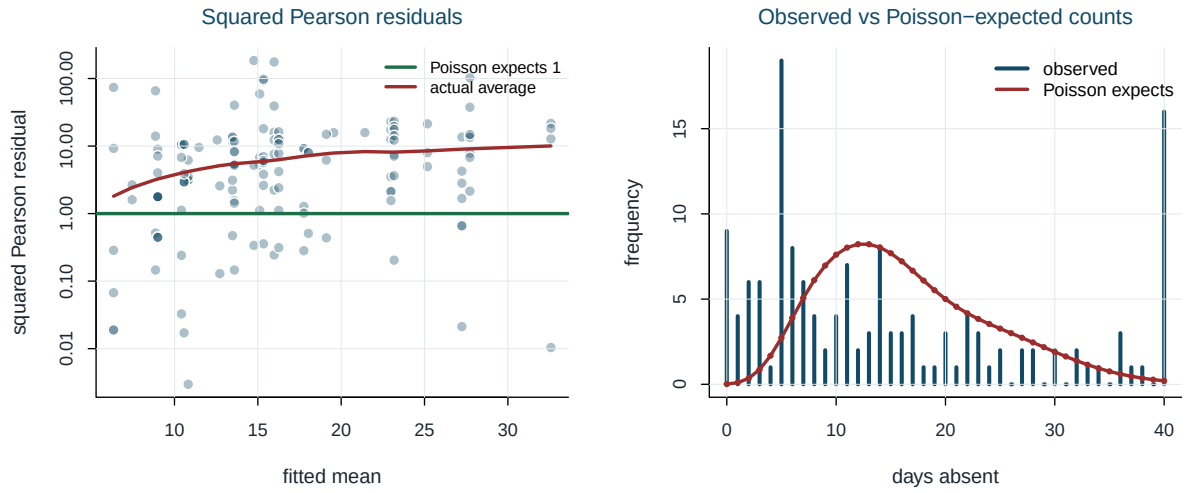


Figure 7.3: Left: if the Poisson held, the average squared Pearson residual would sit on the horizontal line at 1. Right: the Poisson model badly under-predicts both the zeros and the long tail.

7.4.5 Method 5: the simplest check of all

Does the model reproduce the number of zeros, and the size of the biggest values?

```
c(observed_zeros = sum(quine$Days == 0),
  poisson_expects = round(sum(dpois(0, fitted(m_pois))), 1))

#>  observed_zeros poisson_expects
#>               9               0

c(observed_max = max(quine$Days),
  poisson_99th_percentile = round(max(qpois(0.99, fitted(m_pois)))))

#>            observed_max poisson_99th_percentile
#>                      81                      47
```

The Poisson expects essentially no zeros and cannot produce values anywhere near the observed maximum. That is a comprehensive failure.

7.5 Rule out the boring explanation first

Before concluding "the distribution is wrong", check that the *mean model* is right. Overdispersion is often just a missing term.

```
set.seed(21)
n <- 300
x <- runif(n, 0, 3)
z <- rbinom(n, 1, 0.5)                      # an important variable...
y <- rpois(n, exp(0.4 + 0.6 * x + 1.4 * z))  # ...that really matters

## Leave z out: the data look overdispersed
round(dispersion(glm(y ~ x, family = poisson))["dispersion"], 3)

#> dispersion
#>      4.584
```

```r
## Put z in: the "overdispersion" disappears
round(dispersion(glm(y ~ x + z, family = poisson))["dispersion"], 3)


#> dispersion
#>      0.983
```

The data were generated from a perfect Poisson. Omitting a real predictor manufactured a dispersion of 4.58 out of nothing.

Checklist before switching distributions

1. Have you included all the predictors you have?
2. Should any continuous predictor be curved (a spline, a log, a quadratic)?
3. Are there interactions you have ignored?
4. Do you need an offset for unequal exposure?
5. Are there a handful of extreme outliers driving everything?
6. Are the observations clustered?

Only after all six should you conclude the distribution itself is wrong.

7.6 Your options once overdispersion is real

Option	R	When to use it
Quasi-Poisson	`family = quasipoisson`	Quick fix; you only want correct SEs. Variance $= \phi\mu$. No AIC, no likelihood.
Robust (sandwich) SEs	`coeftest(m, vcov = sandwich)`	Same idea, keeps the Poisson likelihood. Good with clustering too.
Negative binomial	`MASS::glm.nb`	The standard answer. Variance $= \mu + \mu^2/\theta$. A real likelihood, so AIC and LRTs work.
Observation-level random effect	`glmer(...+ (1|obs))`	Poisson-lognormal mixture; natural inside a GLMM.
Zero-inflated / hurdle	`zeroinfl`, `hurdle`	Only if the excess is specifically in the zeros.

7.7 Underdispersion

Less common, but it happens: $\hat{\phi} < 1$, data *less* variable than Poisson. Typical causes are a bounded or regulated process (a clinic that always books between 8 and 12 patients), or counts that were rounded or capped.

```r
set.seed(5)
y_under <- rbinom(200, size = 12, prob = 0.5)      # mean 6, variance only 3
mu_ <- glm(y_under ~ 1, family = poisson)
round(dispersion(mu_)["dispersion"], 3)


#> dispersion
#>      0.493
```

Consequences run the other way: standard errors are too *large*, so you lose power but stay conservative. Options are quasi-Poisson (which handles $\phi < 1$ perfectly well), a binomial model if there is a real upper bound, or a Conway–Maxwell–Poisson model (Chapter 11).

The one-line summary of this chapter

Fit the Poisson, compute `sum(residuals(m, type="pearson")^2) / df.residual(m)`, and act on the answer. Near 1: keep it. Above about 1.2: fix it. Above 2: you must fix it.

CHAPTER 8

Quasi-Poisson and robust standard errors

Two repairs that keep your Poisson coefficients and only fix the uncertainty around them. Both are quick; neither requires you to believe a new distribution.

8.1 Quasi-Poisson

The quasi-Poisson does not assume a distribution at all. It assumes only two things:

$$\mathrm{E}(Y_i) = \mu_i = e^{\eta_i}, \qquad \mathrm{Var}(Y_i) = \phi\,\mu_i.$$

That is called a *quasi-likelihood*: enough structure to estimate coefficients and standard errors, not enough to be a probability model.

ϕ is estimated from the Pearson statistic, and every standard error is multiplied by $\sqrt{\hat{\phi}}$.

```
m_quasi <- glm(Days ~ Eth + Sex + Age + Lrn, family = quasipoisson, data = quine)
summary(m_quasi)


#>
#> Call:
#> glm(formula = Days ~ Eth + Sex + Age + Lrn, family = quasipoisson,
#>     data = quine)
#>
#> Coefficients:
#>             Estimate Std. Error t value Pr(>|t|)
#> (Intercept)    2.715      0.235   11.57   <2e-16 ***
#> EthN          -0.534      0.152   -3.51   0.0006 ***
#> SexM           0.162      0.154    1.05   0.2969
#> AgeF1         -0.334      0.254   -1.31   0.1914
#> AgeF2          0.258      0.226    1.14   0.2569
#> AgeF3          0.428      0.246    1.74   0.0838 .
#> LrnSL          0.349      0.189    1.85   0.0668 .
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> (Dispersion parameter for quasipoisson family taken to be 13.17)
#>
#>     Null deviance: 2073.5  on 145  degrees of freedom
#> Residual deviance: 1696.7  on 139  degrees of freedom
#> AIC: NA
#>
#> Number of Fisher Scoring iterations: 5
```

Compare the two fits side by side:

```
cmp <- cbind(
  estimate      = coef(m_pois),
  se_poisson    = coef(summary(m_pois))[, "Std. Error"],
  se_quasi      = coef(summary(m_quasi))[, "Std. Error"],
  p_poisson     = coef(summary(m_pois))[, 4],
  p_quasi       = coef(summary(m_quasi))[, 4]
)
round(cmp, 4)
```

```
#>             estimate se_poisson se_quasi p_poisson p_quasi
#> (Intercept)   2.7154     0.0647   0.2347    0.0000  0.0000
#> EthN         -0.5336     0.0419   0.1520    0.0000  0.0006
#> SexM          0.1616     0.0425   0.1543    0.0001  0.2969
#> AgeF1        -0.3339     0.0701   0.2543    0.0000  0.1914
#> AgeF2         0.2578     0.0624   0.2265    0.0000  0.2569
#> AgeF3         0.4277     0.0677   0.2456    0.0000  0.0838
#> LrnSL         0.3489     0.0520   0.1888    0.0000  0.0668
```

```
## The ratio of the standard errors is exactly sqrt(phi)
round(c(se_ratio = mean(cmp[, "se_quasi"] / cmp[, "se_poisson"]),
        sqrt_phi = sqrt(summary(m_quasi)$dispersion)), 4)
```

```
#> se_ratio sqrt_phi
#>    3.629    3.629
```

Key idea

Quasi-Poisson changes **nothing** about the estimates and multiplies **every** standard error by the same constant $\sqrt{\hat{\phi}}$. Here that constant is 3.63, which is why several predictors that looked significant no longer are.

```
par_book()
k   <- 2:nrow(cmp)
est <- cmp[k, "estimate"]
ylim <- range(c(est - 2*cmp[k, "se_quasi"], est + 2*cmp[k, "se_quasi"]))
plot(seq_along(k) - 0.1, est, pch = 19, col = PAL["primary"], cex = 1.2,
     xlim = c(0.5, length(k) + 0.5), ylim = ylim, xaxt = "n",
     xlab = "", ylab = "coefficient (log scale)",
     main = "Poisson vs quasi-Poisson intervals")
grid_book(); abline(h = 0, lty = 2, col = PAL["grey"], lwd = 1.5)
arrows(seq_along(k) - 0.1, est - 1.96*cmp[k, "se_poisson"],
       seq_along(k) - 0.1, est + 1.96*cmp[k, "se_poisson"],
       code = 3, angle = 90, length = 0.04, col = PAL["primary"], lwd = 2)
points(seq_along(k) + 0.1, est, pch = 19, col = PAL["accent"], cex = 1.2)
arrows(seq_along(k) + 0.1, est - 1.96*cmp[k, "se_quasi"],
       seq_along(k) + 0.1, est + 1.96*cmp[k, "se_quasi"],
       code = 3, angle = 90, length = 0.04, col = PAL["accent"], lwd = 2)
axis(1, at = seq_along(k), labels = rownames(cmp)[k], cex.axis = 0.75)
legend("topright", bty = "n", pch = 19, cex = 0.8,
       col = c(PAL["primary"], PAL["accent"]),
       legend = c("Poisson", "quasi-Poisson"))
```

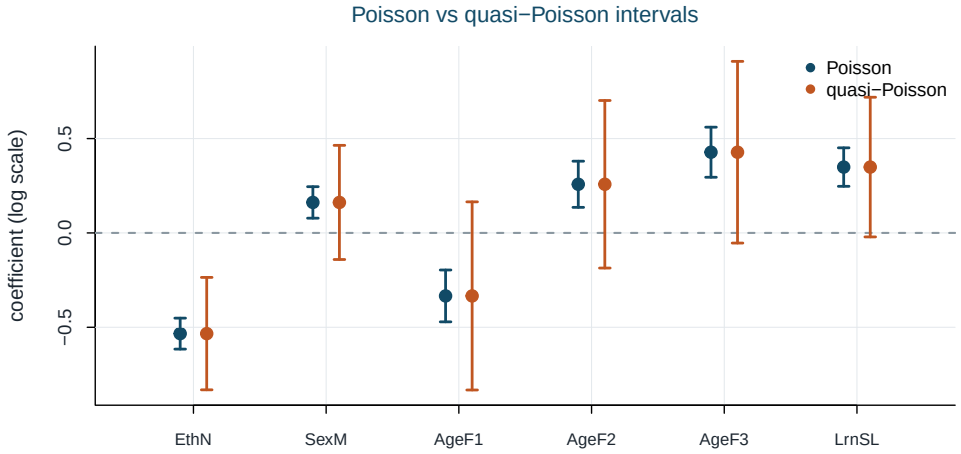


Figure 8.1: The same coefficients with Poisson and quasi-Poisson confidence intervals. The estimates do not move; the intervals widen by a factor of sqrt(phi).

8.1.1 What you give up

```
AIC(m_quasi)

#> [1] NA

logLik(m_quasi)

#> 'log Lik.' NA (df=8)
```

There is no likelihood, so there is no AIC, no BIC, and no likelihood-ratio test. This is the real cost of quasi-likelihood, and it is why model selection with quasi-Poisson is awkward.

The substitute is an F-test, which `anova` will do if you ask:

```
anova(m_quasi, test = "F")

#> Analysis of Deviance Table
#>
#> Model: quasipoisson, link: log
#>
#> Response: Days
#>
#> Terms added sequentially (first to last)
#>
#>
#>      Df Deviance Resid. Df Resid. Dev     F Pr(>F)
#> NULL                  145       2074
#> Eth   1    181.6       144       1892 13.79 0.0003 ***
#> Sex   1     16.9       143       1875  1.28 0.2594
#> Age   3    132.6       140       1743  3.36 0.0208 *
#> Lrn   1     45.8       139       1697  3.48 0.0643 .
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

There is also a quasi-AIC (QAIC), which divides the Poisson log-likelihood by $\hat{\phi}$ and adds one for the estimated dispersion:

```
QAIC <- function(mod_pois, phi, k_extra = 1) {
  k <- length(coef(mod_pois)) + k_extra
  -2 * as.numeric(logLik(mod_pois)) / phi + 2 * k
}
phi <- summary(m_quasi)$dispersion
c(QAIC_full = QAIC(m_pois, phi),
  QAIC_reduced = QAIC(update(m_pois, . ~ . - Sex), phi))


#>    QAIC_full QAIC_reduced
#>        189.6        188.6
```

QAIC needs a fixed phi

> When comparing several models with QAIC, estimate $\hat{\phi}$ *once* from the largest sensible model and use that same value for all of them. Re-estimating ϕ in every model makes the numbers incomparable.

8.2 Robust (sandwich) standard errors

A different route to the same destination. Keep the Poisson model, but compute the standard errors in a way that does not trust the variance assumption at all.

The name comes from the shape of the formula:

$$V = \underbrace{\underbrace{A^{-1}}_{\text{bread}}\ \underbrace{B}_{\text{meat}}\ \underbrace{A^{-1}}_{\text{bread}}}_{\text{sandwich}}$$

where A is what the model *claims* the information is and B is what the residuals *actually* show.

```
library(sandwich); library(lmtest)
coeftest(m_pois, vcov. = sandwich)


#>
#> z test of coefficients:
#>
#>             Estimate Std. Error z value Pr(>|z|)
#> (Intercept)    2.715      0.235   11.54   <2e-16 ***
#> EthN          -0.534      0.153   -3.48   0.0005 ***
#> SexM           0.162      0.155    1.04   0.2972
#> AgeF1         -0.334      0.267   -1.25   0.2105
#> AgeF2          0.258      0.250    1.03   0.3018
#> AgeF3          0.428      0.248    1.73   0.0842 .
#> LrnSL          0.349      0.188    1.86   0.0628 .
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
round(cbind(
  naive_poisson = coef(summary(m_pois))[, "Std. Error"],
  quasi_poisson = coef(summary(m_quasi))[, "Std. Error"],
  robust_HC0    = sqrt(diag(sandwich(m_pois)))
), 4)
```

```
#>             naive_poisson quasi_poisson robust_HC0
#> (Intercept)        0.0647        0.2347     0.2353
#> EthN               0.0419        0.1520     0.1533
#> SexM               0.0425        0.1543     0.1550
#> AgeF1              0.0701        0.2543     0.2667
#> AgeF2              0.0624        0.2265     0.2497
#> AgeF3              0.0677        0.2456     0.2477
#> LrnSL              0.0520        0.1888     0.1875
```

Notice that quasi-Poisson scales all standard errors by the same factor, whereas the sandwich adjusts each one differently, according to where in the data the extra variability actually lives.

Which of the two should I use?

Quasi-Poisson when the extra variability is roughly proportional to the mean everywhere. Sandwich when it is not, or when you have no idea. The sandwich is more general and assumes less; the price is that it needs a decent sample size (say 50+ observations, more with many predictors) to be reliable.

8.2.1 Clustered robust standard errors

The sandwich has a superpower: it can also absorb clustering. If observations come in groups, use a cluster-robust variance.

```
data(epil, package = "MASS")
epil$lbase <- log(epil$base / 4)
me <- glm(y ~ lbase + trt, family = poisson, data = epil)
round(cbind(
  naive     = sqrt(diag(vcov(me))),
  robust    = sqrt(diag(sandwich(me))),
  clustered = sqrt(diag(vcovCL(me, cluster = epil$subject)))
), 4)

#>               naive robust clustered
#> (Intercept)  0.0817 0.2210    0.3678
#> lbase        0.0309 0.0879    0.1496
#> trtprogabide 0.0453 0.1244    0.1953
```

The clustered standard errors are several times larger — because the 236 rows are really only 59 independent patients measured repeatedly.

When cluster-robust SEs are enough

They work well when you have many clusters (roughly 40+) and you only care about population-average effects. With few clusters they are unreliable, and if you want to *describe* the between-cluster variation rather than just adjust for it, you need a mixed model (Chapters 14–15).

8.3 Quasi-Poisson versus negative binomial

Both fix overdispersion; they make different assumptions about *how* the variance grows.

$$\text{Quasi-Poisson: } \operatorname{Var} = \phi\mu \quad \text{(linear in } \mu\text{)} \qquad \text{Negative binomial: } \operatorname{Var} = \mu + \frac{\mu^2}{\theta} \quad \text{(quadratic)}$$

```
par_book()
mu <- seq(0.5, 40, length.out = 300)
plot(mu, mu, type = "l", lwd = 2.5, col = PAL["grey"], log = "y",
     ylim = c(0.5, 3000), xlab = expression(mu ~ "(mean)"),
     ylab = "variance (log scale)", main = "Variance functions compared")
grid_book()
lines(mu, 4 * mu,           lwd = 3, col = PAL["primary"])
lines(mu, mu + mu^2 / 2,    lwd = 3, col = PAL["accent"])
lines(mu, mu + mu^2 / 0.5,  lwd = 3, lty = 2, col = PAL["red"])
legend("topleft", bty = "n", cex = 0.8, lwd = c(2.5, 3, 3, 3), lty = c(1,1,1,2),
       col = c(PAL["grey"], PAL["primary"], PAL["accent"], PAL["red"]),
       legend = c("Poisson: mu", "quasi-Poisson: 4mu",
                  "NB, theta = 2", "NB, theta = 0.5"))
```

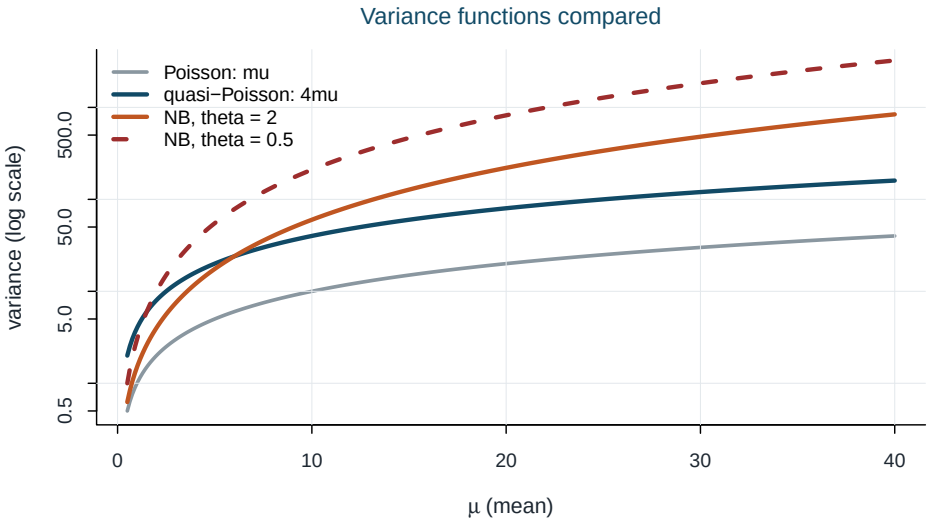


Figure 8.2: How the two repairs assume the variance grows with the mean. They agree in the middle of the data and disagree in the tails, which is where they weight observations differently.

The practical difference is in **weighting**. Quasi-Poisson gives large counts relatively more weight; the negative binomial, whose variance grows faster, downweights them. When your data contain a few very large counts, the two models can give noticeably different coefficients.

	Quasi-Poisson	**Negative binomial**
Variance	$\phi\mu$	$\mu + \mu^2/\theta$
Full probability model	no	yes
AIC / BIC available	no	yes
Likelihood-ratio tests	no (use F)	yes
Predicts a distribution	no	yes
Handles underdispersion	yes ($\phi < 1$)	no
Weighting of large counts	relatively more	relatively less
Extends to mixed models	awkward	easily (`glmmTMB`)

Rule of thumb

Default to the negative binomial. Choose quasi-Poisson if you have underdispersion, if you want the estimates to stay exactly at the Poisson values, or if you specifically believe variance is linear in the mean. If they disagree substantially, plot the squared residuals against the fitted values and see which variance function matches the data.

CHAPTER 9

Negative binomial regression

9.1 Where it comes from

Chapter 7 showed that mixing Poisson distributions with varying rates produces something more variable than a Poisson. Make that precise: let each unit have its own rate λ_i, drawn from a gamma distribution with mean μ_i, and let the count be Poisson given that rate:

$$Y_i \mid \lambda_i \sim \text{Poisson}(\lambda_i), \qquad \lambda_i \sim \text{Gamma}(\theta,\ \theta/\mu_i).$$

Integrate out λ_i and you get exactly the negative binomial:

$$\text{E}(Y_i) = \mu_i, \qquad \text{Var}(Y_i) = \mu_i + \frac{\mu_i^2}{\theta}.$$

The interpretation of theta

θ measures how much units differ from one another beyond what the predictors explain. Large θ (say above 20) means units are nearly interchangeable and the model collapses back to a Poisson. Small θ (below 1) means huge unexplained heterogeneity — lots of zeros and a very long tail.

```
par(mfrow = c(1, 4)); par_book()
for (th in c(50, 5, 1, 0.3)) {
  k <- 0:30
  barplot(dnbinom(k, mu = 5, size = th), names.arg = k, border = "white",
          col = adjustcolor(PAL["accent"], 0.75), cex.names = 0.55,
          ylim = c(0, 0.45), main = bquote(theta == .(th)),
          xlab = "count", ylab = if (th == 50) "probability" else "")
}
```

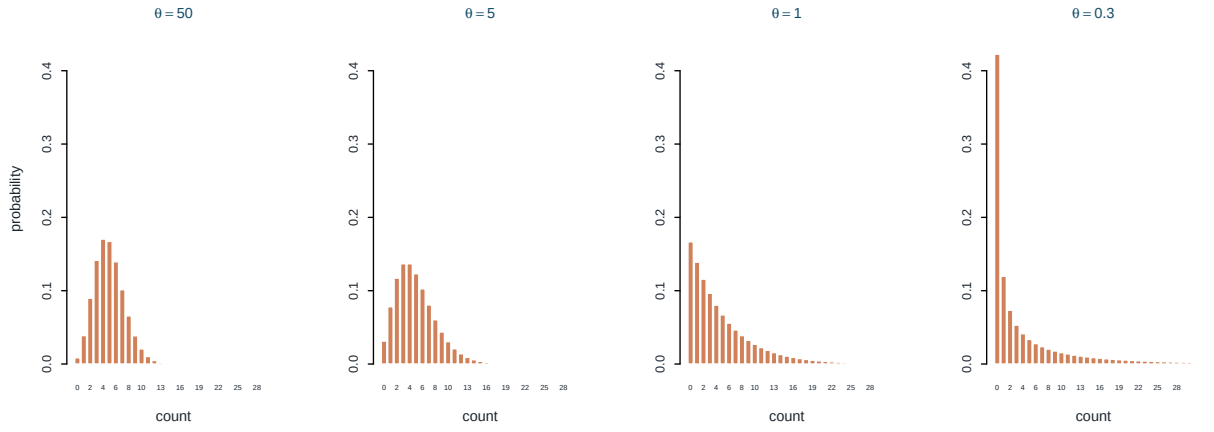


Figure 9.1: The same mean of 5 under four values of theta. As theta shrinks, zeros pile up and the tail stretches out.

Names for the same parameter

R calls it `theta` (MASS) or `size` (`rnbinom`, `dnbinom`). Stata calls its reciprocal `alpha`, so $\alpha = 1/\theta$. Some textbooks use k. They all describe the same thing, but *small* θ means the same as *large* α. Check which convention a paper is using before comparing numbers.

9.2 Fitting it

```
library(MASS)
data(quine)
m_nb <- glm.nb(Days ~ Eth + Sex + Age + Lrn, data = quine)
summary(m_nb)


#>
#> Call:
#> glm.nb(formula = Days ~ Eth + Sex + Age + Lrn, data = quine,
#>     init.theta = 1.274892646, link = log)
#>
#> Coefficients:
#>             Estimate Std. Error z value Pr(>|z|)
#> (Intercept)   2.8946     0.2284   12.67   <2e-16 ***
#> EthN         -0.5694     0.1533   -3.71   0.0002 ***
#> SexM          0.0823     0.1599    0.51   0.6067
#> AgeF1        -0.4484     0.2397   -1.87   0.0614 .
#> AgeF2         0.0881     0.2362    0.37   0.7092
#> AgeF3         0.3569     0.2483    1.44   0.1507
#> LrnSL         0.2921     0.1865    1.57   0.1172
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> (Dispersion parameter for Negative Binomial(1.275) family taken to be 1)
#>
#>     Null deviance: 195.29  on 145  degrees of freedom
#> Residual deviance: 167.95  on 139  degrees of freedom
#> AIC: 1109
#>
#> Number of Fisher Scoring iterations: 1
#>
#>
#>               Theta:  1.275
#>           Std. Err.:  0.161
#>
#>  2 x log-likelihood:  -1093.151
```

The important extra line is `Theta` at the bottom, with its own standard error.

```
c(theta = m_nb$theta, se = m_nb$SE.theta, alpha = 1 / m_nb$theta)


#>  theta     se  alpha
#> 1.2749 0.1610 0.7844
```

With $\hat{\theta} = 1.275$, the variance function is $\mu + \mu^2/1.27$. At a fitted mean of 20, that is a variance of 334 instead of the Poisson's 20 — roughly 17 times as variable.

9.3 Poisson versus negative binomial, side by side

```
m_pois  <- glm(Days ~ Eth + Sex + Age + Lrn, family = poisson, data = quine)
m_quasi <- glm(Days ~ Eth + Sex + Age + Lrn, family = quasipoisson, data = quine)

round(cbind(
  est_poisson = coef(m_pois),
  est_nb      = coef(m_nb),
  se_poisson  = coef(summary(m_pois))[, 2],
  se_quasi    = coef(summary(m_quasi))[, 2],
  se_nb       = coef(summary(m_nb))[, 2]
), 4)
```

```
#>             est_poisson  est_nb se_poisson se_quasi  se_nb
#> (Intercept)      2.7154  2.8946     0.0647   0.2347 0.2284
#> EthN            -0.5336 -0.5694     0.0419   0.1520 0.1533
#> SexM             0.1616  0.0823     0.0425   0.1543 0.1599
#> AgeF1           -0.3339 -0.4484     0.0701   0.2543 0.2397
#> AgeF2            0.2578  0.0881     0.0624   0.2265 0.2362
#> AgeF3            0.4277  0.3569     0.0677   0.2456 0.2483
#> LrnSL            0.3489  0.2921     0.0520   0.1888 0.1865
```

Two things to notice. The coefficients move a little (the NB reweights the large counts), and the standard errors roughly double or triple.

9.3.1 Is the negative binomial actually better? The likelihood-ratio test

The Poisson is the negative binomial in the limit $\theta \to \infty$, so the two are nested and we can test between them:

```
LR <- 2 * (as.numeric(logLik(m_nb)) - as.numeric(logLik(m_pois)))
c(LR_statistic = LR,
  naive_p = pchisq(LR, df = 1, lower.tail = FALSE),
  correct_p = 0.5 * pchisq(LR, df = 1, lower.tail = FALSE))
```

```
#> LR_statistic      naive_p    correct_p
#>    1.192e+03   3.287e-261   1.644e-261
```

Why the p-value is halved

> Under H_0 the extra parameter sits on the *boundary* of its parameter space ($1/\theta = 0$ is the edge). The null distribution is then a 50:50 mixture of χ^2_0 and χ^2_1, so the correct p-value is half the naive one. In practice the evidence is usually so overwhelming that it makes no difference — but the halving is the technically right answer and it matters in borderline cases.

pscl packages this test for you:

```
pscl::odTest(m_nb)
```

```
#> Likelihood ratio test of H0: Poisson, as restricted NB model:
#> n.b., the distribution of the test-statistic under H0 is non-standard
#> e.g., see help(odTest) for details/references
#>
#> Critical value of test statistic at the alpha= 0.05 level: 2.706
#> Chi-Square Test Statistic =  1192.033 p-value = <2e-16
```

9.3.2 Compare on AIC too

```
round(c(AIC_poisson = AIC(m_pois), AIC_negbin = AIC(m_nb),
        BIC_poisson = BIC(m_pois), BIC_negbin = BIC(m_nb)), 1)


#> AIC_poisson  AIC_negbin BIC_poisson  BIC_negbin
#>        2299        1109        2320        1133
```

A drop of 1190 AIC points is not a close call.

9.4 Does it fit? Compare the predicted distributions

The sharpest check for a count model is whether it can reproduce the shape of the data.

```
par(mfrow = c(1, 2)); par_book()
maxk <- 45
obs  <- as.numeric(table(factor(pmin(quine$Days, maxk), levels = 0:maxk)))
e_p  <- sapply(0:maxk, function(k) sum(dpois(k, fitted(m_pois))))
e_nb <- sapply(0:maxk, function(k) sum(dnbinom(k, mu = fitted(m_nb), size = m_nb$theta)))

plot(0:maxk, obs, type = "h", lwd = 3, col = adjustcolor(PAL["primary"], 0.6),
     xlab = "days absent", ylab = "frequency", main = "Observed vs expected")
grid_book()
lines(0:maxk, e_p,  lwd = 2.5, col = PAL["red"])
lines(0:maxk, e_nb, lwd = 2.5, col = PAL["green"])
legend("topright", bty = "n", cex = 0.8, lwd = c(3, 2.5, 2.5),
       col = c(adjustcolor(PAL["primary"], 0.6), PAL["red"], PAL["green"]),
       legend = c("observed", "Poisson", "negative binomial"))

## Hanging rootogram: bars hang from the fitted curve on a square-root scale
rootogram <- function(obs, exp_, main) {
  plot(range(0:(length(obs)-1)), c(-2.5, max(sqrt(exp_)) * 1.15), type = "n",
       xlab = "count", ylab = expression(sqrt("frequency")), main = main)
  grid_book()
  rect(0:(length(obs)-1) - 0.4, sqrt(exp_) - sqrt(obs),
       0:(length(obs)-1) + 0.4, sqrt(exp_),
       col = adjustcolor(PAL["primary"], 0.45), border = "white")
  abline(h = 0, lwd = 2, col = PAL["grey"])
  lines(0:(length(obs)-1), sqrt(exp_), lwd = 2.5, col = PAL["red"])
}
rootogram(obs, e_nb, "Rootogram: negative binomial")
```

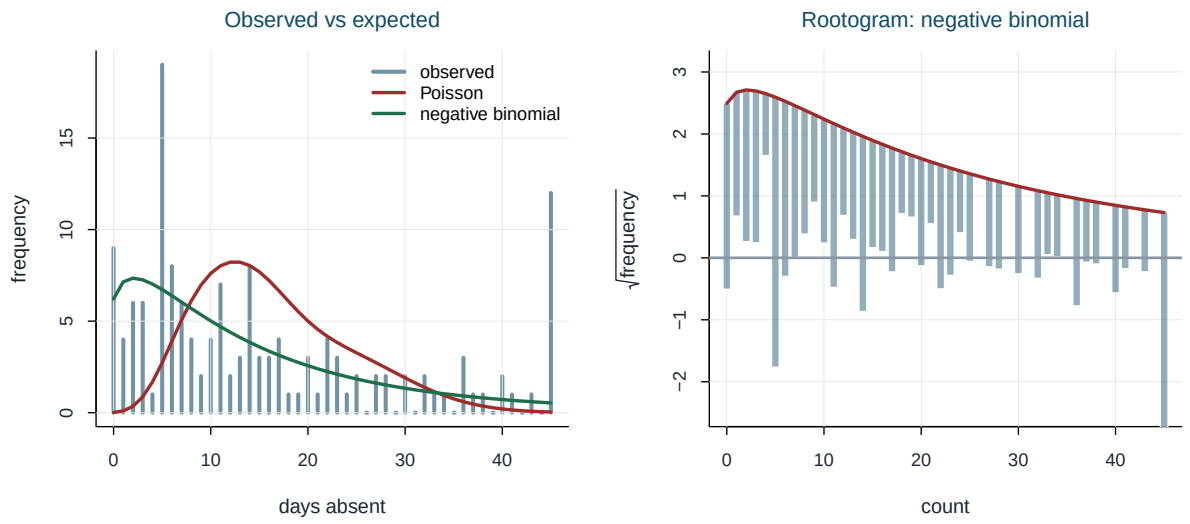


Figure 9.2: Left: expected frequencies of each count under both models against what was observed. Right: a hanging rootogram – bars that hang below zero are counts the model under-predicts.

How to read a rootogram

The red curve is what the model expects. Each bar hangs down from that curve by the observed frequency. If the model is right, every bar just touches the zero line. Bars ending *below* zero mean the model under-predicts that count; bars floating *above* zero mean it over-predicts. It is the single most informative plot for a count model.

9.5 The two flavours: NB2 and NB1

What we have fitted is **NB2**: variance $= \mu + \mu^2/\theta$, quadratic in the mean. There is also **NB1**: variance $= \mu(1 + \alpha)$, linear in the mean — effectively quasi-Poisson but with a genuine likelihood. glmmTMB fits both.

```
library(glmmTMB)
nb2 <- glmmTMB(Days ~ Eth + Sex + Age + Lrn, family = nbinom2, data = quine)
nb1 <- glmmTMB(Days ~ Eth + Sex + Age + Lrn, family = nbinom1, data = quine)
round(c(AIC_nb2 = AIC(nb2), AIC_nb1 = AIC(nb1),
        AIC_glm.nb = AIC(m_nb)), 1)

#>    AIC_nb2    AIC_nb1 AIC_glm.nb
#>       1109       1112       1109
```

`glmmTMB` with `nbinom2` reproduces `glm.nb` exactly, as it should. Here NB2 fits a little better than NB1, so the variance really does grow faster than linearly.

Rule of thumb

Start with NB2 — it is what everyone means by "negative binomial" and it is what `glm.nb` gives you. Try NB1 if the rootogram or the residual plots suggest NB2 is over-correcting in the tail.

9.6 Reporting a negative binomial model

```
tab <- cbind(IRR = exp(coef(m_nb)), exp(confint.default(m_nb)),
             p = coef(summary(m_nb))[, 4])
round(tab, 4)

#>                 IRR   2.5 %  97.5 %      p
#> (Intercept) 18.0759 11.5522 28.2837 0.0000
#> EthN         0.5659  0.4190  0.7643 0.0002
#> SexM         1.0858  0.7937  1.4855 0.6067
#> AgeF1        0.6386  0.3992  1.0217 0.0614
#> AgeF2        1.0921  0.6874  1.7350 0.7092
#> AgeF3        1.4289  0.8783  2.3247 0.1507
#> LrnSL        1.3392  0.9293  1.9301 0.1172
```

The sentence

> "Aboriginal children were absent 1.77 times as often as non-Aboriginal children (IRR 0.57 for non-Aboriginal vs Aboriginal, 95% CI 0.42–0.76), adjusting for sex, age group and learner status. A negative binomial model was used because the counts were strongly overdispersed (Pearson dispersion 13.2 under a Poisson model); the estimated dispersion parameter was $\hat{\theta} = 1.27$."

Always state (a) that you checked for overdispersion, (b) what you found, (c) what you did about it, and (d) the estimated θ. Those four facts let a reader judge your analysis.

9.7 Things that go wrong

9.7.1 θ runs off to infinity

If the data are not actually overdispersed, `glm.nb` may fail to converge or return a huge θ with a huge standard error. That is the model telling you a Poisson would do.

```
set.seed(8)
d0 <- data.frame(x = rnorm(200)); d0$y <- rpois(200, exp(1 + 0.5 * d0$x))
suppressWarnings(nb0 <- glm.nb(y ~ x, data = d0))
c(theta = nb0$theta, se_theta = nb0$SE.theta)

#>    theta se_theta
#>    42501   446490
```

An estimate that large with that much uncertainty means $1/\theta$ is indistinguishable from zero. Use the Poisson.

9.7.2 The negative binomial does not fix everything

It handles *extra variability*. It does not automatically handle *structural zeros*, and it does not handle clustering. Check the rootogram: if only the zero bar is badly off while the rest fits, you need Chapter 10, not a different variance function.

```
c(observed_zeros  = sum(quine$Days == 0),
  poisson_expects = round(sum(dpois(0, fitted(m_pois))), 1),
  negbin_expects  = round(sum(dnbinom(0, mu = fitted(m_nb), size = m_nb$theta)), 1))
```

```
#>  observed_zeros poisson_expects  negbin_expects
#>             9.0             0.0             6.2
```

Here the negative binomial predicts 6.2 zeros against 9 observed — close enough that zero-inflation is not needed for these data. The extra variability was the whole problem, and the negative binomial has absorbed it.

CHAPTER 10

Too many zeros: zero-inflated and hurdle models

10.1 Two kinds of zero

Ask people how many cigarettes they smoked yesterday. You get lots of zeros, but they are not all the same kind of zero:

- **Structural zeros** come from people who do not smoke at all. For them the count *cannot* be anything but zero.
- **Sampling zeros** come from smokers who happened not to smoke yesterday. For them zero is just a low draw from a count process.

A Poisson or negative binomial has only one mechanism, so it must explain both kinds with the same parameter. When the structural group is large, it cannot, and you get more zeros than any single count distribution can produce.

The test question

> "Is there a group in my data for whom the event is impossible, not merely unlikely?" If yes, you probably want zero-inflation. If no — everyone could in principle have a non-zero count — a negative binomial usually handles the zeros by itself.

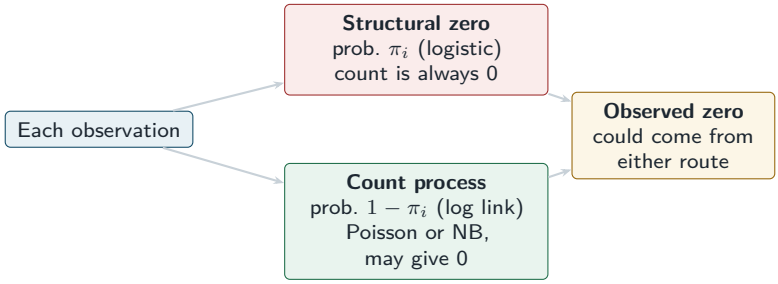
10.2 The zero-inflated model

A zero-inflated model is a *mixture* of two processes:

$$\Pr(Y_i = 0) = \pi_i + (1 - \pi_i)\, f(0; \mu_i)$$

$$\Pr(Y_i = k) = (1 - \pi_i)\, f(k; \mu_i), \qquad k = 1, 2, 3, \ldots$$

where π_i is the probability of being in the "always zero" group (modelled with a logistic regression) and f is a Poisson or negative binomial (modelled with a log link).



10.3 A worked example

`bioChemists` records the number of articles published by 915 PhD biochemistry students in their last three years of study.

```
data(bioChemists, package = "pscl")
str(bioChemists, give.attr = FALSE)


#> 'data.frame': 915 obs. of  6 variables:
#>  $ art : int  0 0 0 0 0 0 0 0 0 0 ...
#>  $ fem : Factor w/ 2 levels "Men","Women": 1 2 2 1 2 2 2 1 1 2 ...
#>  $ mar : Factor w/ 2 levels "Single","Married": 2 1 1 2 1 2 1 2 1 2 ...
#>  $ kid5: num  0 0 0 1 0 2 0 2 0 0 ...
#>  $ phd : num  2.52 2.05 3.75 1.18 3.75 ...
#>  $ ment: int  7 6 6 3 26 2 3 4 6 0 ...


table(bioChemists$art)


#>
#>   0   1   2   3   4   5   6   7   8   9  10  11  12  16  19
#> 275 246 178  84  67  27  17  12   1   2   1   1   2   1   1


round(100 * mean(bioChemists$art == 0), 1)     # percent zeros


#> [1] 30.1
```

30.1% published nothing. Some of those were surely trying and unlucky; others were probably never on a publication track at all. That is exactly the two-group structure zero-inflation describes.

```
mp  <- glm(art ~ fem + mar + kid5 + phd + ment, family = poisson, data = bioChemists)
mnb <- MASS::glm.nb(art ~ fem + mar + kid5 + phd + ment, data = bioChemists)

c(observed_zeros  = sum(bioChemists$art == 0),
  poisson_expects = round(sum(dpois(0, fitted(mp))), 1),
  negbin_expects  = round(sum(dnbinom(0, mu = fitted(mnb), size = mnb$theta)), 1))


#>  observed_zeros poisson_expects  negbin_expects
#>           275.0           191.4           277.8
```

The Poisson expects about 191 zeros against 275 observed — a serious shortfall. The negative binomial does much better on its own, which is already a useful clue.

10.3.1 Fitting a zero-inflated model

The formula has two parts separated by `|`: the count model on the left, the zero-inflation model on the right.

```
library(pscl)
zip <- zeroinfl(art ~ fem + mar + kid5 + phd + ment | fem + mar + kid5 + phd + ment,
                data = bioChemists, dist = "poisson")
summary(zip)


#>
#> Call:
#> zeroinfl(formula = art ~ fem + mar + kid5 + phd + ment | fem + mar + kid5 + phd +
```

```
#>     ment, data = bioChemists, dist = "poisson")
#>
#> Pearson residuals:
#>    Min     1Q Median     3Q    Max
#> -2.325 -0.865 -0.283  0.540  7.298
#>
#> Count model coefficients (poisson with log link):
#>             Estimate Std. Error z value Pr(>|z|)
#> (Intercept)  0.64084    0.12131    5.28  1.3e-07 ***
#> femWomen    -0.20914    0.06340   -3.30  0.00097 ***
#> marMarried   0.10375    0.07111    1.46  0.14456
#> kid5        -0.14332    0.04743   -3.02  0.00251 **
#> phd         -0.00617    0.03101   -0.20  0.84238
#> ment         0.01810    0.00229    7.89  3.1e-15 ***
#>
#> Zero-inflation model coefficients (binomial with logit link):
#>             Estimate Std. Error z value Pr(>|z|)
#> (Intercept) -0.57706    0.50939   -1.13    0.257
#> femWomen     0.10975    0.28008    0.39    0.695
#> marMarried  -0.35401    0.31761   -1.11    0.265
#> kid5         0.21710    0.19648    1.10    0.269
#> phd          0.00127    0.14526    0.01    0.993
#> ment        -0.13411    0.04524   -2.96    0.003 **
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Number of iterations in BFGS optimization: 21
#> Log-likelihood: -1.6e+03 on 12 Df
```

Reading a two-part printout

> **Count model coefficients** are on the log scale and apply to everyone *not* in the structural-zero group. Exponentiate them for rate ratios.
> **Zero-inflation model coefficients** are on the logit scale and predict membership of the "always zero" group. A *positive* coefficient means *more likely to be a structural zero*, i.e. less likely to publish at all.

```
round(exp(coef(zip, model = "count")), 3)     # rate ratios among "potential publishers"

#> (Intercept)    femWomen  marMarried        kid5         phd        ment
#>       1.898       0.811       1.109       0.866       0.994       1.018

round(exp(coef(zip, model = "zero")), 3)      # odds ratios for being a structural zero

#> (Intercept)    femWomen  marMarried        kid5         phd        ment
#>       0.562       1.116       0.702       1.242       1.001       0.874
```

For example, `ment` (articles by the student's mentor) has a negative zero-inflation coefficient, so students with productive mentors are less likely to fall in the never-publishing group — which is exactly what you would expect.

10.3.2 Zero-inflated negative binomial

You can zero-inflate a negative binomial too, which handles excess zeros *and* extra variability at once.

```
zinb <- zeroinfl(art ~ fem + mar + kid5 + phd + ment | fem + mar + kid5 + phd + ment,
                 data = bioChemists, dist = "negbin")
round(c(AIC_poisson = AIC(mp), AIC_negbin = AIC(mnb),
        AIC_zip = AIC(zip), AIC_zinb = AIC(zinb)), 1)

#> AIC_poisson  AIC_negbin     AIC_zip    AIC_zinb
#>        3314        3136        3234        3126
```

What that AIC table says

> The plain Poisson is much the worst. Zero-inflating it helps a lot. But the plain negative binomial does nearly as well as the ZIP with far fewer parameters, and the ZINB is barely better than the NB. In other words: the "excess zeros" here were mostly just overdispersion in disguise.

10.4 Hurdle models

A hurdle model splits the problem differently:

1. **Did you clear the hurdle?** A binary model for zero versus non-zero.
2. **Given that you did, how far?** A *zero-truncated* count model for the positive values.

The crucial difference: in a hurdle model *all* zeros come from the first part. In a zero-inflated model, zeros can come from either part.

```
hnb <- hurdle(art ~ fem + mar + kid5 + phd + ment | fem + mar + kid5 + phd + ment,
              data = bioChemists, dist = "negbin")
summary(hnb)


#>
#> Call:
#> hurdle(formula = art ~ fem + mar + kid5 + phd + ment | fem + mar + kid5 + phd +
#>     ment, data = bioChemists, dist = "negbin")
#>
#> Pearson residuals:
#>    Min     1Q Median     3Q    Max
#> -1.258 -0.804 -0.250  0.474  6.275
#>
#> Count model coefficients (truncated negbin with log link):
#>             Estimate Std. Error z value Pr(>|z|)
#> (Intercept)  0.35512    0.19683    1.80   0.0712 .
#> femWomen    -0.24467    0.09722   -2.52   0.0118 *
#> marMarried   0.10342    0.10943    0.95   0.3446
#> kid5        -0.15326    0.07223   -2.12   0.0338 *
#> phd         -0.00293    0.04807   -0.06   0.9513
#> ment         0.02374    0.00429    5.54  3.1e-08 ***
#> Log(theta)   0.60347    0.22499    2.68   0.0073 **
#> Zero hurdle model coefficients (binomial with logit link):
#>             Estimate Std. Error z value Pr(>|z|)
#> (Intercept)   0.2368     0.2955    0.80    0.423
#> femWomen     -0.2512     0.1591   -1.58    0.114
#> marMarried    0.3262     0.1808    1.80    0.071 .
#> kid5         -0.2852     0.1111   -2.57    0.010 *
#> phd           0.0222     0.0796    0.28    0.780
#> ment          0.0801     0.0130    6.15  7.5e-10 ***
#> ---
```

```
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Theta: count = 1.828
#> Number of iterations in BFGS optimization: 15
#> Log-likelihood: -1.55e+03 on 13 Df
```

The sign convention flips

In `zeroinfl`, the binary part models the probability of being a *zero*. In `hurdle`, it models the probability of being *positive*. So the same predictor gets opposite signs in the two printouts. Check which function produced a table before interpreting its signs.

	Zero-inflated	Hurdle
Where zeros come from	two sources: structural and sampling	one source only
Binary part models	$\Pr(\text{structural zero})$	$\Pr(\text{count} > 0)$
Count part	standard Poisson/NB	zero-*truncated* Poisson/NB
Natural story	a latent subgroup that can never have the event	a two-stage decision: first whether, then how much
Can handle too *few* zeros	no	yes
Typical use	"never-smokers" mixed in with smokers	doctor visits: first you decide to go, then how often

Rule of thumb

If you can name the latent group that structurally cannot have the event, use zero-inflation. If the process is genuinely two decisions in sequence, use a hurdle. If neither story is convincing, you probably just have overdispersion — use a negative binomial.

10.5 Which model reproduces the data best?

Forget the AIC table for a moment and just count what each model predicts.

```
zeros_expected <- c(
  observed = sum(bioChemists$art == 0),
  poisson  = sum(dpois(0, fitted(mp))),
  negbin   = sum(dnbinom(0, mu = fitted(mnb), size = mnb$theta)),
  zip      = sum(predict(zip,  type = "prob")[, 1]),
  zinb     = sum(predict(zinb, type = "prob")[, 1]),
  hurdle   = sum(predict(hnb,  type = "prob")[, 1])
)
round(zeros_expected, 1)

#> observed  poisson   negbin      zip     zinb   hurdle
#>    275.0    191.4    277.8    273.2    285.4    275.0
```

Zero-inflated and hurdle models reproduce the zero count almost exactly — they have a parameter dedicated to it, so that is guaranteed and not evidence of anything. The informative comparison is over the *whole* distribution.

```
par(mfrow = c(1, 2)); par_book()
maxk <- 10
obs  <- as.numeric(table(factor(pmin(bioChemists$art, maxk), levels = 0:maxk)))
e_p  <- sapply(0:maxk, function(k) sum(dpois(k, fitted(mp))))
e_nb <- sapply(0:maxk, function(k) sum(dnbinom(k, mu = fitted(mnb), size = mnb$theta)))
e_zi <- colSums(predict(zip,  type = "prob")[, 1:(maxk + 1)])
e_hu <- colSums(predict(hnb,  type = "prob")[, 1:(maxk + 1)])

plot(0:maxk, obs, type = "h", lwd = 6, col = adjustcolor(PAL["grey"], 0.5),
     xlab = "articles published", ylab = "frequency",
     main = "Observed vs expected frequencies")
grid_book()
lines(0:maxk, e_p,  lwd = 2.5, col = PAL["red"])
lines(0:maxk, e_nb, lwd = 2.5, col = PAL["green"])
lines(0:maxk, e_zi, lwd = 2.5, lty = 2, col = PAL["primary"])
lines(0:maxk, e_hu, lwd = 2.5, lty = 3, col = PAL["accent"])
legend("topright", bty = "n", cex = 0.75, lwd = c(6, 2.5, 2.5, 2.5, 2.5),
       lty = c(1, 1, 1, 2, 3),
       col = c(adjustcolor(PAL["grey"], 0.5), PAL["red"], PAL["green"],
               PAL["primary"], PAL["accent"]),
       legend = c("observed", "Poisson", "neg. binomial", "ZIP", "hurdle NB"))

plot(c(-0.5, maxk + 0.5), c(-6, max(sqrt(e_p)) * 1.1), type = "n",
     xlab = "articles published", ylab = expression(sqrt("frequency")),
     main = "Rootograms: Poisson (red) vs NB (green)")
grid_book()
rect(0:maxk - 0.42, sqrt(e_p) - sqrt(obs), 0:maxk - 0.02, sqrt(e_p),
     col = adjustcolor(PAL["red"], 0.35), border = "white")
rect(0:maxk + 0.02, sqrt(e_nb) - sqrt(obs), 0:maxk + 0.42, sqrt(e_nb),
     col = adjustcolor(PAL["green"], 0.35), border = "white")
abline(h = 0, lwd = 2, col = PAL["grey"])
lines(0:maxk, sqrt(e_p),  lwd = 2, col = PAL["red"])
lines(0:maxk, sqrt(e_nb), lwd = 2, col = PAL["green"])
```

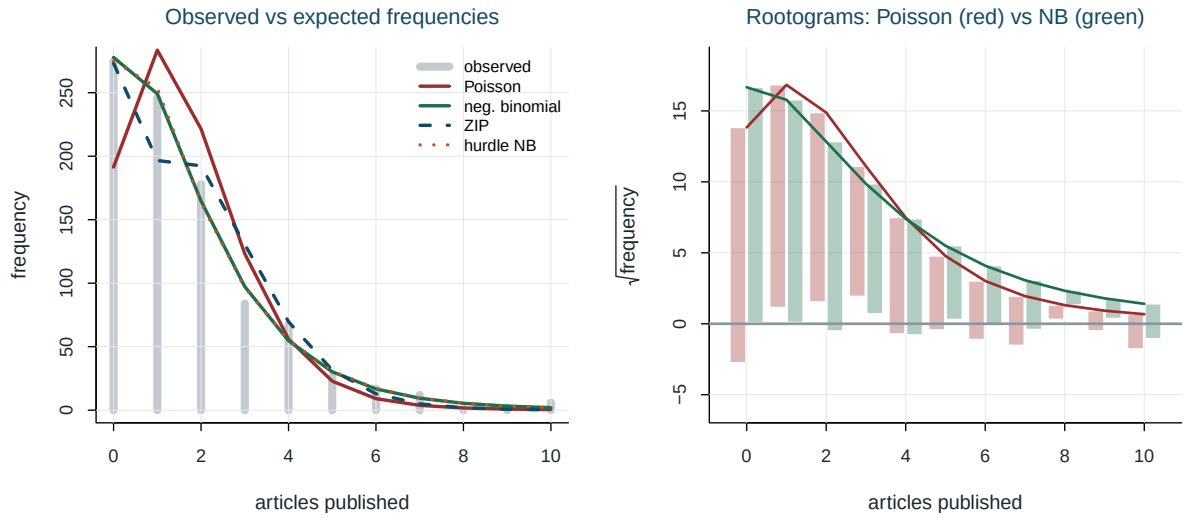


Figure 10.1: Left: expected frequency of each count under four models. Right: rootograms for the Poisson and the negative binomial. The Poisson misses the zeros badly; the negative binomial does not.

10.6 The Vuong test, and why to be careful with it

The Vuong test compares two *non-nested* models by looking at whether one assigns higher likelihood to the individual observations than the other.

```
vuong(zip, mp)      # zero-inflated Poisson vs plain Poisson

#> Vuong Non-Nested Hypothesis Test-Statistic:
#> (test-statistic is asymptotically distributed N(0,1) under the
#>  null that the models are indistinguishible)
#> -------------------------------------------------------------
#>                Vuong z-statistic             H_A  p-value
#> Raw                        4.180 model1 > model2 0.000015
#> AIC-corrected              3.639 model1 > model2  0.00014
#> BIC-corrected              2.333 model1 > model2  0.00983
```

Do not use Vuong for zero-inflation versus its own base model

A zero-inflated Poisson *contains* the Poisson as the special case $\pi = 0$, so the two are nested, not non-nested — and $\pi = 0$ sits on the boundary of the parameter space. Vuong's theory assumes neither. Simulation studies find the test badly behaved for exactly this comparison, and it tends to favour the zero-inflated model too often. Use AIC/BIC and a rootogram instead. Vuong is legitimate for genuinely non-nested comparisons, such as ZIP versus hurdle-NB.

```
vuong(zinb, hnb)    # ZINB vs hurdle-NB: genuinely non-nested, so this is fair

#> Vuong Non-Nested Hypothesis Test-Statistic:
#> (test-statistic is asymptotically distributed N(0,1) under the
#>  null that the models are indistinguishible)
#> -------------------------------------------------------------
#>                Vuong z-statistic             H_A p-value
#> Raw                       0.5552 model1 > model2    0.29
#> AIC-corrected             0.5552 model1 > model2    0.29
#> BIC-corrected             0.5552 model1 > model2    0.29
```

10.7 A case with real zero-inflation

`bioChemists` turned out not to need zero-inflation. Here is a dataset that does: boating trips taken in a year, where most respondents took none.

```
data(RecreationDemand, package = "AER")
round(100 * mean(RecreationDemand$trips == 0), 1)       # percent zeros

#> [1] 63.3

mp_r  <- glm(trips ~ quality + ski + income + userfee + costC + costS + costH,
             family = poisson, data = RecreationDemand)
mnb_r <- MASS::glm.nb(trips ~ quality + ski + income + userfee + costC + costS + costH,
                      data = RecreationDemand)
zinb_r <- zeroinfl(trips ~ quality + ski + income + userfee + costC + costS + costH |
                   quality + income, data = RecreationDemand, dist = "negbin")
hnb_r  <- hurdle(trips ~ quality + ski + income + userfee + costC + costS + costH |
                 quality + income, data = RecreationDemand, dist = "negbin")
```

```
round(c(observed = sum(RecreationDemand$trips == 0),
        poisson  = sum(dpois(0, fitted(mp_r))),
        negbin   = sum(dnbinom(0, mu = fitted(mnb_r), size = mnb_r$theta)),
        zinb     = sum(predict(zinb_r, type = "prob")[, 1]),
        hurdle   = sum(predict(hnb_r,  type = "prob")[, 1])), 1)

#> observed  poisson   negbin     zinb   hurdle
#>    417.0    276.5    423.0    432.9    417.0

round(c(AIC_pois = AIC(mp_r), AIC_nb = AIC(mnb_r),
        AIC_zinb = AIC(zinb_r), AIC_hurdle = AIC(hnb_r)), 1)

#>   AIC_pois     AIC_nb   AIC_zinb AIC_hurdle
#>       3075       1669       1468       1554
```

Here the story is different: the zero-inflated and hurdle models are clearly better than the negative binomial on AIC, and only they get the zero count right. This is what a genuine need for zero-inflation looks like.

The decision procedure

1. Fit Poisson. Count observed zeros against predicted zeros.
2. Fit negative binomial. Count again. Usually the shortfall is gone.
3. If a large shortfall *remains*, and you can tell a story about a structural-zero group, fit ZI or hurdle models.
4. Compare with AIC/BIC and, above all, with a rootogram.

Zero-inflation is over-used

Many published zero-inflated models were fitted because someone saw a lot of zeros and stopped there. Lots of zeros is completely normal for a count with a small mean — a Poisson with mean 0.5 produces 61% zeros all by itself. The question is never "are there many zeros?" but "are there more zeros *than my model predicts*?"

```
round(sapply(c(0.5, 1, 2, 5), function(m) dpois(0, m)), 3)   # % zeros by Poisson mean

#> [1] 0.607 0.368 0.135 0.007
```

CHAPTER 11

The rest of the count family

Poisson, negative binomial, zero-inflated and hurdle models cover most of what you will meet. This chapter is a tour of the remaining members, so that you recognise them when a dataset calls for one.

11.1 Truncated counts

Sometimes zero is impossible *by design*. Length of hospital stay in days is at least 1 for anyone in the sample, because you had to be admitted to be counted. Number of items bought is at least 1 for anyone in a purchase database. Fitting an ordinary Poisson to such data biases everything, because the model keeps assigning probability to a value that could never be observed.

The zero-truncated Poisson has probability mass

$$\Pr(Y = k \mid Y > 0) = \frac{e^{-\mu}\mu^{k}/k!}{1 - e^{-\mu}}, \qquad k = 1, 2, \ldots$$

— the ordinary Poisson divided by the probability of not being zero. That denominator is the entire correction, and it is easy enough to fit directly.

```
## Simulate counts, then throw away the zeros -- as the sampling process would
set.seed(31)
n <- 800
x <- rnorm(n)
y <- rpois(n, exp(0.7 + 0.8 * x))
d <- data.frame(x, y)[y > 0, ]        # zeros are unobservable

## Naive Poisson on the truncated sample: both coefficients are pulled away
round(coef(glm(y ~ x, family = poisson, data = d)), 3)

#> (Intercept)           x
#>       0.869       0.698

## Zero-truncated Poisson, fitted by maximising the truncated log-likelihood
nll_ztp <- function(beta, y, X) {
  mu <- exp(as.vector(X %*% beta))
  -sum(dpois(y, mu, log = TRUE) - log(1 - exp(-mu)))   # divide by P(Y > 0)
}
X   <- model.matrix(~ x, data = d)
fit <- optim(c(0, 0), nll_ztp, y = d$y, X = X, hessian = TRUE, method = "BFGS")
round(rbind(estimate = fit$par,
            se = sqrt(diag(solve(fit$hessian)))), 3)

#>           [,1]  [,2]
#> estimate 0.677 0.825
#> se       0.036 0.027
```

The truth was $\beta_0 = 0.7$, $\beta_1 = 0.8$. The naive fit is noticeably off — the intercept especially, because it is being asked to explain a sample with the zeros removed. The truncated fit recovers both.

In practice you would use `countreg::zerotrunc` or `VGAM::vglm(..., family = pospoisson())` rather than writing the likelihood yourself, but seeing it written out makes clear that there is nothing mysterious going on.

Situation	Model	R
Zeros impossible	zero-truncated Poisson/NB	`hurdle(y ~ x | 1)`, or VGAM's `pospoisson`
Counts capped at a maximum	binomial	`cbind(y, max - y)`
Counts only observed above a threshold	truncated at k	VGAM, countreg
Counts censored ("5 or more")	censored count model	VGAM

11.2 Underdispersion and the Conway–Maxwell–Poisson

When counts are *less* variable than Poisson — regulated processes, scheduled events, clutch sizes in some birds — the negative binomial cannot help. It only ever adds variance. Your options:

- **Quasi-Poisson** with $\hat{\phi} < 1$. Simple and effective; it just shrinks the standard errors.
- **Binomial**, if a real upper bound exists.
- **Conway–Maxwell–Poisson (COM-Poisson)**, which has an extra parameter ν that handles both directions: $\nu > 1$ underdispersed, $\nu = 1$ Poisson, $\nu < 1$ overdispersed. Available in the COMPoissonReg and glmmTMB packages (`family = compois`).
- **Generalised Poisson**, similar idea, in `glmmTMB` as `genpois`.

```
set.seed(12)
du <- data.frame(x = rnorm(300))
du$y <- rbinom(300, size = 10, prob = plogis(0.2 + 0.5 * du$x))   # underdispersed
mu_p <- glm(y ~ x, family = poisson, data = du)
c(dispersion = sum(residuals(mu_p, type = "pearson")^2) / df.residual(mu_p))

#> dispersion
#>     0.4537

library(glmmTMB)
mu_cmp <- glmmTMB(y ~ x, family = compois, data = du)
round(c(AIC_poisson = AIC(mu_p), AIC_compois = AIC(mu_cmp)), 1)

#> AIC_poisson AIC_compois
#>        1198        1124
```

11.3 Beta-binomial: overdispersion for proportions

The binomial's counterpart to the negative binomial. Let each unit's success probability be drawn from a beta distribution instead of being fixed, and the resulting counts are more variable than binomial. This is the principled fix for the overdispersed herd data of Chapter 4.

```
data(cbpp, package = "lme4")
bb <- glmmTMB(cbind(incidence, size - incidence) ~ period,
              family = betabinomial, data = cbpp)
```

```
mb <- glm(cbind(incidence, size - incidence) ~ period, family = binomial, data = cbpp)
round(c(AIC_binomial = AIC(mb), AIC_betabinomial = AIC(bb)), 1)

#>     AIC_binomial AIC_betabinomial
#>            206.1            186.1

round(cbind(binomial = coef(summary(mb))[, 2],
            betabinomial = coef(summary(bb))$cond[, 2]), 4)

#>             binomial betabinomial
#> (Intercept)   0.1449       0.2326
#> period2       0.2915       0.4064
#> period3       0.3129       0.4568
#> period4       0.4131       0.5109
```

Same pattern as before: the coefficients barely move, the standard errors grow.

11.4 Ordinal and multinomial outcomes

Not counts, but they belong on the same map and people often arrive here looking for them.

Outcome	Model	R
Ordered categories (mild / moderate / severe)	proportional odds	`MASS::polr`
Unordered categories (bus / car / train)	multinomial logit	`nnet::multinom`
Ordered, with random effects	ordinal GLMM	`ordinal::clmm`

```
data(housing, package = "MASS")
po <- MASS::polr(Sat ~ Infl + Type + Cont, weights = Freq, data = housing)
round(exp(coef(po)), 3)          # proportional odds ratios

#>    InflMedium      InflHigh TypeApartment    TypeAtrium   TypeTerrace      ContHigh
#>         1.762         3.628         0.564         0.693         0.336         1.434
```

The proportional odds model says one odds ratio applies to *every* cut-point of the ordinal scale. That assumption is worth testing — fit a multinomial model and compare, or use `brant::brant`.

11.5 When the relationship is not a straight line

A GLM assumes the effect of each predictor is linear *on the link scale*. If it is not, you have three options, in increasing order of flexibility:

```
set.seed(19)
n  <- 400
x  <- runif(n, 0, 10)
mu <- exp(1 + 0.9 * sin(x / 1.6))        # a genuinely wiggly truth
y  <- rpois(n, mu)
dg <- data.frame(x, y)

m_lin <- glm(y ~ x, family = poisson, data = dg)
m_pol <- glm(y ~ poly(x, 5), family = poisson, data = dg)
```

```
m_gam <- mgcv::gam(y ~ s(x), family = poisson, data = dg)

par_book()
xs <- data.frame(x = seq(0, 10, length.out = 300))
plot(x, y, pch = 21, bg = adjustcolor(PAL["grey"], 0.35), col = "white", cex = 1,
     xlab = "x", ylab = "count", main = "Linear vs polynomial vs smooth")
grid_book()
lines(xs$x, exp(1 + 0.9 * sin(xs$x / 1.6)), lwd = 3, col = PAL["grey"])
lines(xs$x, predict(m_lin, xs, type = "response"), lwd = 2.5, col = PAL["red"])
lines(xs$x, predict(m_pol, xs, type = "response"), lwd = 2.5, col = PAL["accent"])
lines(xs$x, predict(m_gam, xs, type = "response"), lwd = 2.5, col = PAL["green"])
legend("topright", bty = "n", cex = 0.75, lwd = c(3, 2.5, 2.5, 2.5),
       col = c(PAL["grey"], PAL["red"], PAL["accent"], PAL["green"]),
       legend = c("truth", "linear in x", "poly(x, 5)", "gam s(x)"))
```

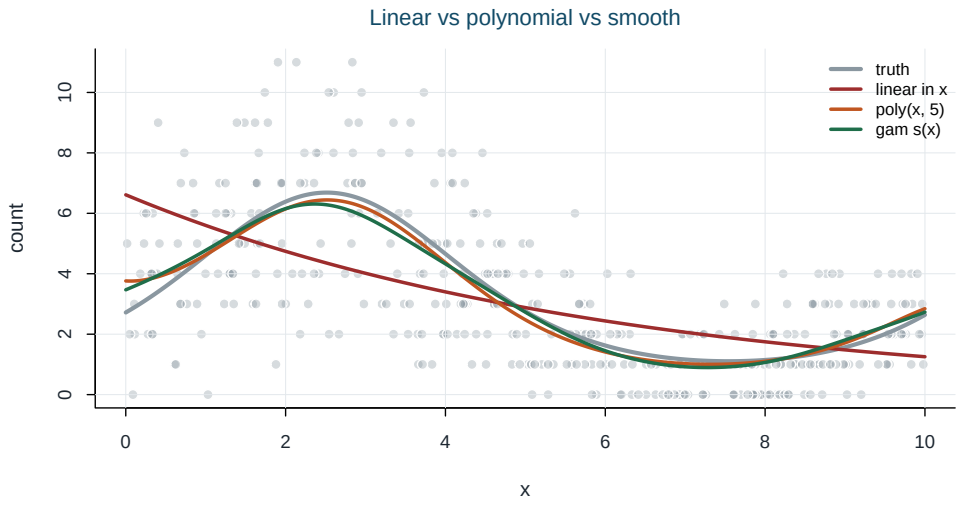


Figure 11.1: A count outcome whose effect is genuinely curved. A straight-line log-link fit misses the shape entirely; a spline or a GAM recovers it.

```
round(c(linear = AIC(m_lin), poly5 = AIC(m_pol), gam = AIC(m_gam)), 1)

#> linear  poly5    gam
#>   1625   1464   1461
```

Curvature masquerades as overdispersion

> The linear fit above will show a large dispersion statistic — not because the Poisson is wrong, but because the mean model is. Always check the shape of your predictors before deciding the distribution is at fault.

11.6 Continuous non-negative outcomes

Costs, durations, biomass, rainfall: positive, skewed, but not counts. These are not covered by anything above.

Outcome	Model	R
Positive, skewed, no zeros	Gamma with log link	`glm(family = Gamma(log))`
Positive, with exact zeros	Tweedie	`glmmTMB(family = tweedie)`
Proportion in $(0, 1)$, not from counts	beta regression	`betareg::betareg`
Any of the above, and you only trust the mean	quasi-Poisson	`family = quasipoisson`

Quasi-Poisson for non-integers

A quasi-Poisson model does not require integer data — it only assumes $\mathrm{E}(y) = \mu$ and $\mathrm{Var}(y) = \phi\mu$. This makes it a robust, surprisingly good choice for positive continuous outcomes when you want multiplicative effects and do not want to commit to a distribution. R will warn about non-integer responses; for quasi-Poisson that warning is safe to ignore.

CHAPTER 12

Checking a count model

Residual plots that work for `lm` are close to useless for count models. With a fitted mean of 2, the residual can only be $-2, -1, 0, 1, 2, \ldots$, so the plot shows a set of diagonal stripes and you learn nothing. This chapter gives you residuals and plots that actually work.

12.1 The four kinds of residual

```
r_resp <- residuals(m_nb, type = "response")   # y - mu
r_pear <- residuals(m_nb, type = "pearson")    # (y - mu) / sqrt(V(mu))
r_dev  <- residuals(m_nb, type = "deviance")   # signed sqrt of deviance contribution
r_work <- residuals(m_nb, type = "working")    # used internally by the fitting algorithm
round(head(cbind(response = r_resp, pearson = r_pear,
                 deviance = r_dev, working = r_work), 4), 3)

#>   response pearson deviance working
#> 1   -24.29  -1.019   -1.910  -0.924
#> 2   -15.29  -0.641   -0.832  -0.582
#> 3   -12.29  -0.515   -0.625  -0.467
#> 4   -14.63  -0.815   -1.194  -0.745
```

Type	Definition	Use it for
Response	$y - \hat{\mu}$	Almost nothing; scale depends on μ
Pearson	$(y - \hat{\mu})/\sqrt{V(\hat{\mu})}$	Dispersion; squared and summed it gives X^2
Deviance	signed $\sqrt{d_i}$	R's default; the most symmetric of the classical ones
Quantile	simulation-based (below)	Everything else — this is the one to use

None of the first three is normally distributed for count data, so a normal Q–Q plot of them is misleading.

```
par(mfrow = c(1, 2)); par_book()
plot(fitted(m_nb), r_dev, pch = 21, bg = adjustcolor(PAL["primary"], 0.35),
     col = "white", cex = 1.1, xlab = "fitted mean", ylab = "deviance residual",
     main = "Deviance residuals: stripey and hard to read")
grid_book(); abline(h = 0, col = PAL["grey"], lwd = 1.5)
lines(lowess(fitted(m_nb), r_dev), col = PAL["red"], lwd = 2.5)

qqnorm(r_dev, pch = 21, bg = adjustcolor(PAL["primary"], 0.35), col = "white",
       cex = 1.1, main = "Normal Q-Q of deviance residuals")
qqline(r_dev, col = PAL["red"], lwd = 2.5); grid_book()
```

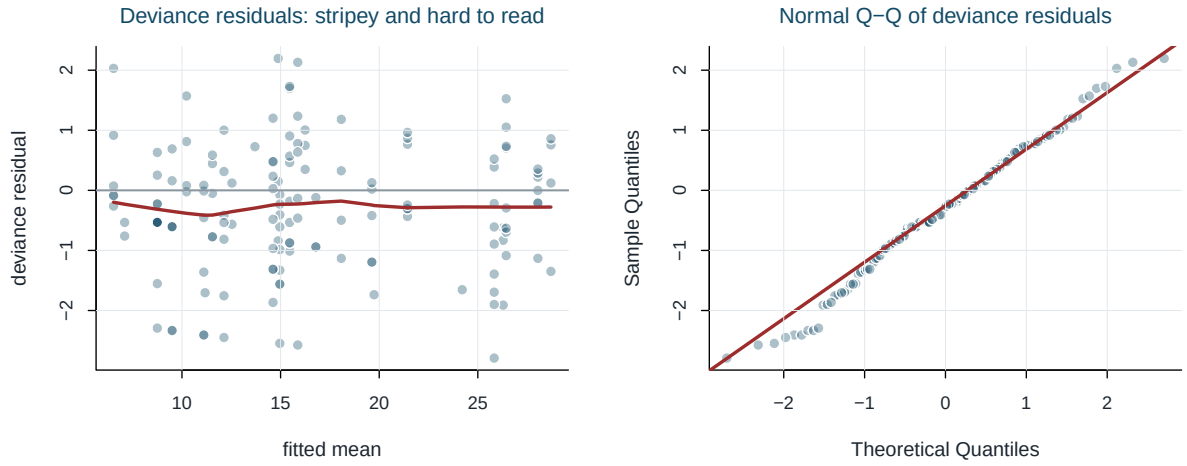


Figure 12.1: Deviance residuals against fitted values for a count model. The diagonal stripes are the discreteness of the outcome, not a model failure – which is exactly why these plots are so hard to read.

12.2 Randomised quantile residuals: the tool that works

The idea is beautiful. If the model is correct, then for each observation the value $F(y_i; \hat{\mu}_i)$ — the model's own cumulative probability at the observed value — should be uniform on $(0, 1)$. For discrete data F jumps, so we draw a uniform value *within* the jump. Push the result through `qnorm` and, if the model is right, you get residuals that are exactly standard normal — however discrete the outcome is.

```
qresid <- function(y, pfun, ...) {
  a <- pfun(y - 1, ...)          # CDF just below the observed value
  b <- pfun(y,     ...)          # CDF at the observed value
  u <- runif(length(y), pmin(a, b), pmax(a, b))
  qnorm(pmin(pmax(u, 1e-10), 1 - 1e-10))
}
set.seed(101)
q_pois <- qresid(quine$Days, ppois,   lambda = fitted(m_pois))
q_nb   <- qresid(quine$Days, pnbinom, mu = fitted(m_nb), size = m_nb$theta)
round(c(sd_qres_poisson = sd(q_pois), sd_qres_negbin = sd(q_nb)), 3)

#> sd_qres_poisson  sd_qres_negbin
#>           3.129           1.006
```

If the model fits, those standard deviations should be near 1. The Poisson's is 3.13; the negative binomial's is 1.01.

```
par(mfrow = c(2, 2)); par_book()
for (nm in c("Poisson", "Negative binomial")) {
  qr <- if (nm == "Poisson") q_pois else q_nb
  ft <- if (nm == "Poisson") fitted(m_pois) else fitted(m_nb)
  cl <- if (nm == "Poisson") PAL["red"] else PAL["green"]

  qqnorm(qr, pch = 21, bg = adjustcolor(cl, 0.45), col = "white", cex = 1.1,
         main = paste(nm, ": Q-Q of quantile residuals"))
  qqline(qr, col = PAL["primary"], lwd = 2.5); grid_book()

  plot(ft, qr, pch = 21, bg = adjustcolor(cl, 0.45), col = "white", cex = 1.1,
       log = "x", xlab = "fitted mean (log scale)", ylab = "quantile residual",
```

```
       ylim = c(-4, 4), main = paste(nm, ": residuals vs fitted"))
  grid_book(); abline(h = c(-2, 0, 2), lty = c(3, 1, 3), col = PAL["grey"], lwd = 1.5)
  lines(lowess(ft, qr), col = PAL["primary"], lwd = 2.5)
}
```

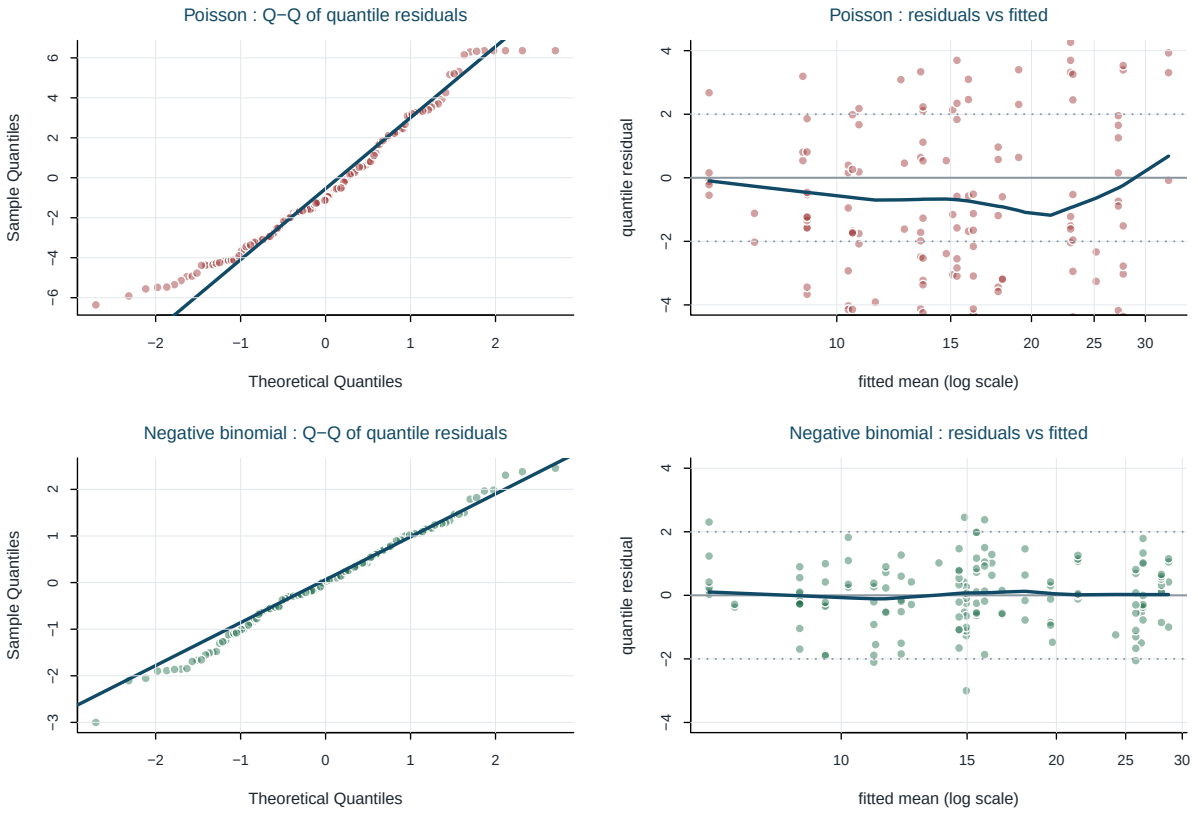


Figure 12.2: Randomised quantile residuals. Top row: the Poisson model – the Q-Q plot bends away from the line and the residuals fan out. Bottom row: the negative binomial – points on the line, no pattern against the fitted values. This is what a good count model looks like.

What to look for in a quantile-residual plot

Q–Q plot: points on the diagonal. Bending up at the right means the model under-predicts the tail.
Residuals vs fitted: a flat cloud between roughly -2 and 2, with the smoother near zero. A trend means the mean model is wrong; a funnel means the variance function is wrong.
Residuals vs each predictor: the same, one plot per predictor. This is where you find a missing quadratic term.

The DHARMa package automates all of this with a simulation-based version of the same idea, and is worth installing for serious work: `DHARMa::simulateResiduals(model)` followed by `plot()`.

12.3 Simulation-based checks you can always do

Even without any package, you can ask: *if my model were true, what would data from it look like?* Simulate from the fitted model and compare summaries with the real data.

```
set.seed(5)
sim_stats <- function(model, nsim = 500) {
  s <- simulate(model, nsim = nsim)
  data.frame(sd = apply(s, 2, sd),
             max = apply(s, 2, max),
             p_over20 = apply(s, 2, function(z) mean(z > 20)))
}
sp <- sim_stats(m_pois); sn <- sim_stats(m_nb)
obs <- c(sd = sd(quine$Days), max = max(quine$Days), p_over20 = mean(quine$Days > 20))

par(mfrow = c(1, 3)); par_book()
for (v in c("sd", "max", "p_over20")) {
  ttl <- c(sd = "standard deviation", max = "maximum",
           p_over20 = "proportion above 20")[v]
  rng <- range(c(sp[[v]], sn[[v]], obs[v]))
  hist(sn[[v]], breaks = 25, col = adjustcolor(PAL["green"], 0.5), border = "white",
       xlim = rng, xlab = ttl, main = ttl, freq = FALSE)
  hist(sp[[v]], breaks = 25, col = adjustcolor(PAL["red"], 0.45), border = "white",
       add = TRUE, freq = FALSE)
  abline(v = obs[v], lwd = 3, col = PAL["primary"])
}
```

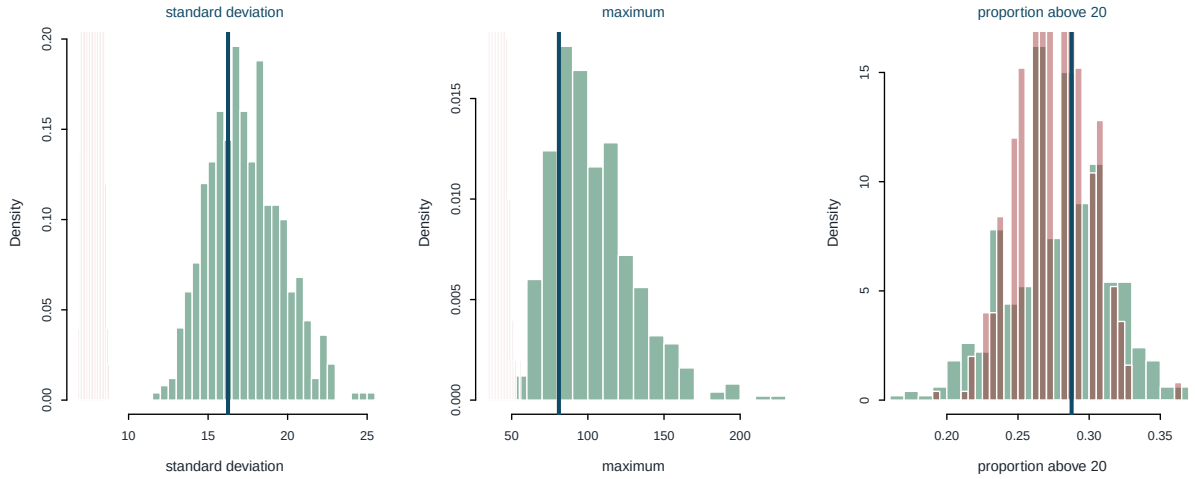


Figure 12.3: Simulating 500 datasets from each fitted model and comparing three summary statistics with the observed values (red lines). The Poisson cannot produce data as variable or as extreme as what was observed; the negative binomial can.

```
round(rbind(
  observed = obs,
  poisson_mean = colMeans(sp),
  negbin_mean  = colMeans(sn),
  poisson_pvalue = colMeans(sweep(sp, 2, obs, ">=")),
  negbin_pvalue  = colMeans(sweep(sn, 2, obs, ">="))
), 3)

#>                    sd     max p_over20
#> observed       16.253  81.000    0.288
#> poisson_mean    7.766  40.196    0.274
#> negbin_mean    17.327 103.068    0.277
#> poisson_pvalue  0.000   0.000    0.346
#> negbin_pvalue   0.648   0.804    0.424
```

Reading those “p-values”

Each is the proportion of simulated datasets whose statistic was at least as large as the observed one. Values very close to 0 or 1 mean the model cannot reproduce that feature of your data. Values in the middle mean it can. The Poisson fails on all three; the negative binomial is comfortable.

12.4 Is the variance function right?

Bin the fitted values, and within each bin compare the observed variance with what each model claims it should be.

```
par_book()
b   <- cut(fitted(m_nb), breaks = quantile(fitted(m_nb), seq(0, 1, 0.125)),
           include.lowest = TRUE)
mb_ <- tapply(quine$Days, b, mean)
vb  <- tapply(quine$Days, b, var)
plot(mb_, vb, pch = 19, cex = 1.5, col = PAL["primary"], log = "xy",
     xlab = "observed mean in bin", ylab = "observed variance in bin",
     main = "Which variance function matches the data?")
grid_book()
mu <- seq(min(mb_) * 0.8, max(mb_) * 1.2, length.out = 100)
lines(mu, mu, lwd = 2.5, col = PAL["red"])
lines(mu, mu + mu^2 / m_nb$theta, lwd = 2.5, col = PAL["green"])
phi <- sum(residuals(m_pois, type = "pearson")^2) / df.residual(m_pois)
lines(mu, phi * mu, lwd = 2.5, lty = 2, col = PAL["accent"])
legend("topleft", bty = "n", cex = 0.75, lwd = 2.5, lty = c(1, 1, 2),
       col = c(PAL["red"], PAL["green"], PAL["accent"]),
       legend = c("Poisson: mu", "NB: mu + mu^2/theta", "quasi-Poisson: phi*mu"))
```

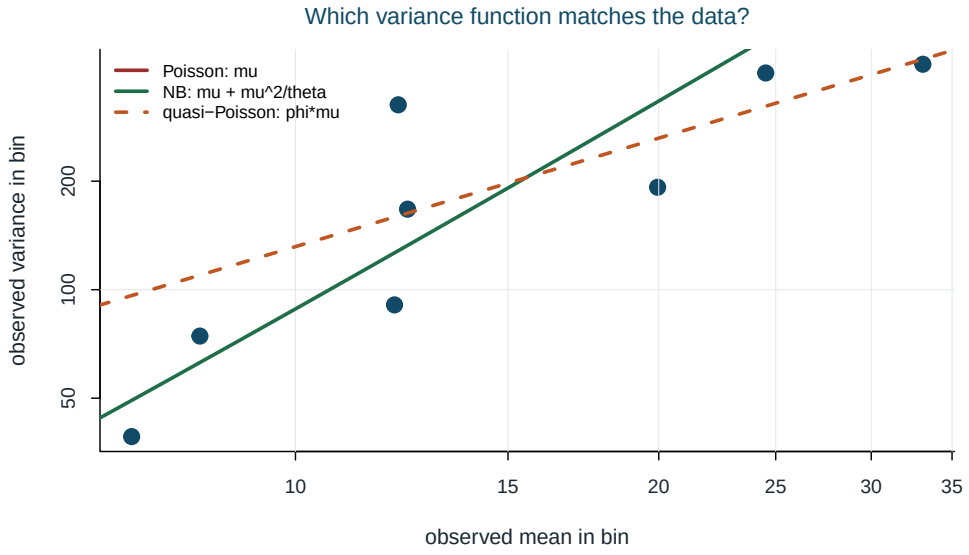


Figure 12.4: Observed variance within bins of fitted value, against the variance each model assumes. The negative binomial’s quadratic variance function tracks the data; the Poisson’s identity line does not.

12.5 Influence and outliers

```
par(mfrow = c(1, 2)); par_book()
cd <- cooks.distance(m_nb)
plot(cd, type = "h", lwd = 2, col = PAL["primary"], xlab = "observation",
     ylab = "Cook's distance", main = "Influence on the fit")
grid_book(); abline(h = 4 / nrow(quine), lty = 2, col = PAL["red"])
big <- which(cd > 4 / nrow(quine))
text(big, cd[big], labels = big, pos = 3, cex = 0.6, col = PAL["red"])

dfb <- dfbeta(m_nb)
matplot(dfb[, -1], type = "l", lty = 1, lwd = 1.6,
        col = c(PAL["primary"], PAL["accent"], PAL["green"], PAL["red"], PAL["purple"]),
        xlab = "observation", ylab = "change in coefficient",
        main = "dfbeta: influence on each coefficient")
grid_book(); abline(h = 0, col = PAL["grey"])
legend("topleft", bty = "n", cex = 0.6, lty = 1, lwd = 1.6,
       col = c(PAL["primary"], PAL["accent"], PAL["green"], PAL["red"], PAL["purple"]),
       legend = colnames(dfb)[-1])
```

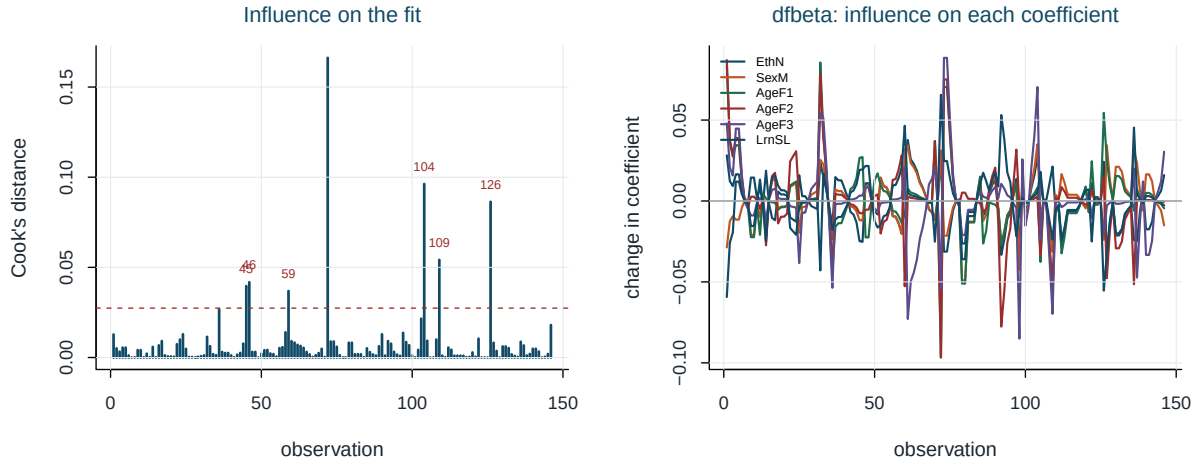


Figure 12.5: Left: Cook's distance flags observations that move the coefficients. Right: how much each coefficient shifts when a single observation is dropped, standardised (dfbeta).

```
worst <- which.max(cd)
c(observation = worst, days = quine$Days[worst], fitted = round(fitted(m_nb)[worst], 1))

#> observation.72           days      fitted.72
#>           72.0           67.0           14.9

round(cbind(with_all = coef(m_nb),
            without_worst = coef(update(m_nb, data = quine[-worst, ]))), 4)

#>             with_all without_worst
#> (Intercept)   2.8946        2.8343
#> EthN         -0.5694       -0.6117
#> SexM          0.0823        0.0047
#> AgeF1        -0.4484       -0.2663
#> AgeF2         0.0881        0.2965
#> AgeF3         0.3569        0.4878
#> LrnSL         0.2921        0.1615
```

What to do with an influential point

Do not delete it because it is influential. Look at it. If it is a data-entry error, fix it. If it is real, either keep it and report that the model is sensitive to it, or use a model that accommodates it — which for counts usually means a longer-tailed distribution rather than deletion.

12.6 A checklist you can run every time

```
check_count_model <- function(model, y, theta = NULL) {
  mu  <- fitted(model)
  X2  <- sum(residuals(model, type = "pearson")^2)
  df  <- df.residual(model)
  p0  <- if (is.null(theta)) dpois(0, mu) else dnbinom(0, mu = mu, size = theta)
  data.frame(
    dispersion       = round(X2 / df, 3),
    dispersion_p     = signif(pchisq(X2, df, lower.tail = FALSE), 3),
    zeros_observed   = sum(y == 0),
    zeros_expected   = round(sum(p0), 1),
    max_observed     = max(y),
    mean_observed    = round(mean(y), 2),
    mean_fitted      = round(mean(mu), 2)
  )
}
rbind(poisson = check_count_model(m_pois, quine$Days),
      negbin  = check_count_model(m_nb, quine$Days, theta = m_nb$theta))

#>         dispersion dispersion_p zeros_observed zeros_expected max_observed mean_observed
#> poisson     13.167    1.45e-292              9            0.0           81         16.46
#> negbin       0.991     5.13e-01              9            6.2           81         16.46
#>         mean_fitted
#> poisson       16.46
#> negbin        16.47
```

The five checks, in order

1. Dispersion statistic near 1?
2. Predicted zeros ≈ observed zeros?
3. Quantile-residual Q–Q plot on the line?
4. Quantile residuals flat against fitted values *and* against every predictor?
5. Rootogram bars touching zero?

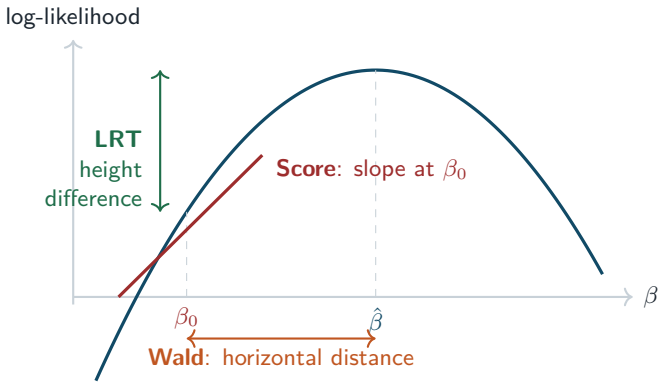
Pass all five and your count model is defensible.

CHAPTER 13

Tests, comparisons and model selection

13.1 The three classical tests

Every likelihood-based model comes with three ways to test a hypothesis. They answer the same question and usually agree, but they measure different things.



Test	What it measures	In R
Likelihood ratio	How much the log-likelihood drops when you impose the null	`anova(m1, m2, test="LRT")`, `lmtest::lrtest`
Wald	How many standard errors $\hat{\beta}$ is from the null value	the z column of `summary()`
Score (Lagrange multiplier)	The slope of the log-likelihood at the null	`anova(..., test="Rao")`

All three are asymptotically equivalent. In finite samples the likelihood-ratio test is generally the most reliable, and the Wald test is the least — but the Wald test is the one printed by default, which is why it is the one everybody uses.

```
full <- glm(low ~ age + lwt + race + smoke + ht + ui, family = binomial, data = bw)
red  <- update(full, . ~ . - race)                # drop a 2-df factor

## Likelihood ratio
anova(red, full, test = "LRT")


#> Analysis of Deviance Table
#>
#> Model 1: low ~ age + lwt + smoke + ht + ui
#> Model 2: low ~ age + lwt + race + smoke + ht + ui
#>   Resid. Df Resid. Dev Df Deviance Pr(>Chi)
#> 1       183        212
#> 2       181        204  2     7.83     0.02 *
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

## Score (Rao)
anova(red, full, test = "Rao")
```

```
#> Analysis of Deviance Table
#>
#> Model 1: low ~ age + lwt + smoke + ht + ui
#> Model 2: low ~ age + lwt + race + smoke + ht + ui
#>   Resid. Df Resid. Dev Df Deviance  Rao Pr(>Chi)
#> 1       183        212
#> 2       181        204  2     7.83 7.78     0.02 *
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
## Wald, for the same 2-df hypothesis
car::Anova(full, type = "II")["race", ]
```

```
#> Analysis of Deviance Table (Type II tests)
#>
#> Response: low
#>      LR Chisq Df Pr(>Chisq)
#> race     7.83  2       0.02 *
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The Wald test misbehaves for large effects

When a coefficient is large, its standard error is inflated too, and the Wald statistic can shrink back towards zero — the Hauck–Donner effect. In the extreme case of complete separation (Chapter 4) the Wald p-value approaches 1 for a perfect predictor. The likelihood-ratio test does not have this problem. If a Wald test and an LRT disagree, believe the LRT.

13.2 Comparing nested models

Two models are *nested* when one is a special case of the other. Then you can use the difference in deviance:

$$\Delta D = D_{\text{reduced}} - D_{\text{full}} \sim \chi^2_{\Delta \text{df}} \quad \text{under the null.}$$

```
m1 <- MASS::glm.nb(Days ~ Age, data = quine)
m2 <- MASS::glm.nb(Days ~ Age + Eth, data = quine)
m3 <- MASS::glm.nb(Days ~ Age + Eth + Sex + Lrn, data = quine)
anova(m1, m2, m3)
```

```
#> Likelihood ratio tests of Negative Binomial Models
#>
#> Response: Days
#>                   Model theta Resid. df    2 x log-lik.   Test    df LR stat.   Pr(Chi)
#> 1                   Age 1.147       142           -1108
#> 2             Age + Eth 1.249       141           -1096 1 vs 2     1    12.01 0.0005279
#> 3 Age + Eth + Sex + Lrn 1.275       139           -1093 2 vs 3     2     2.65 0.2657784
```

Do not compare models fitted to different data

If a predictor has missing values, adding it silently drops rows, and the two models are then fitted to different datasets — so the deviance difference is meaningless. Guard against it by subsetting to complete cases first: `d <- na.omit(d[, all.vars(formula_of_largest_model)])`.

```
d <- quine
d$Lrn[1:20] <- NA                          # pretend some values are missing
a <- MASS::glm.nb(Days ~ Age, data = d)
b <- MASS::glm.nb(Days ~ Age + Lrn, data = d)
c(n_a = nobs(a), n_b = nobs(b))            # different sample sizes -- do not compare


#> n_a n_b
#> 146 126
```

13.3 Comparing non-nested models: AIC and BIC

$$\text{AIC} = -2\ell + 2k, \qquad \text{BIC} = -2\ell + k \log n$$

where ℓ is the maximised log-likelihood and k the number of parameters. Lower is better; only differences matter.

```
mp   <- glm(Days ~ Eth + Sex + Age + Lrn, family = poisson, data = quine)
mnb  <- MASS::glm.nb(Days ~ Eth + Sex + Age + Lrn, data = quine)
mnb2 <- MASS::glm.nb(Days ~ Eth * Age + Sex + Lrn, data = quine)

tab <- data.frame(
  model = c("Poisson", "NB", "NB with Eth x Age"),
  df    = c(length(coef(mp)), length(coef(mnb)) + 1, length(coef(mnb2)) + 1),
  logLik = round(c(logLik(mp), logLik(mnb), logLik(mnb2)), 2),
  AIC   = round(c(AIC(mp), AIC(mnb), AIC(mnb2)), 1),
  BIC   = round(c(BIC(mp), BIC(mnb), BIC(mnb2)), 1)
)
tab$dAIC <- round(tab$AIC - min(tab$AIC), 1)
tab


#>               model df  logLik  AIC  BIC   dAIC
#> 1           Poisson  7 -1142.6 2299 2320 1194.5
#> 2                NB  8  -546.6 1109 1133    4.5
#> 3 NB with Eth x Age 11  -541.3 1105 1138    0.0
```

Reading ΔAIC

$\Delta < 2$: the models are practically indistinguishable.
$2 < \Delta < 7$: moderate evidence for the better one.
$\Delta > 10$: strong evidence; the worse model has essentially no support.
BIC penalises complexity more heavily as n grows, so it prefers smaller models. Use AIC when you care about prediction, BIC when you believe a true model exists and want to find it.

AIC cannot cross model families

Comparing AIC between a Poisson and a negative binomial is fine — both are likelihoods for the same response. Comparing AIC between a model for `y` and a model for `log(y)` is not, because the likelihoods are on different scales. And quasi-Poisson models have no likelihood at all, so they have no AIC. Use QAIC (Chapter 8) there.

13.4 Akaike weights

Turn ΔAIC into something interpretable:

```
w <- exp(-0.5 * tab$dAIC); w <- w / sum(w)
data.frame(model = tab$model, dAIC = tab$dAIC, weight = round(w, 4))
```

```
#>               model   dAIC weight
#> 1           Poisson 1194.5 0.0000
#> 2                NB    4.5 0.0953
#> 3 NB with Eth x Age    0.0 0.9047
```

The weight is the relative probability that each model is the best of those considered, given the data and this criterion.

13.5 Tests specific to count models

```
## 1. Is there overdispersion? (Chapter 7)
AER::dispersiontest(mp, trafo = 2)
```

```
#>
#>  Overdispersion test
#>
#> data:  mp
#> z = 5.3, p-value = 5e-08
#> alternative hypothesis: true alpha is greater than 0
#> sample estimates:
#>  alpha
#> 0.6362
```

```
## 2. Is the negative binomial better than the Poisson? (boundary-corrected LRT)
pscl::odTest(mnb)
```

```
#> Likelihood ratio test of H0: Poisson, as restricted NB model:
#> n.b., the distribution of the test-statistic under H0 is non-standard
#> e.g., see help(odTest) for details/references
#>
#> Critical value of test statistic at the alpha= 0.05 level: 2.706
#> Chi-Square Test Statistic =  1192.033 p-value = <2e-16
```

```
## 3. Non-nested comparison, e.g. zero-inflated vs hurdle
data(bioChemists, package = "pscl")
z <- pscl::zeroinfl(art ~ fem + mar + kid5 + ment | fem + ment,
                    data = bioChemists, dist = "negbin")
h <- pscl::hurdle(art ~ fem + mar + kid5 + ment | fem + ment,
                  data = bioChemists, dist = "negbin")
pscl::vuong(z, h)
```

```
#> Vuong Non-Nested Hypothesis Test-Statistic:
#> (test-statistic is asymptotically distributed N(0,1) under the
#>  null that the models are indistinguishible)
#> -------------------------------------------------------------
#>                Vuong z-statistic             H_A p-value
#> Raw                    0.8668 model1 > model2    0.19
#> AIC-corrected          0.8668 model1 > model2    0.19
#> BIC-corrected          0.8668 model1 > model2    0.19
```

13.6 How much does the model explain?

There is no single R^2 for a GLM. Several competing versions exist, and they give different numbers.

```
pseudo_r2 <- function(model, null_model) {
  L1 <- as.numeric(logLik(model)); L0 <- as.numeric(logLik(null_model))
  n  <- nobs(model)
  c(McFadden   = 1 - L1 / L0,
    CoxSnell   = 1 - exp(2 * (L0 - L1) / n),
    Nagelkerke = (1 - exp(2 * (L0 - L1) / n)) / (1 - exp(2 * L0 / n)))
}
round(pseudo_r2(mnb, MASS::glm.nb(Days ~ 1, data = quine)), 4)

#>   McFadden   CoxSnell Nagelkerke
#>     0.0225     0.1580     0.1581

round(pseudo_r2(full, glm(low ~ 1, family = binomial, data = bw)), 4)

#>   McFadden   CoxSnell Nagelkerke
#>     0.1309     0.1500     0.2110
```

How to read a pseudo-R^2

> McFadden's is the most common and the most pessimistic; values of 0.2–0.4 already indicate a good fit, so do not be alarmed by a small number. Nagelkerke's is rescaled to reach 1 and looks more like the R^2 you are used to. Report which one you used, because they are not comparable with each other.

13.7 Cross-validation

For prediction, none of the above beats simply holding data out. Here is K-fold cross-validation for a count model, scored with mean absolute error and with the log score.

```
set.seed(2)
K     <- 10
folds <- sample(rep(1:K, length.out = nrow(quine)))
score <- function(fit_fun, pred_fun) {
  mae <- ll <- numeric(K)
  for (k in 1:K) {
    tr <- quine[folds != k, ]; te <- quine[folds == k, ]
    m  <- fit_fun(tr)
    p  <- predict(m, newdata = te, type = "response")
    mae[k] <- mean(abs(te$Days - p))
    ll[k]  <- mean(pred_fun(te$Days, p, m))
  }
  c(MAE = mean(mae), mean_logLik = mean(ll))
}
round(rbind(
  poisson = score(function(d) glm(Days ~ Eth + Sex + Age + Lrn, poisson, d),
                  function(y, p, m) dpois(y, p, log = TRUE)),
  negbin  = score(function(d) MASS::glm.nb(Days ~ Eth + Sex + Age + Lrn, data = d),
                  function(y, p, m) dnbinom(y, mu = p, size = m$theta, log = TRUE))
), 4)

#>           MAE mean_logLik
```

```
#> poisson 11.87      -8.819
#> negbin  11.83      -3.831
```

The two models predict the mean about equally well (similar MAE) — as they should, since overdispersion does not bias the mean. But the negative binomial assigns far higher probability to the data actually observed, which is what the log score captures.

The distinction that matters

Getting the *mean* right and getting the *uncertainty* right are separate achievements. Mean absolute error only checks the first. Log score, coverage of prediction intervals, and rootograms check the second. Overdispersion is entirely a failure of the second kind.

13.8 A note on stepwise selection

```
step_model <- MASS::stepAIC(mnb2, direction = "both", trace = FALSE)
```

```
formula(step_model)

#> Days ~ Eth + Age + Eth:Age
```

It runs, and it is tempting. But automated stepwise selection inflates R^2, biases coefficients away from zero, produces standard errors and p-values that are known to be wrong, and is unstable — resample your data and you often get a different model.

Rule of thumb

Prefer a model specified from subject knowledge. If you must select, use penalisation (glmnet) or model averaging rather than stepwise, and if you do use stepwise, report that you did and treat the resulting p-values as descriptive rather than inferential.

CHAPTER 14

Mixed models I: why and how

14.1 The problem: not all observations are independent

Every model so far has assumed each row of your data is a fresh, independent piece of information. Very often it is not:

- the same patient measured at five visits;
- 30 pupils in each of 40 schools;
- repeated litters from the same mother;
- several plots inside each farm, several farms inside each district.

Two pupils from the same school are more alike than two pupils picked at random. So your 1200 pupils do not carry 1200 pupils' worth of information about schools. Treating them as if they do makes your standard errors far too small.

Effective sample size

> If observations within a cluster are correlated, your real sample size is somewhere between the number of clusters and the number of observations. In the worst case — perfectly correlated observations — 30 measurements on one person tell you exactly as much as one measurement.

14.2 How bad is it? A simulation

```
set.seed(77)
sim_cluster <- function(n_group = 20, n_per = 10, sd_group = 1.2) {
  g   <- rep(1:n_group, each = n_per)
  x   <- rep(rnorm(n_group), each = n_per)      # a GROUP-level predictor
  b_g <- rep(rnorm(n_group, 0, sd_group), each = n_per)   # group random effect
  y   <- 0 + 0 * x + b_g + rnorm(n_group * n_per, 0, 1)   # TRUE effect of x is 0
  d   <- data.frame(y, x, g = factor(g))
  p_lm  <- summary(lm(y ~ x, data = d))$coefficients["x", 4]
  p_lmm <- {
    m <- lme4::lmer(y ~ x + (1 | g), data = d)
    2 * pnorm(abs(summary(m)$coefficients["x", "t value"]), lower.tail = FALSE)
  }
  c(ignoring_clusters = p_lm, mixed_model = p_lmm)
}
ps <- t(replicate(400, sim_cluster()))
round(colMeans(ps < 0.05), 3)          # how often each finds a "significant" effect

#> ignoring_clusters       mixed_model
#>             0.465             0.082
```

The true effect of `x` is exactly zero. Ignoring the clustering, we declare significance 46% of the time instead of 5%. The mixed model stays near the advertised rate.

Watch out

Clustering hurts most for predictors that vary *between* clusters (treatment assigned per school, exposure assigned per farm). For predictors that vary *within* a cluster the damage is usually smaller — but it is never zero.

14.3 Fixed effects versus random effects

	Fixed effect	Random effect
The levels are	the specific ones you care about	a sample from a larger population
You estimate	one coefficient per level	one *variance*
Cost in df	$k - 1$ parameters	1 parameter
Can predict new levels	no	yes
Handles unbalanced data	poorly	well
Typical examples	treatment, sex, dose	subject, school, batch, litter, plot

The practical test

Ask: "if I repeated this study, would I get the same levels of this variable?" Same treatments, same sexes — fixed. A different set of schools, patients or batches — random.

14.4 The random-intercept model

$$y_{ij} = \underbrace{\beta_0 + \beta_1 x_{ij}}_{\text{fixed part}} + \underbrace{u_j}_{\text{random intercept}} + \varepsilon_{ij}, \qquad u_j \sim N(0, \sigma_u^2), \quad \varepsilon_{ij} \sim N(0, \sigma^2).$$

Each group j gets its own intercept $\beta_0 + u_j$, and those intercepts are assumed to be drawn from a normal distribution whose variance we estimate.

```
library(lme4)
data(sleepstudy)
str(sleepstudy)

#> 'data.frame': 180 obs. of  3 variables:
#>  $ Reaction: num  250 259 251 321 357 ...
#>  $ Days    : num  0 1 2 3 4 5 6 7 8 9 ...
#>  $ Subject : Factor w/ 18 levels "308","309","310",..: 1 1 1 1 1 1 1 1 1 1 ...

m_ri <- lmer(Reaction ~ Days + (1 | Subject), data = sleepstudy)
summary(m_ri)

#> Linear mixed model fit by REML ['lmerMod']
#> Formula: Reaction ~ Days + (1 | Subject)
#>    Data: sleepstudy
#>
#> REML criterion at convergence: 1786
#>
#> Scaled residuals:
#>    Min     1Q Median     3Q    Max
#> -3.226 -0.553  0.011  0.519  4.251
```

```
#>
#> Random effects:
#>  Groups   Name        Variance Std.Dev.
#>  Subject  (Intercept) 1378     37.1
#>  Residual              960     31.0
#> Number of obs: 180, groups:  Subject, 18
#>
#> Fixed effects:
#>             Estimate Std. Error t value
#> (Intercept)  251.405      9.747    25.8
#> Days          10.467      0.804    13.0
#>
#> Correlation of Fixed Effects:
#>      (Intr)
#> Days -0.371
```

14.4.1 Reading the random effects table

```
VarCorr(m_ri)
```

```
#>  Groups   Name        Std.Dev.
#>  Subject  (Intercept) 37.1
#>  Residual             31.0
```

```
as.data.frame(VarCorr(m_ri))
```

```
#>        grp        var1 var2   vcov sdcor
#> 1  Subject (Intercept) <NA> 1378.2 37.12
#> 2 Residual        <NA> <NA>  960.5 30.99
```

In plain English

The `Subject` variance is the spread of the subject-specific intercepts: how much people differ from each other on average. The `Residual` variance is the spread of measurements within a person. Together they partition the total variability into "between people" and "within people".

14.4.2 The intraclass correlation

$$\text{ICC} = \frac{\sigma_u^2}{\sigma_u^2 + \sigma^2}$$

— the proportion of total variance that is between groups, and equivalently the correlation between two observations from the same group.

```
vc <- as.data.frame(VarCorr(m_ri))
icc <- vc$vcov[1] / sum(vc$vcov)
round(c(between_subject_var = vc$vcov[1], within_subject_var = vc$vcov[2],
        ICC = icc), 3)
```

```
#> between_subject_var  within_subject_var                 ICC
#>           1378.179             960.457               0.589
```

An ICC of 0.59 means about 59% of the variation in reaction time is between people rather than within them. Any ICC noticeably above 0 means clustering must be modelled.

14.5 Random slopes

A random intercept says everyone starts at a different level. A random slope says everyone *responds* differently too.

$$y_{ij} = \beta_0 + \beta_1 x_{ij} + u_{0j} + u_{1j} x_{ij} + \varepsilon_{ij}$$

```
m_rs <- lmer(Reaction ~ Days + (Days | Subject), data = sleepstudy)
summary(m_rs)$varcor


#>  Groups   Name        Std.Dev. Corr
#>  Subject  (Intercept) 24.74
#>           Days         5.92    0.07
#>  Residual             25.59


anova(m_ri, m_rs)


#> Data: sleepstudy
#> Models:
#> m_ri: Reaction ~ Days + (1 | Subject)
#> m_rs: Reaction ~ Days + (Days | Subject)
#>       npar  AIC  BIC logLik deviance Chisq Df Pr(>Chisq)
#> m_ri    4 1802 1815   -897     1794
#> m_rs    6 1764 1783   -876     1752  42.1  2    7.1e-10 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The likelihood-ratio test strongly favours the random-slope model: subjects really do differ in how badly sleep deprivation affects them.

```
par(mfrow = c(1, 2)); par_book()
plot(sleepstudy$Days, sleepstudy$Reaction, type = "n",
     xlab = "days of sleep deprivation", ylab = "reaction time (ms)",
     main = "Every subject has their own line")
grid_book()
for (s in unique(sleepstudy$Subject)) {
  d <- subset(sleepstudy, Subject == s)
  cf <- coef(m_rs)$Subject[as.character(s), ]
  abline(cf[[1]], cf[[2]], col = adjustcolor(PAL["grey"], 0.75), lwd = 1.2)
  points(d$Days, d$Reaction, pch = 21, bg = adjustcolor(PAL["primary"], 0.20),
         col = "white", cex = 0.8)
}
abline(fixef(m_rs)[1], fixef(m_rs)[2], lwd = 4, col = PAL["accent"])
legend("topleft", bty = "n", cex = 0.8, lwd = c(4, 1.2),
       col = c(PAL["accent"], PAL["grey"]),
       legend = c("population average", "individual subjects"))

re <- ranef(m_rs)$Subject
plot(re[, 1], re[, 2], pch = 21, bg = adjustcolor(PAL["accent"], 0.6), col = "white",
     cex = 1.6, xlab = "random intercept (deviation, ms)",
     ylab = "random slope (deviation, ms/day)",
     main = "Subject-level deviations")
grid_book(); abline(h = 0, v = 0, lty = 3, col = PAL["grey"])
text(re[, 1], re[, 2], rownames(re), pos = 3, cex = 0.55, col = PAL["grey"])
```

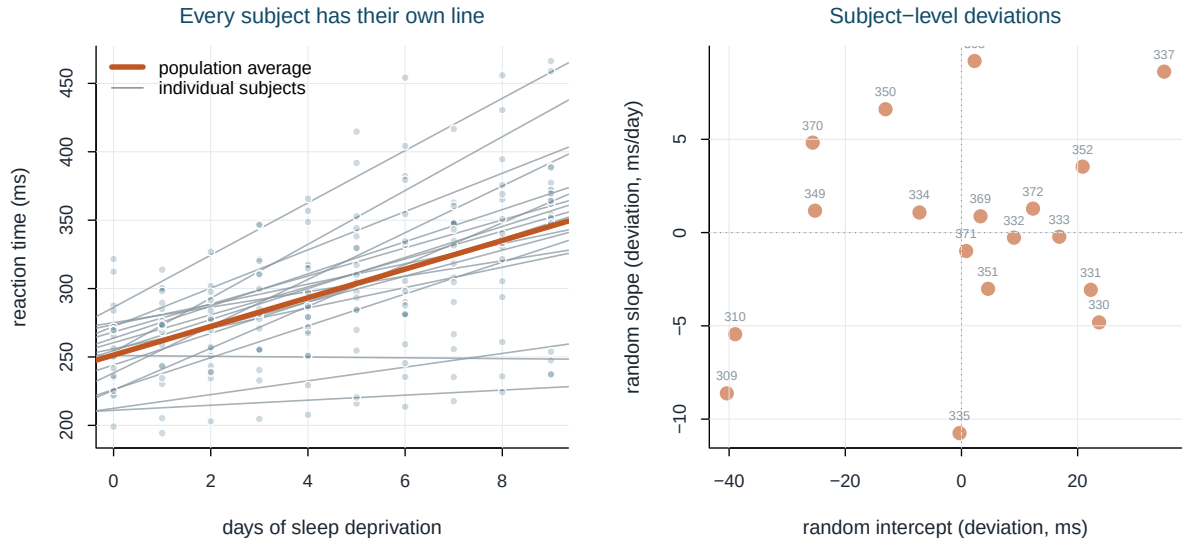


Figure 14.1: Left: each subject's own trajectory with the population average in bold. Right: subject-specific intercepts and slopes from the mixed model. The spread of the points is exactly what the random-effects variances measure.

14.6 Shrinkage: the best feature of mixed models

Fit a separate regression to each subject and you get 18 completely independent estimates, each noisy. The mixed model instead pulls each subject's estimate towards the overall average — by a lot if that subject has little data or is extreme, hardly at all if the subject has plenty of consistent data.

```
par_book()
indiv <- t(sapply(split(sleepstudy, sleepstudy$Subject),
                  function(d) coef(lm(Reaction ~ Days, data = d))))
mixed <- as.matrix(coef(m_rs)$Subject)
plot(rbind(indiv, mixed), type = "n", xlab = "intercept", ylab = "slope (ms/day)",
     main = "Shrinkage towards the population mean")
grid_book()
arrows(indiv[, 1], indiv[, 2], mixed[, 1], mixed[, 2], length = 0.06,
       col = adjustcolor(PAL["grey"], 0.9), lwd = 1.5)
points(indiv[, 1], indiv[, 2], pch = 21, bg = adjustcolor(PAL["red"], 0.5),
       col = "white", cex = 1.4)
points(mixed[, 1], mixed[, 2], pch = 21, bg = adjustcolor(PAL["green"], 0.7),
       col = "white", cex = 1.4)
points(fixef(m_rs)[1], fixef(m_rs)[2], pch = 4, cex = 2.4, lwd = 3, col = PAL["primary"])
legend("topright", bty = "n", cex = 0.75, pch = c(21, 21, 4),
       pt.bg = c(adjustcolor(PAL["red"], 0.5), adjustcolor(PAL["green"], 0.7), NA),
       col = c("white", "white", PAL["primary"]),
       legend = c("separate lm per subject", "mixed model (shrunk)", "population average"))
```



Figure 14.2: Arrows run from each subject's own separate regression fit to the mixed model's estimate. Estimates furthest from the centre are pulled in the most: this is shrinkage, and it is what makes mixed-model estimates more accurate on average.

Key idea

Shrinkage is not a bug or a fudge. It is the mathematically optimal way to combine what one subject tells you with what all the other subjects tell you. It is why a mixed model beats fitting each group separately, especially when groups are small or unbalanced.

14.7 Specifying random effects in R

The formula syntax is compact and worth learning properly.

Syntax	Meaning
`(1 | g)`	random intercept per level of g
`(x | g)`	random intercept and random slope for x, correlated
`(0 + x | g)`	random slope only, no random intercept
`(1 | g) + (0 + x | g)`	random intercept and slope, forced uncorrelated
`(x || g)`	shorthand for the line above
`(1 | g1/g2)`	g2 nested inside g1; same as `(1|g1) + (1|g1:g2)`
`(1 | g1) + (1 | g2)`	crossed random effects
`(1 | obs)`	observation-level random effect (Chapter 15)

Nested versus crossed depends on your labels

If classroom is labelled 1, 2, 3 *within every school*, then class "1" in school A and class "1" in school B are different classes and you must write `(1|school/class)`. If every class has a globally unique ID, `(1|school) + (1|class)` gives the same fit. Getting this wrong silently pools unrelated groups together.

14.8 REML or ML?

`lmer` uses REML (restricted maximum likelihood) by default. REML gives less biased variance estimates; ML gives comparable likelihoods for models with different fixed effects.

You want to...	Use	R
compare models differing in *fixed* effects	ML	`REML = FALSE`
compare models differing in *random* effects	REML (or ML)	default
report final variance estimates	REML	default

```
a <- lmer(Reaction ~ Days + (Days | Subject), data = sleepstudy, REML = FALSE)
b <- lmer(Reaction ~ 1    + (Days | Subject), data = sleepstudy, REML = FALSE)
anova(b, a)                # comparing fixed effects: both fitted with ML
```

```
#> Data: sleepstudy
#> Models:
#> b: Reaction ~ 1 + (Days | Subject)
#> a: Reaction ~ Days + (Days | Subject)
#>   npar  AIC  BIC logLik deviance Chisq Df Pr(>Chisq)
#> b    5 1785 1801   -888     1775
#> a    6 1764 1783   -876     1752  23.5  1  0.0000012 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Helpfully, `anova()` on `merMod` objects automatically refits with ML when you compare models, so this trap is largely handled for you.

14.9 Where are the p-values?

`lmer` deliberately prints t values without p-values, because the correct denominator degrees of freedom for an unbalanced mixed model are genuinely not known. Your options:

```
## 1. Likelihood-ratio test (recommended; refit with ML)
anova(b, a)$`Pr(>Chisq)`[2]
```

```
#> [1] 0.000001226
```

```
## 2. Wald with a normal approximation -- fine when you have many clusters
tv <- summary(m_rs)$coefficients["Days", "t value"]
c(t = tv, p_normal_approx = 2 * pnorm(abs(tv), lower.tail = FALSE))
```

```
#>               t p_normal_approx
#>       6.771e+00       1.275e-11
```

```
## 3. Profile-likelihood confidence intervals -- often the most informative
confint(m_rs, method = "profile", oldNames = FALSE)
```

```
#>                                2.5 %  97.5 %
#> sd_(Intercept)|Subject        14.3814  37.716
#> cor_Days.(Intercept)|Subject  -0.4815   0.685
#> sd_Days|Subject                3.8012   8.753
#> sigma                         22.8983  28.858
#> (Intercept)                  237.6807 265.130
#> Days                           7.3587  13.576
```

A fourth option is `lmerTest::lmer`, which adds Satterthwaite or Kenward–Roger degrees of freedom and prints p-values in the usual place. With few clusters (under about 30) Kenward–Roger is the safest choice.

Rule of thumb

Many clusters (40+): the normal approximation is fine.
Few clusters (under 30): use Kenward–Roger, or bootstrap (`confint(m, method = "boot")`).
Comparing nested models: use the likelihood-ratio test, refitted with ML.

CHAPTER 15

Mixed models II: GLMMs for binary and count data

A generalised linear mixed model is exactly what its name says: a GLM (Chapter 2) plus random effects (Chapter 14). The linear predictor gains a random term:

$$g(\mu_{ij}) = \underbrace{\beta_0 + \beta_1 x_{ij}}_{\text{fixed}} + \underbrace{u_j}_{\text{random}}, \qquad u_j \sim N(0, \sigma_u^2).$$

Everything you know about links and families carries straight over.

15.1 Binomial GLMM

Back to the `cbpp` data from Chapter 4: disease incidence in 15 herds of Ethiopian cattle over four time periods. Herds differ, and we could not fix the resulting overdispersion properly with quasi-binomial. Now we can.

```
library(lme4)
data(cbpp)
m_glm  <- glm(cbind(incidence, size - incidence) ~ period,
              family = binomial, data = cbpp)
m_glmm <- glmer(cbind(incidence, size - incidence) ~ period + (1 | herd),
                family = binomial, data = cbpp)
summary(m_glmm)
```

```
#> Generalized linear mixed model fit by maximum likelihood (Laplace Approximation) [glmerMod
#> ]
#>  Family: binomial  ( logit )
#> Formula: cbind(incidence, size - incidence) ~ period + (1 | herd)
#>    Data: cbpp
#>
#>      AIC      BIC   logLik deviance df.resid
#>    194.1    204.2    -92.0    184.1       51
#>
#> Scaled residuals:
#>    Min     1Q Median     3Q    Max
#> -2.382 -0.789 -0.203  0.514  2.879
#>
#> Random effects:
#>  Groups Name        Variance Std.Dev.
#>  herd   (Intercept) 0.412    0.642
#> Number of obs: 56, groups:  herd, 15
#>
#> Fixed effects:
#>             Estimate Std. Error z value Pr(>|z|)
#> (Intercept)   -1.398      0.231   -6.05  1.5e-09 ***
#> period2       -0.992      0.303   -3.27  0.00107 **
#> period3       -1.128      0.323   -3.49  0.00047 ***
#> period4       -1.580      0.422   -3.74  0.00018 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
#>
#> Correlation of Fixed Effects:
#>         (Intr) perid2 perid3
#> period2 -0.363
#> period3 -0.340  0.280
#> period4 -0.260  0.213  0.198
```

```
round(cbind(
  est_glm  = coef(m_glm),
  est_glmm = fixef(m_glmm),
  se_glm   = coef(summary(m_glm))[, 2],
  se_glmm  = coef(summary(m_glmm))[, 2]
), 4)
```

```
#>             est_glm est_glmm se_glm se_glmm
#> (Intercept)  -1.269  -1.3983 0.1449  0.2312
#> period2      -1.171  -0.9919 0.2915  0.3032
#> period3      -1.301  -1.1282 0.3129  0.3228
#> period4      -1.782  -1.5797 0.4131  0.4220
```

```
anova(m_glmm, m_glm)                # is the herd random effect needed?
```

```
#> Data: cbpp
#> Models:
#> m_glm: cbind(incidence, size - incidence) ~ period
#> m_glmm: cbind(incidence, size - incidence) ~ period + (1 | herd)
#>        npar AIC BIC logLik deviance Chisq Df Pr(>Chisq)
#> m_glm     4 206 214    -99      198
#> m_glmm    5 194 204    -92      184    14  1    0.00018 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Key idea

> The herd variance is 0.412 on the logit scale, and the likelihood-ratio test says it is clearly needed. Notice the coefficients changed slightly and the standard errors grew: exactly the correction that the quasi-binomial approximated, but now with an interpretable variance component attached.

15.1.1 ICC for a binomial GLMM

The residual variance is not free on the logit scale — it is fixed at $\pi^2/3 \approx 3.29$, the variance of the standard logistic distribution. So:

```
sig2_herd <- as.data.frame(VarCorr(m_glmm))$vcov[1]
icc <- sig2_herd / (sig2_herd + pi^2 / 3)
round(c(herd_variance = sig2_herd, latent_residual = pi^2/3, ICC = icc), 4)
```

```
#>   herd_variance latent_residual             ICC
#>          0.4123          3.2899          0.1114
```

About 11% of the variation on the latent scale is between herds. That is substantial, and it is why treating cows as independent would have been badly wrong.

```
par(mfrow = c(1, 2)); par_book()
cbpp$prop <- cbpp$incidence / cbpp$size
plot(as.integer(cbpp$period) + runif(nrow(cbpp), -0.15, 0.15), cbpp$prop,
     pch = 21, bg = adjustcolor(PAL["primary"], 0.5), col = "white",
     cex = 0.6 + 2 * cbpp$size / max(cbpp$size), xaxt = "n",
     xlab = "period", ylab = "observed proportion infected",
     main = "Raw data (point size = herd size)")
axis(1, at = 1:4); grid_book()
pp <- predict(m_glmm, newdata = data.frame(period = factor(1:4)),
              re.form = NA, type = "response")
lines(1:4, pp, lwd = 3, col = PAL["accent"])

re  <- ranef(m_glmm, condVar = TRUE)$herd
sds <- sqrt(attr(ranef(m_glmm, condVar = TRUE)$herd, "postVar")[1, 1, ])
o   <- order(re[, 1])
plot(re[o, 1], seq_along(o), pch = 19, col = PAL["primary"], cex = 1.2, yaxt = "n",
     xlim = range(re[, 1] + 2 * sds, re[, 1] - 2 * sds),
     xlab = "herd random intercept (log-odds)", ylab = "",
     main = "Herd effects (caterpillar plot)")
axis(2, at = seq_along(o), labels = rownames(re)[o], las = 1, cex.axis = 0.6)
grid_book(); abline(v = 0, lty = 2, col = PAL["red"], lwd = 2)
arrows(re[o, 1] - 1.96 * sds[o], seq_along(o), re[o, 1] + 1.96 * sds[o],
       seq_along(o), code = 3, angle = 90, length = 0.02, col = PAL["primary"])
```

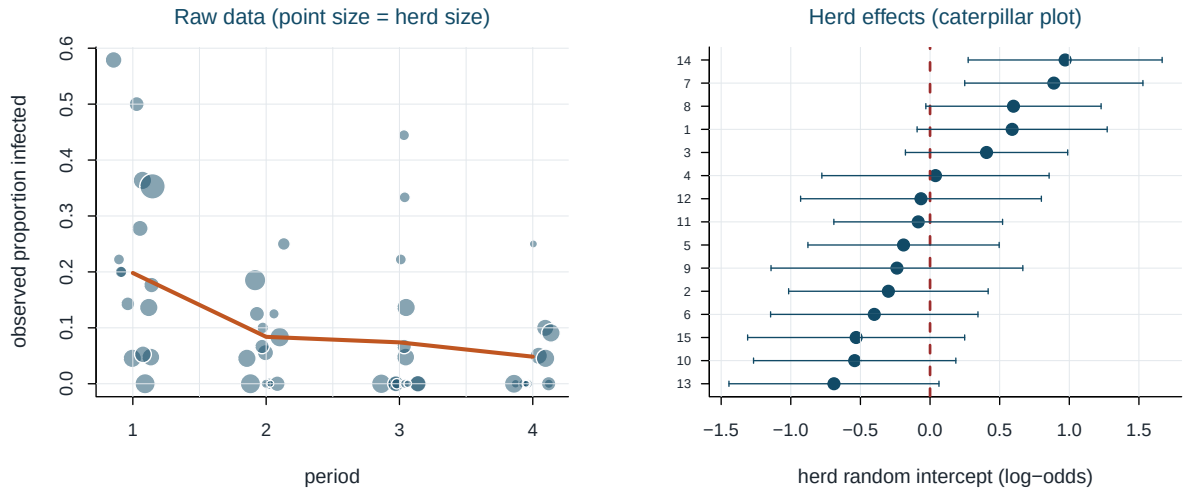


Figure 15.1: Left: observed incidence proportions by herd and period. Right: herd-level random intercepts with prediction intervals; herds well above zero had consistently more disease than the fixed effects alone predict.

15.2 Poisson GLMM

The `grouseticks` data counts ticks on red grouse chicks. Chicks are in broods, broods are on locations — two levels of nesting.

```
data(grouseticks)
grouseticks$cHEIGHT <- scale(grouseticks$HEIGHT, scale = FALSE)   # centre it
gp <- glmer(TICKS ~ cHEIGHT + YEAR + (1 | BROOD) + (1 | LOCATION),
            family = poisson, data = grouseticks)
summary(gp)

#> Generalized linear mixed model fit by maximum likelihood (Laplace Approximation) [glmerMod
```

```
#> ]
#>  Family: poisson  ( log )
#> Formula: TICKS ~ cHEIGHT + YEAR + (1 | BROOD) + (1 | LOCATION)
#>    Data: grouseticks
#>
#>      AIC      BIC   logLik deviance df.resid
#>   1987.9   2011.9   -987.9   1975.9      397
#>
#> Scaled residuals:
#>    Min     1Q Median     3Q    Max
#> -3.096 -0.771 -0.435  0.681  5.646
#>
#> Random effects:
#>  Groups   Name        Variance Std.Dev.
#>  BROOD    (Intercept) 0.592    0.770
#>  LOCATION (Intercept) 0.330    0.574
#> Number of obs: 403, groups:  BROOD, 118; LOCATION, 63
#>
#> Fixed effects:
#>             Estimate Std. Error z value Pr(>|z|)
#> (Intercept)  0.46686    0.19202    2.43  0.01504 *
#> cHEIGHT     -0.02355    0.00347   -6.80  1.1e-11 ***
#> YEAR96       1.16558    0.22979    5.07  3.9e-07 ***
#> YEAR97      -0.97793    0.25482   -3.84  0.00012 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Correlation of Fixed Effects:
#>         (Intr) cHEIGH YEAR96
#> cHEIGHT  0.010
#> YEAR96  -0.717  0.050
#> YEAR97  -0.599  0.048  0.510
```

```
print(VarCorr(gp), comp = "Variance")


#>  Groups   Name        Variance
#>  BROOD    (Intercept) 0.592
#>  LOCATION (Intercept) 0.330


## Is there still overdispersion left over?
overdisp_glmm <- function(model) {
  rp <- residuals(model, type = "pearson")
  rdf <- nobs(model) - length(fixef(model)) -
         length(unlist(lapply(lme4::VarCorr(model), function(z) z[lower.tri(z, TRUE)])))
  X2 <- sum(rp^2)
  c(chisq = X2, df = rdf, ratio = X2 / rdf,
    p = pchisq(X2, rdf, lower.tail = FALSE))
}
round(overdisp_glmm(gp), 4)


#>   chisq      df   ratio       p
#> 691.137 397.000   1.741   0.000
```

Even with two random effects the dispersion is 1.7 — there is extra variability *between individual chicks*, not just between broods and locations.

15.3 Observation-level random effects

The trick: give every single observation its own random effect. That makes the model a Poisson–lognormal mixture, which absorbs any leftover overdispersion.

```
grouseticks$obs <- factor(seq_len(nrow(grouseticks)))
gp_olre <- glmer(TICKS ~ cHEIGHT + YEAR + (1 | BROOD) + (1 | LOCATION) + (1 | obs),
                 family = poisson, data = grouseticks)
round(c(AIC_plain = AIC(gp), AIC_olre = AIC(gp_olre)), 1)

#> AIC_plain  AIC_olre
#>      1988      1794

print(VarCorr(gp_olre), comp = "Variance")

#>  Groups   Name        Variance
#>  obs      (Intercept) 0.293
#>  BROOD    (Intercept) 0.563
#>  LOCATION (Intercept) 0.280

round(overdisp_glmm(gp_olre), 4)

#>    chisq       df    ratio        p
#> 163.2102 396.0000   0.4121   1.0000
```

An improvement of 193 AIC points, and the dispersion is now near 1.

Why an observation-level random effect works

It says: on top of everything the model already knows, each chick has its own random multiplier on its tick rate. That is precisely the "unobserved heterogeneity" story from Chapter 7 — but with a lognormal mixing distribution instead of a gamma one, which is what makes it fit naturally inside a GLMM.

OLRE needs enough data

With small counts and few observations per group, an observation-level random effect can be weakly identified and `glmer` may struggle to converge. If it does, use a negative binomial GLMM instead — it does the same job with a different mixing distribution and usually fits more easily.

15.4 Negative binomial GLMM

Two routes. `glmer.nb` in lme4, and `glmmTMB`, which is faster and far more flexible.

```
library(glmmTMB)
gnb <- glmmTMB(TICKS ~ cHEIGHT + YEAR + (1 | BROOD) + (1 | LOCATION),
               family = nbinom2, data = grouseticks)
summary(gnb)

#>  Family: nbinom2  ( log )
#> Formula:          TICKS ~ cHEIGHT + YEAR + (1 | BROOD) + (1 | LOCATION)
#> Data: grouseticks
```

```
#>
#>      AIC      BIC   logLik deviance df.resid
#>   1791.8   1819.8   -888.9   1777.8      396
#>
#> Random effects:
#>
#> Conditional model:
#>  Groups   Name        Variance Std.Dev.
#>  BROOD    (Intercept) 0.487    0.698
#>  LOCATION (Intercept) 0.352    0.593
#> Number of obs: 403, groups:  BROOD, 118; LOCATION, 63
#>
#> Dispersion parameter for nbinom2 family (): 3.29
#>
#> Conditional model:
#>             Estimate Std. Error z value Pr(>|z|)
#> (Intercept)  0.52371    0.19475    2.69  0.00716 **
#> cHEIGHT     -0.02368    0.00351   -6.75  1.5e-11 ***
#> YEAR96       1.16717    0.23289    5.01  5.4e-07 ***
#> YEAR97      -0.98824    0.25710   -3.84  0.00012 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
round(c(poisson_glmm = AIC(gp), olre = AIC(gp_olre), negbin_glmm = AIC(gnb)), 1)

#> poisson_glmm         olre  negbin_glmm
#>         1988         1794         1792
```

Choosing between OLRE and NB GLMM

> They solve the same problem with different mixing distributions — lognormal versus gamma — and usually give very similar answers. Prefer the negative binomial GLMM: it fits faster, converges more reliably, gives you θ as an interpretable number, and extends cleanly to zero-inflation. Use OLRE when you want to stay inside lme4 or when you need the observation-level effects themselves.

15.5 Zero-inflated GLMM

glmmTMB handles zero-inflation and random effects together — something pscl cannot do.

```
data(epil, package = "MASS")
epil$lbase <- log(epil$base / 4)
epil$lage  <- log(epil$age)

m_pois_re <- glmmTMB(y ~ lbase + trt + lage + (1 | subject),
                     family = poisson, data = epil)
m_nb_re   <- glmmTMB(y ~ lbase + trt + lage + (1 | subject),
                     family = nbinom2, data = epil)
m_zi_re   <- glmmTMB(y ~ lbase + trt + lage + (1 | subject),
                     ziformula = ~ 1, family = nbinom2, data = epil)
round(c(poisson_re = AIC(m_pois_re), negbin_re = AIC(m_nb_re),
        zi_negbin_re = AIC(m_zi_re)), 1)

#>   poisson_re    negbin_re zi_negbin_re
#>         1352         1266         1261
```

Adding zero-inflation buys nothing here (AIC barely moves, and the extra parameter costs 2 points), which tells you these seizure counts have no structural-zero group. That is a genuinely useful negative result.

```
summary(m_nb_re)
```

```
#>  Family: nbinom2  ( log )
#> Formula:          y ~ lbase + trt + lage + (1 | subject)
#> Data: epil
#>
#>      AIC      BIC   logLik deviance df.resid
#>   1266.4   1287.2   -627.2   1254.4      230
#>
#> Random effects:
#>
#> Conditional model:
#>  Groups  Name        Variance Std.Dev.
#>  subject (Intercept) 0.231    0.481
#> Number of obs: 236, groups:  subject, 59
#>
#> Dispersion parameter for nbinom2 family (): 7.19
#>
#> Conditional model:
#>              Estimate Std. Error z value Pr(>|z|)
#> (Intercept)    -1.108      1.199   -0.92    0.356
#> lbase           1.028      0.101   10.15   <2e-16 ***
#> trtprogabide   -0.314      0.151   -2.08    0.037 *
#> lage            0.332      0.343    0.97    0.334
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
ci <- confint(m_nb_re, parm = "beta_")
round(exp(ci), 3)
```

```
#>              2.5 % 97.5 % Estimate
#> (Intercept)  0.031  3.465    0.330
#> lbase        2.291  3.407    2.794
#> trtprogabide 0.543  0.982    0.730
#> lage         0.711  2.732    1.394
```

The reportable result

> "Adjusting for baseline seizure rate and age, progabide was associated with an incidence rate ratio of 0.73 (95% CI 0.54–0.98), from a negative binomial mixed model with a random intercept per patient." The interval excludes 1, but only just, so this is weak evidence of a reduction in seizure rate rather than a firm finding.

Compare that with what you would have concluded from the naive Poisson GLM of Chapter 6, which ignored the repeated measurements entirely:

```
naive <- glm(y ~ lbase + trt + lage, family = poisson, data = epil)
round(rbind(
  naive_poisson_glm = c(IRR = exp(coef(naive)["trtprogabide"]),
                        exp(confint.default(naive)["trtprogabide", ])),
  negbin_mixed      = c(IRR = exp(fixef(m_nb_re)$cond["trtprogabide"]),
                        exp(ci["trtprogabide", 1:2]))
), 3)
```

```
#>                   IRR.trtprogabide 2.5 % 97.5 %
#> naive_poisson_glm            0.983 0.895  1.081
#> negbin_mixed                 0.730 0.543  0.982
```

Same point estimate, an interval roughly 2.4 times wider. The naive model was not wrong about the effect; it was wrong about how sure we should be.

15.6 Predicting from a GLMM

The single most confusing part of GLMMs. There are two different predictions and they answer different questions.

```
nd <- data.frame(period = factor(1:4), herd = "1")

## Conditional: "for THIS herd"
round(predict(m_glmm, newdata = nd, type = "response"), 4)

#>      1      2      3      4
#> 0.3082 0.1418 0.1260 0.0841

## Marginal-ish: "for a typical herd, random effect set to zero"
round(predict(m_glmm, newdata = nd, re.form = NA, type = "response"), 4)

#>      1      2      3      4
#> 0.1981 0.0839 0.0740 0.0484
```

`re.form = NA` is not the population average

> Setting the random effect to zero gives you the prediction for a herd with an *average latent value*, not the average prediction across herds. Because the inverse link is non-linear, those differ — and the gap grows with the random effect variance. Chapter 16 explains this properly and shows how to get a true population average.

```
## Predicting for a herd that was never observed: allow.new.levels
predict(m_glmm, newdata = data.frame(period = factor(1), herd = "brand_new"),
        allow.new.levels = TRUE, type = "response")

#>      1
#> 0.1981
```

15.7 Simulating from a GLMM to check it

```
set.seed(4)
sims <- simulate(m_nb_re, nsim = 300)
p0   <- sapply(sims, function(z) mean(z == 0))
mx   <- sapply(sims, function(z) max(z))
par(mfrow = c(1, 2)); par_book()
hist(p0, breaks = 25, col = adjustcolor(PAL["primary"], 0.55), border = "white",
     xlab = "proportion of zeros", main = "Zeros: simulated vs observed")
abline(v = mean(epil$y == 0), lwd = 3, col = PAL["red"])
hist(mx, breaks = 25, col = adjustcolor(PAL["primary"], 0.55), border = "white",
     xlab = "maximum count", main = "Maximum: simulated vs observed")
abline(v = max(epil$y), lwd = 3, col = PAL["red"])
```

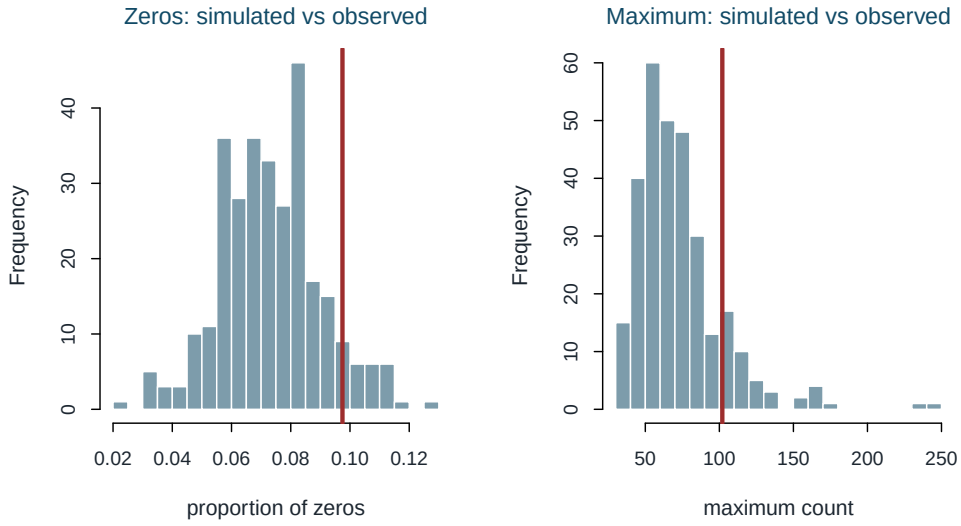


Figure 15.2: Simulating 300 datasets from the negative binomial GLMM and comparing the proportion of zeros and the maximum count with what was observed. Both observed values sit comfortably inside the simulated distributions.

15.8 A summary of the GLMM toolkit

Model	lme4	glmmTMB
Linear mixed	`lmer(y ~ x + (1|g))`	`glmmTMB(y ~ x + (1|g))`
Logistic mixed	`glmer(..., binomial)`	`family = binomial`
Poisson mixed	`glmer(..., poisson)`	`family = poisson`
Negative binomial mixed	`glmer.nb(...)`	`family = nbinom2` (or `nbinom1`)
Beta-binomial mixed	not available	`family = betabinomial`
Zero-inflated mixed	not available	`ziformula = ~ 1`
Hurdle mixed	not available	`family = truncated_nbinom2`
Overdispersed Poisson	`+ (1|obs)` (OLRE)	`family = nbinom2`
Spatial / temporal correlation	not available	`ou()`, `ar1()`, `exp()`

Rule of thumb

Learn lme4 first — it is the standard and its syntax is everywhere. Move to glmmTMB the moment you need a negative binomial, zero-inflation, a beta-binomial, or correlation structures. The formula syntax is identical, so switching costs you nothing.

CHAPTER 16

Conditional vs marginal effects, and GEE

16.1 Two different questions

Take a logistic mixed model for whether patients recover, with a random effect per hospital. There are two sensible questions you might be asking:

1. **Conditional (subject-specific).** "For a *given hospital*, how much does the treatment change the odds of recovery?"
2. **Marginal (population-average).** "Across *all hospitals*, how much does the treatment change the odds of recovery?"

For a linear model these are the same number. For a logistic model they are not — and the difference can be large.

Why they differ

> The inverse-logit curve is bent. Averaging probabilities across hospitals and then converting to odds is not the same as converting each hospital's odds and then averaging. Whenever the link is non-linear, "average of the effects" and "effect on the average" part company.

16.2 Seeing the attenuation

```
set.seed(123)
n_g <- 200; n_per <- 20
g   <- rep(1:n_g, each = n_per)
x   <- rbinom(n_g * n_per, 1, 0.5)
u   <- rep(rnorm(n_g, 0, 2), each = n_per)     # big between-group variation
eta <- -0.5 + 1.5 * x + u                      # TRUE conditional effect = 1.5
y   <- rbinom(length(eta), 1, plogis(eta))
d   <- data.frame(y, x, g = factor(g))

m_cond <- lme4::glmer(y ~ x + (1 | g), family = binomial, data = d)
m_marg <- glm(y ~ x, family = binomial, data = d)
round(c(true_conditional = 1.5,
        glmm_conditional = lme4::fixef(m_cond)["x"],
        glm_marginal     = coef(m_marg)["x"]), 3)

#>   true_conditional glmm_conditional.x     glm_marginal.x
#>              1.500              1.687              1.134
```

The mixed model estimates 1.69, in the region of the true conditional effect of 1.5. The ordinary logistic regression returns 1.13 — far below it, and nowhere near the truth we put in. That gap is not sampling error and it is not bias: the two models are estimating genuinely different quantities, and only one of them is the number we simulated.

There is a good approximation for how much the marginal effect shrinks:

$$\beta_{\text{marginal}} \approx \frac{\beta_{\text{conditional}}}{\sqrt{1 + 0.346\,\sigma_u^2}}$$

```
sig2 <- as.data.frame(lme4::VarCorr(m_cond))$vcov[1]
round(c(conditional = lme4::fixef(m_cond)["x"],
        predicted_marginal = lme4::fixef(m_cond)["x"] / sqrt(1 + 0.346 * sig2),
        actual_marginal = coef(m_marg)["x"]), 3)

#>        conditional.x predicted_marginal.x    actual_marginal.x
#>                1.687                1.161                1.134
```

```
par_book()
xs <- seq(-4, 4, length.out = 300)
plot(xs, plogis(xs), type = "n", ylim = c(0, 1), xlab = "linear predictor for x = 0",
     ylab = "probability", main = "Averaging bent curves flattens them")
grid_book()
us <- rnorm(60, 0, 2)
for (u_ in us) lines(xs, plogis(xs + u_), col = adjustcolor(PAL["grey"], 0.5), lwd = 1)
avg <- sapply(xs, function(z) mean(plogis(z + us)))
lines(xs, plogis(xs), lwd = 3, col = PAL["primary"])
lines(xs, avg, lwd = 4, col = PAL["accent"])
legend("topleft", bty = "n", cex = 0.8, lwd = c(1, 3, 4),
       col = c(PAL["grey"], PAL["primary"], PAL["accent"]),
       legend = c("individual groups", "a typical group (u = 0)",
                  "average across groups"))
```

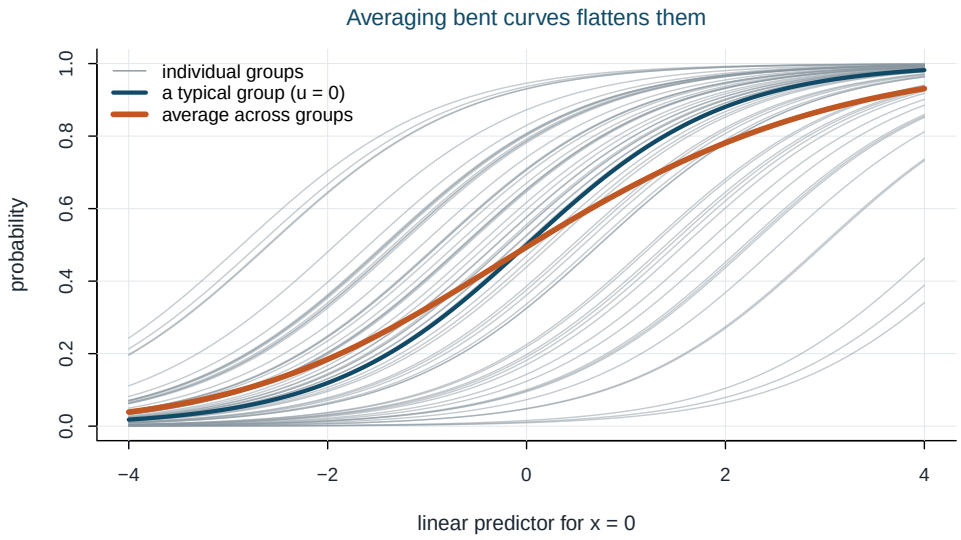


Figure 16.1: Thin grey curves are individual groups, each with the same conditional slope. Their average (the thick orange curve) is visibly flatter than any of them: that flattening is the attenuation of the marginal effect.

The exception that saves count models

For a **log** link — Poisson and negative binomial — the slope coefficients are the *same* conditionally and marginally. Only the intercept shifts, by $\sigma_u^2/2$. That is because $\mathrm{E}[e^u] = e^{\sigma_u^2/2}$ is a constant multiplier that the log link absorbs entirely into the intercept. So this whole headache applies to logistic mixed models, not to Poisson ones.

```
set.seed(9)
u2 <- rep(rnorm(300, 0, 0.8), each = 8)
x2 <- rnorm(2400)
```

```
y2 <- rpois(2400, exp(0.5 + 0.7 * x2 + u2))
d2 <- data.frame(y2, x2, g2 = factor(rep(1:300, each = 8)))
round(c(true_slope = 0.7,
        glmm  = lme4::fixef(lme4::glmer(y2 ~ x2 + (1 | g2), family = poisson, data = d2))["x2"],
        glm   = coef(glm(y2 ~ x2, family = poisson, data = d2))["x2"]), 3)

#> true_slope    glmm.x2     glm.x2
#>      0.700      0.673      0.677
```

Both recover 0.7. The slopes agree, exactly as the theory says.

16.3 Generalised estimating equations (GEE)

If the population-average effect is what you want, you can estimate it directly. GEE models the marginal mean, specifies a *working* correlation structure within clusters, and then computes robust standard errors that stay valid even if that working structure is wrong.

```
library(geepack)
data(epil, package = "MASS")
epil$lbase <- log(epil$base / 4)
epil <- epil[order(epil$subject), ]              # GEE requires sorted clusters

gee_ex <- geeglm(y ~ lbase + trt, id = subject, family = poisson,
                 corstr = "exchangeable", data = epil)
summary(gee_ex)


#>
#> Call:
#> geeglm(formula = y ~ lbase + trt, family = poisson, data = epil,
#>     id = subject, corstr = "exchangeable")
#>
#>  Coefficients:
#>              Estimate Std.err  Wald Pr(>|W|)
#> (Intercept)    -0.312   0.365  0.73     0.39
#> lbase           1.176   0.148 62.90  2.2e-15 ***
#> trtprogabide   -0.103   0.194  0.28     0.59
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Correlation structure = exchangeable
#> Estimated Scale Parameters:
#>
#>             Estimate Std.err
#> (Intercept)     4.81    1.16
#>   Link = identity
#>
#> Estimated Correlation Parameters:
#>       Estimate Std.err
#> alpha    0.404  0.0735
#> Number of clusters:   59  Maximum cluster size: 4
```

```
gee_ind <- geeglm(y ~ lbase + trt, id = subject, family = poisson,
                  corstr = "independence", data = epil)
gee_ar1 <- geeglm(y ~ lbase + trt, id = subject, family = poisson,
                  corstr = "ar1", data = epil)
round(cbind(
  independence = coef(gee_ind),
  exchangeable = coef(gee_ex),
```

```
  ar1          = coef(gee_ar1)
), 4)

#>              independence exchangeable    ar1
#> (Intercept)        -0.312       -0.312 -0.353
#> lbase               1.177        1.177  1.195
#> trtprogabide       -0.103       -0.103 -0.122

round(cbind(
  se_independence = coef(summary(gee_ind))[, 2],
  se_exchangeable = coef(summary(gee_ex))[, 2],
  se_ar1          = coef(summary(gee_ar1))[, 2]
), 4)

#>      se_independence se_exchangeable se_ar1
#> [1,]           0.365           0.365  0.386
#> [2,]           0.148           0.148  0.158
#> [3,]           0.194           0.194  0.193
```

corstr	Assumes	Use when
"independence"	no within-cluster correlation	always safe; least efficient
"exchangeable"	every pair equally correlated	clustering with no natural order (pupils in schools)
"ar1"	correlation decays with distance in time	evenly spaced repeated measures
"unstructured"	every pair its own correlation	few time points, many clusters

Choose between them with the QIC, GEE's analogue of AIC:

```
round(rbind(independence = QIC(gee_ind),
            exchangeable = QIC(gee_ex),
            ar1          = QIC(gee_ar1)), 1)

#>                QIC  QICu Quasi Lik  CIC params  QICC
#> independence -5838 -5857      2931 12.5      3 -5838
#> exchangeable -5838 -5857      2931 12.5      3 -5837
#> ar1          -5836 -5856      2931 13.0      3 -5836
```

Why the working structure matters less than you would think

> The robust "sandwich" standard errors are consistent even if the working correlation is wrong. Getting it approximately right buys efficiency — narrower intervals — not validity. That is GEE's central selling point.

16.4 GLMM or GEE?

	GLMM	GEE
Estimates	subject-specific (conditional)	population-average (marginal)
Random effects	explicitly modelled	not modelled, only adjusted for
Gives you σ_u^2, ICC	yes	no
Predict for a specific cluster	yes	no
Predict for a new cluster	yes	only the average
Handles missing data	MAR	MCAR (unless weighted)
Needs many clusters	works with moderate numbers	prefers 40+
Likelihood, AIC	yes	no (QIC instead)
Computationally	slower, can fail to converge	fast and robust

Which to use

GLMM when the clusters themselves are of interest — you want to know how much hospitals vary, or to predict for a specific hospital, or the clustering is part of the science.
GEE when clusters are a nuisance to be adjusted for, and you want a policy statement about the population: "if everybody switched treatment, the average rate would change by this much."
Either for Poisson or negative binomial slopes — they agree.

16.5 Getting marginal effects out of a GLMM

You do not have to abandon the mixed model to get a population-average answer. Integrate the fitted model over the random-effect distribution:

```
data(cbpp, package = "lme4")
m_glmm <- lme4::glmer(cbind(incidence, size - incidence) ~ period + (1 | herd),
                      family = binomial, data = cbpp)
sig <- sqrt(as.data.frame(lme4::VarCorr(m_glmm))$vcov[1])
nd  <- data.frame(period = factor(1:4))
eta <- predict(m_glmm, newdata = nd, re.form = NA)      # conditional on u = 0

## Average the probability over the distribution of the random effect
marg <- sapply(eta, function(e)
  integrate(function(u) plogis(e + u) * dnorm(u, 0, sig), -8 * sig, 8 * sig)$value)

round(data.frame(period = 1:4,
                 conditional_u0 = plogis(eta),
                 population_average = marg), 4)

#>   period conditional_u0 population_average
#> 1      1         0.1981             0.2161
#> 2      2         0.0839             0.0971
#> 3      3         0.0740             0.0862
#> 4      4         0.0484             0.0573
```

The population-average probabilities are pulled towards 0.5 relative to the typical-herd probabilities, which is the same attenuation seen earlier.

The emmeans package will do this for you, and gives standard errors too:

```r
library(emmeans)
emmeans(m_glmm, ~ period, type = "response", bias.adjust = TRUE, sigma = sig)
```

```
#>  period   prob     SE  df asymp.LCL asymp.UCL
#>  1      0.2179 0.0371 Inf    0.1533     0.298
#>  2      0.0971 0.0263 Inf    0.0562     0.161
#>  3      0.0861 0.0251 Inf    0.0477     0.149
#>  4      0.0570 0.0225 Inf    0.0258     0.120
#>
#> Confidence level used: 0.95
#> Intervals are back-transformed from the logit scale
#> Bias adjustment applied based on sigma = 0.64207
```

The three numbers you can report, and what each means

Conditional with $u = 0$: a herd of exactly average latent risk.
Population average: the mean across all herds — what a policy-maker wants.
Conditional on a specific u_j: that particular herd — what its veterinarian wants.
Say which one you are reporting. Most papers do not, and readers then cannot tell.

CHAPTER 17

When the model will not fit

Mixed models fail to converge more often than any other model in this book. This chapter is a field guide to the warnings and what to do about each one.

17.1 "boundary (singular) fit"

```
set.seed(15)
d <- data.frame(g = factor(rep(1:8, each = 6)), x = rnorm(48))
d$y <- 2 + 0.6 * d$x + rnorm(48)          # NO group effect at all
m <- lme4::lmer(y ~ x + (1 | g), data = d)
lme4::VarCorr(m)

#>  Groups   Name        Std.Dev.
#>  g        (Intercept) 0.307
#>  Residual             0.969

lme4::isSingular(m)

#> [1] FALSE
```

What it means

A variance component was estimated as zero (or a correlation as exactly ± 1). The model is sitting on the edge of its parameter space. The estimate is not *wrong*, but the model is more complex than the data can support.

Common causes and their fixes:

Cause	Fix
There genuinely is no group-level variation	Drop the random effect; report why
Too few groups (fewer than about 5)	Fit the grouping as a fixed effect instead
Random slope not supported by the data	Simplify: `(x|g)` → `(1|g)`
Correlation between intercept and slope near ± 1	Use `(x||g)` to force independence
Too many random terms	Remove them one at a time, comparing with LRT

```
m_simple <- lm(y ~ x, data = d)
round(c(lmer_slope = lme4::fixef(m)["x"], lm_slope = coef(m_simple)["x"]), 4)

#> lmer_slope.x   lm_slope.x
#>        0.687        0.713
```

With a zero variance component, the mixed model has collapsed to the ordinary linear model anyway — which is why the estimates match exactly.

17.2 "Model failed to converge with max|grad| . . . "

This says the optimiser stopped somewhere the gradient is not quite zero. Sometimes it is a real failure; often it is a false alarm from a conservative check.

17.2.1 Step 1: scale your predictors

By far the most common cause, and the easiest fix. Optimisers struggle when predictors are on wildly different scales.

```
set.seed(6)
n  <- 600
dd <- data.frame(
  g      = factor(rep(1:30, each = 20)),
  income = rnorm(n, 50000, 15000),        # ~10^4
  age    = rnorm(n, 40, 12)               # ~10^1
)
dd$y <- rbinom(n, 1, plogis(-3 + 0.00002 * dd$income + 0.02 * dd$age +
                            rep(rnorm(30, 0, 0.5), each = 20)))

## Unscaled: the two predictors differ by four orders of magnitude
bad <- lme4::glmer(y ~ income + age + (1 | g), family = binomial, data = dd)

#> Warning:  Some predictor variables are on very different scales:  consider rescaling
#> Warning in checkConv(attr(opt, "derivs"), opt$par, ctrl = control$checkConv, :  unable to
evaluate scaled gradient
#> Warning in checkConv(attr(opt, "derivs"), opt$par, ctrl = control$checkConv, :  Model
failed to converge:  degenerate Hessian with 1 negative eigenvalues

## Scaled: same model, well-conditioned
dd$income_s <- scale(dd$income); dd$age_s <- scale(dd$age)
good <- lme4::glmer(y ~ income_s + age_s + (1 | g), family = binomial, data = dd)
round(c(logLik_unscaled = logLik(bad), logLik_scaled = logLik(good)), 4)

#> logLik_unscaled   logLik_scaled
#>            -305            -305
```

The two models are statistically identical, but the scaled version is far easier to optimise. Rescale coefficients afterwards by dividing by the standard deviation of the original variable.

Rule of thumb

> Centre and scale every continuous predictor before fitting a GLMM. It costs one line, it never hurts, and it prevents most convergence warnings.

17.2.2 Step 2: try other optimisers

If several independent optimisers land in the same place, the warning was a false alarm.

```
af <- lme4::allFit(good, verbose = FALSE)
ll <- sort(sapply(af[!sapply(af, is, "error")], logLik), decreasing = TRUE)
round(ll, 5)

#>                         bobyqa     nloptwrap.NLOPT_LN_BOBYQA                    nlminbwrap
#>                           -305                           -305                          -305
#>                   Nelder_Mead nloptwrap.NLOPT_LN_NELDERMEAD
#>                           -305                           -305
```

```
round(diff(range(ll)), 8)          # spread across optimisers

#> [1] 4e-08
```

All optimisers agree to 3.7×10^{-8} of a log-likelihood unit. That is convergence, whatever the warning said.

```
## Change the optimiser directly
lme4::glmer(y ~ income_s + age_s + (1 | g), family = binomial, data = dd,
            control = lme4::glmerControl(optimizer = "bobyqa",
                                         optCtrl = list(maxfun = 2e5)))
```

17.2.3 Step 3: increase the accuracy of the likelihood

For binomial GLMMs with a single random effect, `nAGQ` controls how accurately the random effects are integrated out. The default (`nAGQ = 1`, the Laplace approximation) can be poor with small clusters or rare events.

```
data(cbpp, package = "lme4")
f <- cbind(incidence, size - incidence) ~ period + (1 | herd)
round(sapply(c(1, 5, 15), function(q)
  lme4::fixef(lme4::glmer(f, family = binomial, data = cbpp, nAGQ = q))), 4)

#>                [,1]   [,2]   [,3]
#> (Intercept) -1.398 -1.399 -1.399
#> period2     -0.992 -0.991 -0.991
#> period3     -1.128 -1.128 -1.128
#> period4     -1.580 -1.579 -1.579
```

The estimates barely move, so the Laplace approximation was fine here. If they *do* move noticeably, use the higher `nAGQ` value.

`nAGQ > 1` has limits

> It only works with a *single*, scalar random effect. With crossed random effects or random slopes you are restricted to `nAGQ = 1`. If accuracy is a concern there, switch to glmmTMB, which uses a different and generally more accurate approximation, or go Bayesian with brms.

17.2.4 Step 4: simplify the random structure

The most reliable fix of all. Random effects are expensive: `(1 + x + z | g)` estimates six parameters (three variances and three correlations) from however many groups you have.

Random structure	Parameters	Groups you need
`(1 | g)`	1	5+ (ideally 20+)
`(x || g)`	2	20+
`(x | g)`	3	30+
`(x + z | g)`	6	50+
`(x * z | g)`	10	rarely justified

The order to remove things

1. Correlations between random terms (`|` → `||`). 2. Random slopes for interactions. 3. Random slopes for predictors with little within-group variation. 4. Whole random effects with tiny estimated variances. Never remove a random effect that reflects the actual design of your study just because it is inconvenient — if pupils are in schools, that structure is real.

17.3 "fixed-effect model matrix is rank deficient"

Two of your predictors carry the same information — perfectly collinear, or a factor level that never occurs in the data.

```
dz <- data.frame(x = rnorm(50)); dz$x2 <- 2 * dz$x        # exact duplicate
dz$y  <- rpois(50, exp(1 + 0.4 * dz$x))
mz <- glm(y ~ x + x2, family = poisson, data = dz)
coef(mz)

#> (Intercept)           x          x2
#>       1.161       0.333          NA
```

R drops the redundant column and returns `NA`. Find the culprit before proceeding:

```
## Which factor combinations never occur?
data(quine, package = "MASS")
table(quine$Age, quine$Lrn)

#>
#>      AL SL
#>   F0 19  8
#>   F1 15 31
#>   F2 16 24
#>   F3 33  0

## Variance inflation factors for continuous predictors
car::vif(glm(Days ~ Eth + Sex + Age + Lrn, family = poisson, data = quine))

#>     GVIF Df GVIF^(1/(2*Df))
#> Eth 1.00  1            1.00
#> Sex 1.09  1            1.04
#> Age 1.65  3            1.09
#> Lrn 1.61  1            1.27
```

A GVIF (last column, squared and scaled) above about 2–3 is worth investigating; above 5 usually means two predictors are measuring the same thing.

Notice also that `Age = F3` never occurs with `Lrn = SL`. Any model with an `Age:Lrn` interaction would be rank deficient for exactly that reason.

17.4 glmmTMB warnings

glmmTMB reports problems differently.

Message	Meaning and fix
`NA/NaN function evaluation`	Usually harmless — the optimiser probed an illegal region and backed out. Check the fit converged.
`Model convergence problem; non-positive-definite Hessian`	Serious. A parameter is unidentified. Simplify the model.
`Std. Errors are NaN`	Same cause: the model is over-parameterised.
Variance estimate at zero	Same as a singular fit in lme4. Drop that term.

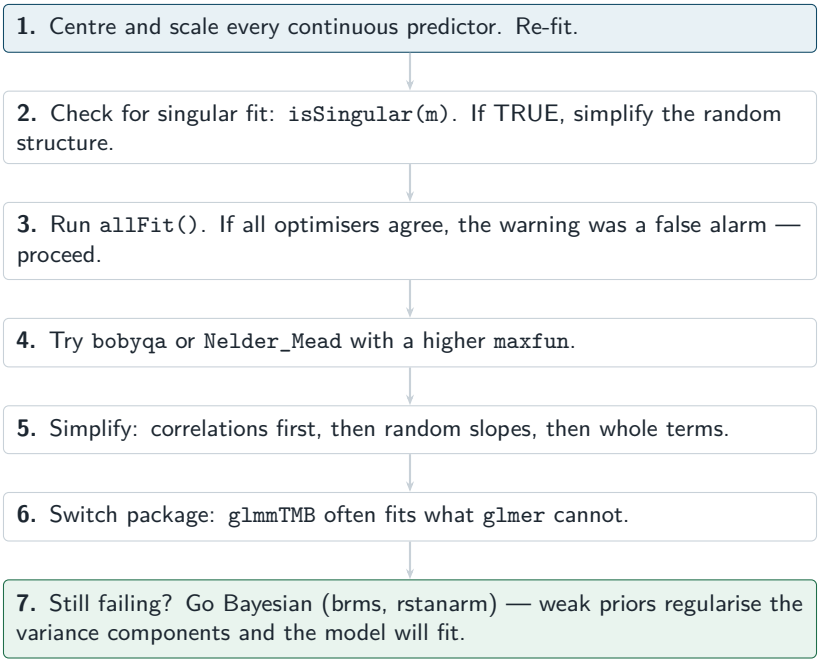
```
library(glmmTMB)
data(grouseticks, package = "lme4")
grouseticks$cH <- scale(grouseticks$HEIGHT)
gt <- glmmTMB(TICKS ~ cH + YEAR + (1 | BROOD), family = nbinom2, data = grouseticks)
gt$fit$convergence            # 0 means the optimiser converged

#> [1] 0

gt$sdr$pdHess                 # TRUE means the Hessian is well behaved

#> [1] TRUE
```

17.5 A triage flowchart



17.6 Warnings you can safely ignore

- `non-integer #successes in a binomial glm!` — expected when you use `weights` with proportions.
- `non-integer x = ...` from a quasi-Poisson on continuous data — expected, and harmless, because quasi-likelihood does not require integers.
- `max|grad| = 0.0015` where `allFit` shows all optimisers agreeing to many decimal places.

- `fitted probabilities numerically 0 or 1` *if* you have checked there is no separation and the coefficients are sensible.

Warnings you must never ignore

Coefficients in the hundreds. Standard errors of `NaN` or in the thousands. A non-positive-definite Hessian. A variance component of exactly zero that your design says should not be zero. Each of these means a number in your output is not what it claims to be.

CHAPTER 18

A complete worked analysis

This chapter runs the whole workflow end to end on one dataset. It is simulated, deliberately: because we know the true answer, you can see at each step whether the method is recovering it or being fooled.

18.1 The scenario

A health programme is evaluated across 40 clinics. Each clinic enrols patients, and for each patient we record the number of malaria episodes in a follow-up period. Key features, all of which are typical of real data:

- follow-up time differs between patients ⇒ we need an **offset**;
- patients are nested in clinics ⇒ we need a **random effect**;
- patients differ beyond what we measured ⇒ **overdispersion**;
- the treatment is assigned **at clinic level**, which is where ignoring clustering does the most damage.

```
set.seed(2025)
n_clinic  <- 40
n_per     <- 40
clinic    <- rep(1:n_clinic, each = n_per)
treated   <- rep(rbinom(n_clinic, 1, 0.5), each = n_per)    # assigned per CLINIC
clinic_re <- rep(rnorm(n_clinic, 0, 0.45), each = n_per)    # clinic variation
age       <- round(runif(n_clinic * n_per, 1, 60))
bednet    <- rbinom(n_clinic * n_per, 1, 0.4)
fu_years  <- round(runif(n_clinic * n_per, 0.25, 2.0), 2)  # unequal follow-up

## TRUE model: log rate = 0.2 - 0.45*treated - 0.012*age - 0.35*bednet + clinic RE
log_rate  <- 0.2 - 0.45 * treated - 0.012 * age - 0.35 * bednet + clinic_re
mu        <- exp(log_rate + log(fu_years))
episodes  <- rnbinom(length(mu), mu = mu, size = 2.5)      # overdispersed

mal <- data.frame(
  episodes, fu_years, clinic = factor(clinic),
  treated = factor(treated, 0:1, c("control", "programme")),
  age, bednet = factor(bednet, 0:1, c("no", "yes"))
)
str(mal)


#> 'data.frame': 1600 obs. of  6 variables:
#>  $ episodes: num  0 0 0 0 1 2 0 1 0 0 ...
#>  $ fu_years: num  1.72 0.83 0.84 1.33 0.72 1.93 1.02 1.35 1.99 0.81 ...
#>  $ clinic  : Factor w/ 40 levels "1","2","3","4",..: 1 1 1 1 1 1 1 1 1 1 ...
#>  $ treated : Factor w/ 2 levels "control","programme": 2 2 2 2 2 2 2 2 2 2 ...
#>  $ age     : num  44 40 45 34 41 43 55 54 8 44 ...
#>  $ bednet  : Factor w/ 2 levels "no","yes": 1 2 1 1 2 1 2 1 1 2 ...
```

Quantity	True value
Programme rate ratio	$e^{-0.45} = 0.638$
Bednet rate ratio	$e^{-0.35} = 0.705$
Rate ratio per year of age	$e^{-0.012} = 0.988$
Clinic SD (log scale)	0.45
Dispersion θ	2.5

18.2 Step 1: look at the data

```
par(mfrow = c(2, 2)); par_book()
barplot(table(factor(mal$episodes, levels = 0:max(mal$episodes))),
        col = adjustcolor(PAL["primary"], 0.7), border = "white",
        xlab = "malaria episodes", ylab = "patients", main = "Outcome distribution")

hist(mal$fu_years, breaks = 20, col = adjustcolor(PAL["accent"], 0.7),
     border = "white", xlab = "follow-up (years)", main = "Exposure varies 8-fold")

rate_by_clinic <- tapply(mal$episodes, mal$clinic, sum) /
                  tapply(mal$fu_years, mal$clinic, sum)
trt_by_clinic  <- tapply(as.integer(mal$treated), mal$clinic, mean)
stripchart(rate_by_clinic ~ ifelse(trt_by_clinic > 1.5, "programme", "control"),
           vertical = TRUE, method = "jitter", pch = 21,
           bg = adjustcolor(PAL["green"], 0.6), col = "white", cex = 1.3,
           ylab = "clinic episode rate per year", main = "Clinic-level rates")
grid_book()

plot(mal$age, mal$episodes / mal$fu_years, pch = 21,
     bg = adjustcolor(PAL["grey"], 0.35), col = "white", cex = 0.8,
     xlab = "age (years)", ylab = "crude rate per year", main = "Rate against age")
grid_book()
lines(lowess(mal$age, mal$episodes / mal$fu_years), lwd = 3, col = PAL["red"])
```

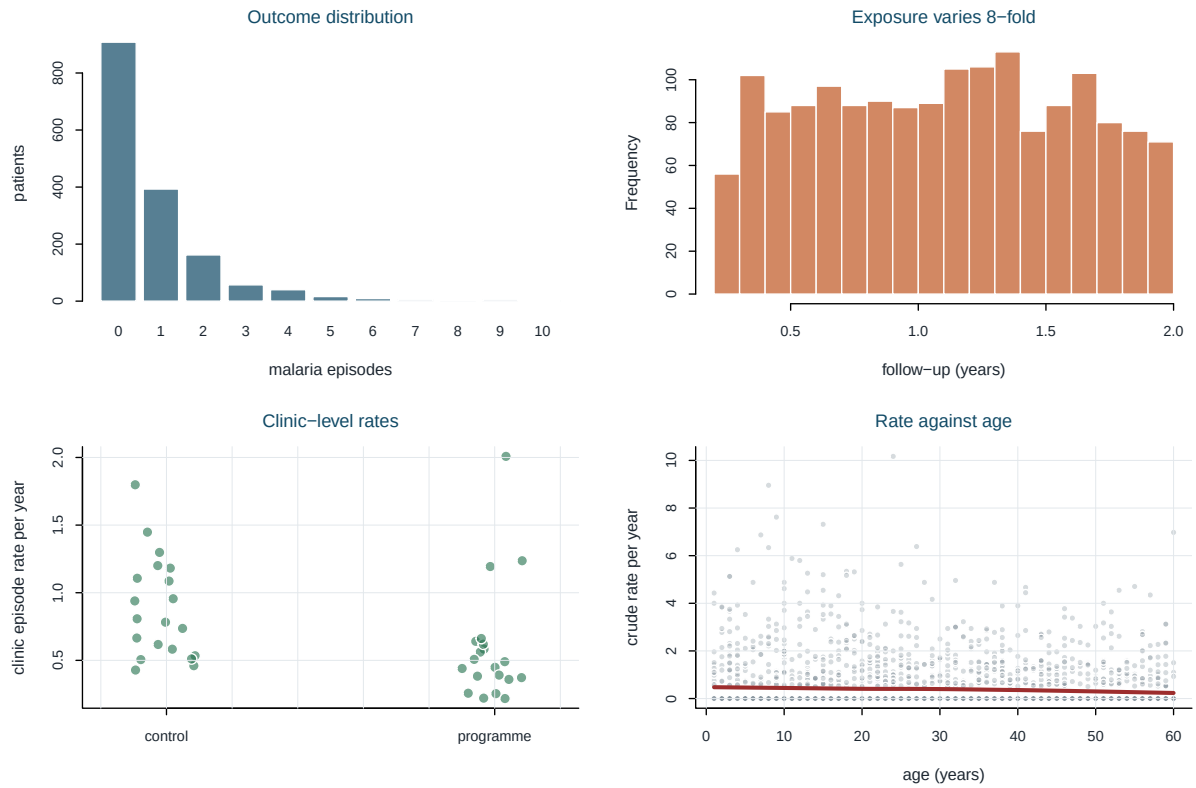


Figure 18.1: Exploratory plots. Note the long right tail, the very different follow-up times, and the visible spread between clinic-level rates – each of which points at a specific modelling decision.

```
c(n_patients = nrow(mal), n_clinics = nlevels(mal$clinic),
  mean_episodes = round(mean(mal$episodes), 2),
  var_episodes = round(var(mal$episodes), 2),
  percent_zero = round(100 * mean(mal$episodes == 0), 1))
```

```
#>    n_patients     n_clinics mean_episodes  var_episodes  percent_zero
#>       1600.00         40.00          0.81          1.79         56.80
```

Variance is already about 2.2 times the mean in the raw data. That is a hint, not proof — some of that spread will be explained by the predictors, and the rest is what the dispersion check in Step 4 has to judge.

18.3 Step 2: the naive model (what not to do)

```
m0 <- glm(episodes ~ treated + age + bednet, family = poisson, data = mal)
round(exp(cbind(IRR = coef(m0), confint.default(m0))), 4)
```

```
#>                     IRR 2.5 % 97.5 %
#> (Intercept)       1.665 1.484  1.869
#> treatedprogramme 0.700 0.626  0.781
#> age              0.986 0.983  0.989
#> bednetyes        0.637 0.565  0.718
```

Two errors are already baked in: no offset, so patients followed longer look sicker; and no clustering, so the intervals are far too narrow. The programme rate ratio is 0.7 against a truth of 0.638.

18.4 Step 3: add the offset

```
m1 <- glm(episodes ~ treated + age + bednet + offset(log(fu_years)),
          family = poisson, data = mal)
round(exp(cbind(IRR = coef(m1), confint.default(m1))), 4)

#>                     IRR 2.5 % 97.5 %
#> (Intercept)       1.505 1.341  1.689
#> treatedprogramme 0.693 0.620  0.773
#> age               0.986 0.983  0.989
#> bednetyes         0.646 0.573  0.728
```

Much better — the programme estimate is now 0.692, close to the true 0.638.

18.5 Step 4: check dispersion

```
disp <- function(m) {
  X2 <- sum(residuals(m, type = "pearson")^2); df <- df.residual(m)
  c(dispersion = X2 / df, p = pchisq(X2, df, lower.tail = FALSE))
}
round(disp(m1), 4)

#> dispersion          p
#>       1.67       0.00

AER::dispersiontest(m1, trafo = 2)

#>
#>  Overdispersion test
#>
#> data:  m1
#> z = 8, p-value = 5e-15
#> alternative hypothesis: true alpha is greater than 0
#> sample estimates:
#> alpha
#> 0.807
```

Dispersion is 1.67, decisively above 1. The standard errors from `m1` are too small by roughly $\sqrt{1.67} = 1.29$.

Before blaming the distribution, check the mean model (Chapter 7's advice):

```
set.seed(3)
qres <- function(y, pfun, ...) {
  a <- pfun(y - 1, ...); b <- pfun(y, ...)
  qnorm(pmin(pmax(runif(length(y), pmin(a,b), pmax(a,b)), 1e-10), 1 - 1e-10))
}
par_book()
qr1 <- qres(mal$episodes, ppois, lambda = fitted(m1))
plot(mal$age, qr1, pch = 21, bg = adjustcolor(PAL["primary"], 0.3), col = "white",
     cex = 0.9, xlab = "age", ylab = "quantile residual",
     main = "Is the effect of age linear on the log scale?")
grid_book(); abline(h = 0, col = PAL["grey"], lwd = 1.5)
lines(lowess(mal$age, qr1), lwd = 3, col = PAL["red"])
```

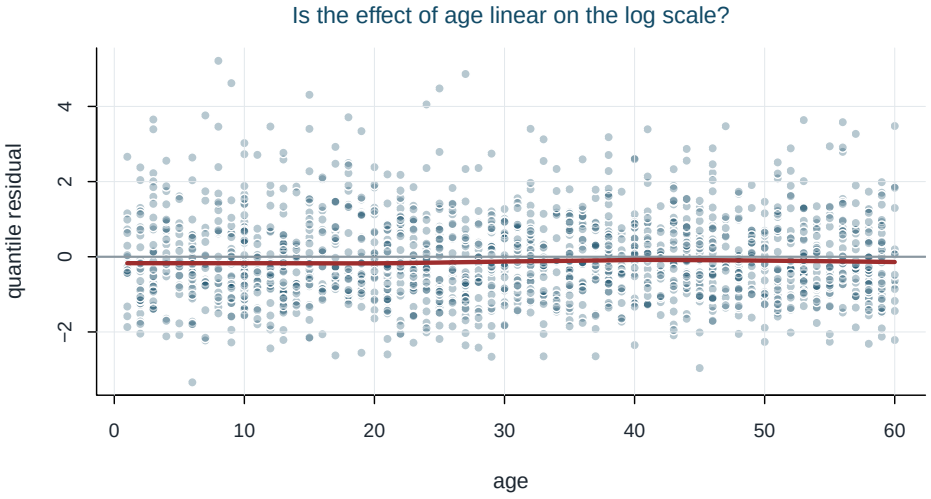


Figure 18.2: Quantile residuals against age. Flat, so a linear term for age is adequate and the overdispersion is not caused by a missing curve.

```
m1_spline <- glm(episodes ~ treated + splines::ns(age, 4) + bednet +
                 offset(log(fu_years)), family = poisson, data = mal)
anova(m1, m1_spline, test = "LRT")

#> Analysis of Deviance Table
#>
#> Model 1: episodes ~ treated + age + bednet + offset(log(fu_years))
#> Model 2: episodes ~ treated + splines::ns(age, 4) + bednet + offset(log(fu_years))
#>   Resid. Df Resid. Dev Df Deviance Pr(>Chi)
#> 1      1596       2308
#> 2      1593       2306  3     2.75     0.43
```

No evidence of curvature. The overdispersion is real, not a misspecified mean.

18.6 Step 5: negative binomial

```
m2 <- MASS::glm.nb(episodes ~ treated + age + bednet + offset(log(fu_years)),
                   data = mal)
c(theta = round(m2$theta, 3), true_theta = 2.5)

#>      theta true_theta
#>       1.28       2.50

round(c(AIC_poisson = AIC(m1), AIC_negbin = AIC(m2)), 1)

#> AIC_poisson  AIC_negbin
#>        3975        3742

pscl::odTest(m2)

#> Likelihood ratio test of H0: Poisson, as restricted NB model:
#> n.b., the distribution of the test-statistic under H0 is non-standard
#> e.g., see help(odTest) for details/references
#>
#> Critical value of test statistic at the alpha= 0.05 level: 2.706
#> Chi-Square Test Statistic =  234.766 p-value = <2e-16
```

$\hat{\theta} = 1.28$, noticeably *smaller* than the true value of 2.5 — and a smaller θ means more dispersion. That is not a mistake, it is a symptom. This model has only one place to put extra variability, so it is absorbing the patient-level heterogeneity we built in *and* the clinic-level variation it has no way to represent. Step 6 separates the two, and $\hat{\theta}$ moves back towards its true value once it does.

18.7 Step 6: add the random effect

```
library(glmmTMB)
m3 <- glmmTMB(episodes ~ treated + age + bednet + offset(log(fu_years)) +
              (1 | clinic), family = nbinom2, data = mal)
summary(m3)
```

```
#>  Family: nbinom2  ( log )
#> Formula:          episodes ~ treated + age + bednet + offset(log(fu_years)) + (1 |
#>     clinic)
#> Data: mal
#>
#>      AIC      BIC   logLik deviance df.resid
#>     3653     3686    -1821     3641     1594
#>
#> Random effects:
#>
#> Conditional model:
#>  Groups Name        Variance Std.Dev.
#>  clinic (Intercept) 0.181    0.426
#> Number of obs: 1600, groups:  clinic, 40
#>
#> Dispersion parameter for nbinom2 family (): 1.87
#>
#> Conditional model:
#>                  Estimate Std. Error z value Pr(>|z|)
#> (Intercept)        0.2975     0.1239    2.40   0.0163 *
#> treatedprogramme  -0.4499     0.1536   -2.93   0.0034 **
#> age               -0.0126     0.0021   -5.98  2.2e-09 ***
#> bednetyes         -0.4032     0.0765   -5.27  1.4e-07 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
sd_clinic <- sqrt(as.numeric(VarCorr(m3)$cond$clinic))
c(estimated_clinic_sd = round(sd_clinic, 3), true_clinic_sd = 0.45,
  estimated_theta = round(sigma(m3), 3), true_theta = 2.5)
```

```
#> estimated_clinic_sd      true_clinic_sd     estimated_theta          true_theta
#>               0.426               0.450               1.872               2.500
```

Both the clinic standard deviation and θ are now close to their true values. Splitting the variation into "between clinics" and "between patients" has let each parameter find its own level.

18.8 Step 7: compare everything

```
truth <- c(treatedprogramme = -0.45, age = -0.012, bednetyes = -0.35)
get <- function(m, nm) {
  cf <- if (inherits(m, "glmmTMB")) fixef(m)$cond else coef(m)
  se <- if (inherits(m, "glmmTMB")) sqrt(diag(vcov(m)$cond))
        else sqrt(diag(vcov(m)))
```

```
  c(est = cf[nm], se = se[nm])
}
tab <- t(sapply(list(
  "1. Poisson, no offset"     = m0,
  "2. Poisson + offset"       = m1,
  "3. Negative binomial"      = m2,
  "4. NB + clinic random eff" = m3
), get, nm = "treatedprogramme"))
out <- data.frame(
  IRR       = round(exp(tab[, 1]), 3),
  lower     = round(exp(tab[, 1] - 1.96 * tab[, 2]), 3),
  upper     = round(exp(tab[, 1] + 1.96 * tab[, 2]), 3),
  CI_width  = round(exp(tab[, 1] + 1.96*tab[, 2]) - exp(tab[, 1] - 1.96*tab[, 2]), 3)
)
out$covers_truth <- out$lower <= exp(-0.45) & out$upper >= exp(-0.45)
out

#>                             IRR lower upper CI_width covers_truth
#> 1. Poisson, no offset     0.700 0.626 0.781    0.155         TRUE
#> 2. Poisson + offset       0.692 0.620 0.773    0.153         TRUE
#> 3. Negative binomial      0.701 0.606 0.811    0.205         TRUE
#> 4. NB + clinic random eff 0.638 0.472 0.862    0.390         TRUE
```

```
par_book()
k <- nrow(out)
plot(out$IRR, k:1, xlim = range(c(out$lower, out$upper, exp(-0.45))) * c(0.95, 1.05),
     ylim = c(0.5, k + 0.5), pch = 19, cex = 1.6, col = PAL["primary"], yaxt = "n",
     xlab = "rate ratio for the programme", ylab = "",
     main = "Each correction widens the interval")
axis(2, at = k:1, labels = rownames(out), las = 1, cex.axis = 0.65)
grid_book()
arrows(out$lower, k:1, out$upper, k:1, code = 3, angle = 90, length = 0.05,
       lwd = 2.5, col = PAL["primary"])
abline(v = exp(-0.45), lwd = 2.5, lty = 2, col = PAL["red"])
abline(v = 1, lwd = 1.5, lty = 3, col = PAL["grey"])
text(exp(-0.45), k + 0.42, "truth", col = PAL["red"], cex = 0.75, pos = 4)
```

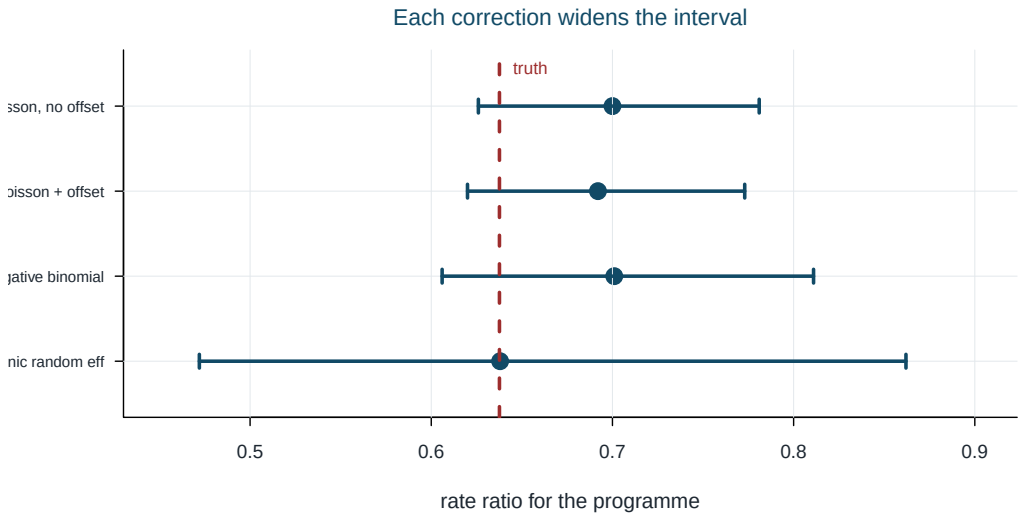


Figure 18.3: The programme effect under four models. The truth is the dashed line. Each fix widens the interval; only the last model both centres on the truth and reports honest uncertainty.

What the table shows

Model 1 is wrong and confident. Model 2 fixes the point estimate but is still overconfident. Model 3 widens the interval by accounting for patient heterogeneity. Model 4 widens it again by accounting for clinic heterogeneity — and its interval is roughly 2.5 times as wide as model 2's. That extra width is not a loss. It is the honest amount of uncertainty in a study where treatment was assigned to 40 clinics, not to 1000 patients.

18.9 Step 8: check the final model

```
set.seed(8)
mu_hat <- fitted(m3)
qr3 <- qres(mal$episodes, pnbinom, mu = mu_hat, size = sigma(m3))
par(mfrow = c(1, 2)); par_book()
qqnorm(qr3, pch = 21, bg = adjustcolor(PAL["green"], 0.4), col = "white", cex = 0.9,
       main = "Quantile residuals: final model")
qqline(qr3, lwd = 2.5, col = PAL["primary"]); grid_book()

sims <- simulate(m3, nsim = 300)
p0 <- sapply(sims, function(z) mean(z == 0))
hist(p0, breaks = 25, col = adjustcolor(PAL["primary"], 0.55), border = "white",
     xlab = "proportion of zeros", main = "Simulated vs observed zeros")
abline(v = mean(mal$episodes == 0), lwd = 3, col = PAL["red"])
```

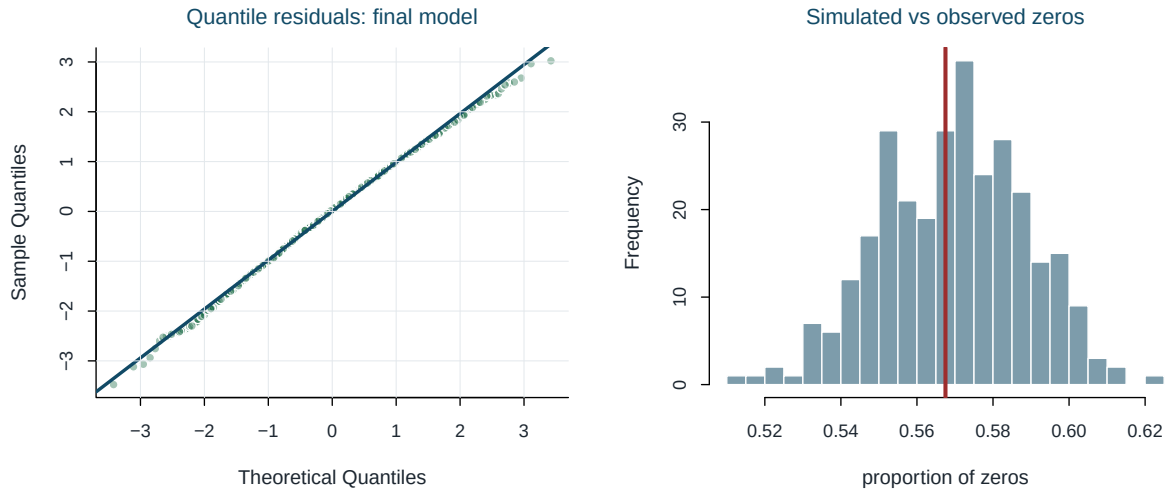


Figure 18.4: Diagnostics for the final model: quantile residuals lie on the normal line, and simulated datasets reproduce both the number of zeros and the largest observed count.

```
round(c(sd_qres = sd(qr3),
        observed_zeros = mean(mal$episodes == 0),
        simulated_zeros_mean = mean(p0),
        simulated_p = mean(p0 >= mean(mal$episodes == 0))), 4)

#>              sd_qres       observed_zeros simulated_zeros_mean          simulated_p
#>                0.983                0.568                0.570                0.573
```

Quantile residuals have standard deviation 0.98, sit on the normal line, and the model reproduces the zeros. The model fits.

18.10 Step 9: report it

```
ci <- confint(m3, parm = "beta_")
report <- data.frame(
  IRR   = round(exp(ci[, "Estimate"]), 3),
  lower = round(exp(ci[, 1]), 3),
  upper = round(exp(ci[, 2]), 3)
)
rownames(report) <- rownames(ci)
report


#>                     IRR lower upper
#> (Intercept)       1.346 1.056 1.716
#> treatedprogramme 0.638 0.472 0.862
#> age               0.987 0.983 0.992
#> bednetyes         0.668 0.575 0.776
```

The paragraph you would write

> "We modelled the number of malaria episodes per patient with a negative binomial mixed model, using log(follow-up years) as an offset and a random intercept for clinic. Patients in programme clinics had an incidence rate ratio of 0.64 (95% CI 0.47–0.86) compared with control clinics, adjusting for age and bednet use. Bednet use was associated with an IRR of 0.67 (95% CI 0.58–0.78). Counts were strongly overdispersed under a Poisson model (Pearson dispersion 1.7); the fitted dispersion parameter was $\hat{\theta} = 1.87$ and the between-clinic standard deviation was 0.43 on the log scale."

18.11 The workflow as a checklist

1. **Look at the data.** Distribution of the outcome, proportion of zeros, variance vs mean, exposure, cluster sizes.
2. **Identify the structure.** Is there exposure (offset)? Are rows clustered? Are predictors at the individual or the group level?
3. **Fit the Poisson with the offset.** This is the reference point.
4. **Check the mean model.** Quantile residuals against every predictor; test for curvature before blaming the distribution.
5. **Measure dispersion.** Pearson X^2/df, `dispersiontest`, predicted vs observed zeros.
6. **Fix what is broken.** Negative binomial for overdispersion; random effects for clustering; zero-inflation only if zeros are still short.
7. **Check the final model.** Quantile residual Q–Q, rootogram, simulation-based checks, influence.
8. **Report on the response scale.** Rate ratios with confidence intervals, what you checked, what you found, what you did about it.

CHAPTER 19

Cheat sheets

19.1 Which model do I fit?

Situation	R call
Yes/no outcome	`glm(y ~ x, binomial)`
k of n successes	`glm(cbind(k, n-k) ~ x, binomial)`
Proportions, overdispersed	`glm(..., quasibinomial)` or `glmmTMB(..., betabinomial)`
Count, dispersion ≈ 1	`glm(y ~ x, poisson)`
Count, unequal exposure	`glm(y ~ x + offset(log(t)), poisson)`
Count, dispersion > 1	`MASS::glm.nb(y ~ x)`
Count, dispersion > 1, want Poisson estimates	`glm(..., quasipoisson)`
Count, dispersion < 1	`glm(..., quasipoisson)` or `glmmTMB(..., compois)`
Count, extra zeros	`pscl::zeroinfl(y ~ x | z)`
Count, two-stage process	`pscl::hurdle(y ~ x | z)`
Count, zeros impossible	`pscl::hurdle(y ~ x | 1)`
Clustered data	`lme4::glmer(y ~ x + (1|g), family)`
Clustered + overdispersed count	`glmmTMB(y ~ x + (1|g), nbinom2)`
Clustered + zero-inflated	`glmmTMB(..., ziformula = ~ 1)`
Population-average effects	`geepack::geeglm(..., id = g)`
Non-linear predictor effects	`mgcv::gam(y ~ s(x), family)`

19.2 How do I interpret this coefficient?

Link	Transform	Sentence
identity	none	"... is associated with a change of β units in y."
logit	e^{β}	"... multiplies the *odds* of the outcome by e^{β}."
log	e^{β}	"... multiplies the expected *count* (or rate) by e^{β}."
log, small β	100β	"... changes the expected count by about 100β%."
probit	—	"... changes the underlying latent score by β SDs."
cloglog	e^{β}	"... multiplies the *hazard* of the event by e^{β}."

19.3 The essential diagnostic code

```
## ---- Dispersion ------------------------------------------------------
sum(residuals(m, type = "pearson")^2) / df.residual(m)      # want ~ 1
AER::dispersiontest(m, trafo = 2)                            # formal test

## ---- Zeros -----------------------------------------------------------
c(observed = sum(y == 0), expected = sum(dpois(0, fitted(m))))

## ---- Randomised quantile residuals -----------------------------------
```

```r
qres <- function(y, pfun, ...) {
  a <- pfun(y - 1, ...); b <- pfun(y, ...)
  qnorm(pmin(pmax(runif(length(y), pmin(a,b), pmax(a,b)), 1e-10), 1 - 1e-10))
}
r <- qres(y, ppois, lambda = fitted(m))         # or pnbinom(mu=, size=)
qqnorm(r); qqline(r)
plot(fitted(m), r); abline(h = 0)

## ---- Simulation-based check ------------------------------------------
s <- simulate(m, nsim = 500)
mean(apply(s, 2, function(z) mean(z == 0)) >= mean(y == 0))

## ---- Influence -------------------------------------------------------
plot(cooks.distance(m), type = "h"); abline(h = 4 / nobs(m), lty = 2)

## ---- Mixed models ----------------------------------------------------
lme4::isSingular(m)               # TRUE = simplify the random structure
lme4::allFit(m)                   # do all optimisers agree?
performance::check_model(m)       # if you have the performance package
```

19.4 Model comparison quick reference

Comparison	Tool	Notes
Nested, same family	`anova(m1, m2, test="LRT")`	The default choice
Nested, quasi-family	`anova(m1, m2, test="F")`	No likelihood available
Poisson vs NB	`pscl::odTest`	Halves the p-value (boundary parameter)
Non-nested	AIC / BIC	Only within the same response scale
ZI vs hurdle	`pscl::vuong`	Legitimate here; not for ZIP vs Poisson
Fixed effects in a GLMM	`anova` with `REML=FALSE`	Must refit with ML
Random effects in a GLMM	`anova(m1, m2)`	p-value is conservative (boundary)
Predictive accuracy	cross-validation	Score with log-likelihood, not just MAE

19.5 Ten mistakes to avoid

1. **Reporting raw coefficients from a log or logit model.** Exponentiate them.
2. **Forgetting `type = "response"` in `predict()`.** You get the link scale and will not notice until the numbers look strange.
3. **Forgetting the offset** when exposure varies. This can reverse the direction of an effect (Chapter 6).
4. **Ignoring overdispersion.** Your p-values will be several times too small (Chapter 7).
5. **Calling an odds ratio a risk ratio.** They only agree for rare outcomes.
6. **Reaching for zero-inflation because there are lots of zeros.** A Poisson with mean 0.5 is 61% zeros. Compare observed with *predicted* zeros.
7. **Ignoring clustering.** Repeated measures need random effects or cluster-robust standard errors.

8. **Comparing AIC across different response scales** — e.g. a model for `y` against a model for `log(y)`.
9. **Trusting a stepwise-selected model's p-values.** They are not valid after selection.
10. **Blaming the distribution when the mean model is wrong.** Check for missing predictors and curvature before switching families.

19.6 Glossary

Term	Meaning
AIC	$-2\ell + 2k$. Lower is better; compares non-nested models on the same data.
BLUP	Best linear unbiased predictor — the estimated random effect for a group.
Canonical link	The link that makes the maths cleanest: logit for binomial, log for Poisson.
Deviance	GLM analogue of the residual sum of squares.
Dispersion (ϕ)	Ratio of actual variance to what the family assumes.
GEE	Generalised estimating equations: marginal models with robust standard errors.
GLM	Generalised linear model: family + linear predictor + link.
GLMM	GLM with random effects.
ICC	Proportion of variance that is between groups.
IRR	Incidence rate ratio, e^{β} from a log link.
Link function	Maps the mean onto the unrestricted linear-predictor scale.
Linear predictor (η)	$\beta_0 + \beta_1 x_1 + \ldots$
Marginal effect	Population-average effect, averaged over random effects.
Conditional effect	Effect within a given cluster; what a GLMM reports.
NB1 / NB2	Negative binomial with variance linear ($\mu(1+\alpha)$) / quadratic ($\mu + \mu^2/\theta$) in the mean.
Odds	$p/(1-p)$.
Odds ratio	Ratio of two odds; e^{β} from a logit link.
Offset	A predictor with its coefficient fixed at 1; used for exposure.
OLRE	Observation-level random effect; absorbs overdispersion in a GLMM.
Overdispersion	Variance exceeds what the assumed family allows.
Pearson residual	$(y - \hat{\mu})/\sqrt{V(\hat{\mu})}$.
Quantile residual	Simulation-based residual that is normal when the model is right.
Quasi-likelihood	Specifies only mean and variance, not a full distribution.
REML	Restricted maximum likelihood; less biased variance components.
Rootogram	Plot of observed vs expected count frequencies on a square-root scale.
Sandwich estimator	Variance estimate robust to a misspecified variance function.
Separation	A predictor perfectly predicts the outcome; estimates diverge.
Shrinkage	Pulling group estimates towards the overall mean.
Theta (θ)	Negative binomial dispersion parameter; large $\theta \to$ Poisson.
Zero-inflation	Extra zeros from a separate "structural zero" process.

19.7 Where to go next

Topic	Suggested reading
GLMs in general	Dobson & Barnett, *An Introduction to Generalized Linear Models*
Applied logistic regression	Hosmer, Lemeshow & Sturdivant, *Applied Logistic Regression*
Count data	Cameron & Trivedi, *Regression Analysis of Count Data*
Count models in R	Zeileis, Kleiber & Jackman, "Regression Models for Count Data in R", *Journal of Statistical Software* 27(8)
Mixed models	Gelman & Hill, *Data Analysis Using Regression and Multilevel/Hierarchical Models*
Mixed models in R	Bates et al., "Fitting Linear Mixed-Effects Models Using lme4", *JSS* 67(1)
GLMMs in ecology	Bolker et al. (2009), *Trends in Ecology & Evolution* 24(3)
glmmTMB	Brooks et al. (2017), *The R Journal* 9(2)
Quantile residuals	Dunn & Smyth (1996), *Journal of Computational and Graphical Statistics* 5(3)
Rootograms	Kleiber & Zeileis (2016), *The American Statistician* 70(3)
Bayesian versions of everything here	McElreath, *Statistical Rethinking*; the brms package

The last word

> Every model in this book is a claim about how your data were generated. The arithmetic will run whatever you claim. The whole craft lies in checking whether the claim is defensible — which is why the diagnostic chapters matter more than the fitting ones. Fit the simple model, look hard at how it fails, and let the failure tell you what to fit next.